An22 · เอกสารสำหรับ System Analyst

เอกสารฐานข้อมูล

ER diagram · data dictionary · คำอธิบายตาราง · ขั้นตอนธุรกิจ → ตาราง · CRUD matrix · ความปลอดภัย

ฐานข้อมูล	PostgreSQL 18.6 · schema `public` · 44 ตาราง
Migration	Flyway V1–V10 (`an22-api/db/migration`)
สร้างเมื่อ	2026-11-14 15:12 จาก catalog ของ DB ที่ migrate จริง (`docs/db-manual/build.sh`)

สารบัญ

1 ภาพรวมฐานข้อมูล

สถาปัตยกรรม · หลายบริษัทใน DB เดียว · กลุ่มตาราง · ข้อตกลงของ schema · วิธีอ่านเอกสาร

2 ER diagram

ภาพรวม · กลุ่ม A แพลตฟอร์ม (SaaS) · กลุ่ม B บริษัท สาขา ผู้ใช้ · กลุ่ม C สินค้าและข้อมูลหลัก · กลุ่ม D ราคาและโปรโมชั่น · กลุ่ม E สต็อกและการโอน · กลุ่ม F กะขายและ POS · กลุ่ม G เอกสารขาย

3 คำอธิบายตารางและ data dictionary

A แพลตฟอร์ม (SaaS) · B บริษัท สาขา ผู้ใช้ · C สินค้าและข้อมูลหลัก · D ราคาและโปรโมชั่น · E สต็อกและการโอน · F กะขายและ POS · G เอกสารขาย

4 ขั้นตอนธุรกิจ → ตาราง

สมัครใช้งาน (บริษัทใหม่) · เข้าสู่ระบบ และจำกัด login ผิด · รับสินค้าเข้าคลัง · ตัดสต็อกออก (admin) · โอนสินค้าระหว่างสาขา · เปิดกะ · ขาย 1 บิล · ยกเลิกบิล (void วันเดียวกัน) · ใบกำกับภาษีเต็มรูป (ออกทีหลัง / ยกเลิกแล้วออกใหม่) · ใบลดหนี้ (คืนสินค้าตั้งแต่วันถัดไป) · ใบเปลี่ยนสินค้า · ปิดกะ อนุมัติ และปรับปรุงยอด · พักบิล / เรียกบิลคืน · ค่าบริการ SaaS (job รายวัน) · ขอลบบริษัท และ purge · รูปแบบเลขเอกสาร

5 CRUD matrix

6 ความปลอดภัยของฐานข้อมูล

role · Row-Level Security · สิทธิ์ต่อตาราง · ข้อมูลส่วนบุคคลและความลับ

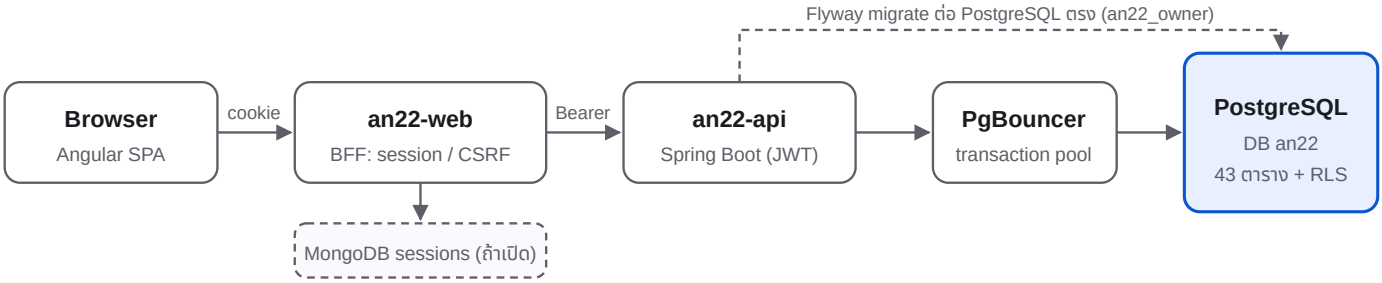
ภาคผนวก

ก MongoDB session ของ an22-web · ข ประวัติ migration · ค การสร้างเอกสารนี้ใหม่ · ง คำศัพท์

บทที่ 1 ภาพรวมฐานข้อมูล

ระบบหน้าร้านค้าปลีกไอที / เครื่องใช้ไฟฟ้า (สินค้ามี serial, ประกัน, ราคารวม VAT) แบบ SaaS — หลายบริษัท แต่ละบริษัทมีได้หลายสาขา

1.1 สถาปัตยกรรม



เส้นทางของข้อมูล — browser ไม่เคยถือ token; api ต่อ DB ผ่าน PgBouncer เสมอ (role an22_app / an22_platform)

- **an22-web** (Node.js) เสิร์ฟหน้าเว็บ ถือ session ของ login (cookie httpOnly) และ proxy `/api` ไป api — เก็บ session ในหน่วยความจำหรือ MongoDB (ภาคผนวก ก) **ไม่แตะ PostgreSQL**
- **an22-api** (Java Spring Boot) เป็นทางเดียวที่อ่าน / เขียน PostgreSQL และ**เป็นเจ้าของ schema**: Flyway migration `db/migration/V<n>__*.sql` (SQL ล้วน ห้ามแก้ไฟล์ที่ apply แล้ว เพิ่ม version ใหม่แทน)
- กฎธุรกิจ (คิดราคา โปร ทุน การชำระ ฯลฯ) อยู่ในโค้ด `rules/` ของ api — DB บังคับเฉพาะความถูกต้องพื้นฐาน (FK, unique, CHECK, ห้ามช่วงราคาทับ) เป็นตาข่ายชั้นสุดท้าย
- production: Azure Database for PostgreSQL Flexible Server + PgBouncer ในตัว; ทุก query อยู่ใน transaction (transaction pooling)

1.2 หลายบริษัทใน DB เดียว (multi-tenant)

- ทุกบริษัทอยู่ใน database เดียว schema เดียว — **ทุกตารางของร้านมีคอลัมน์** `tenant_id` (= `dt_companies.id`) และเปิด Row-Level Security: แต่ละ transaction เห็นเฉพาะแถวของบริษัทที่ api ตั้งไว้ (`app.tenant_id`) — รายละเอียดบทที่ 6
- บริษัทมาจาก token ของผู้ใช้เท่านั้น ไม่รับจาก URL / body; สาขาที่ทำงานมาจาก header `X-Branch-Id` ที่ api ตรวจสิทธิ์แล้ว
- ข้อมูลแบ่ง 2 ระดับ: **ระดับบริษัท** (มีแค่ `tenant_id`: SKU, ราคา, โปร, ผู้ใช้, ข้อมูลหลัก — ทุกสาขาใช้ร่วม) และ **ระดับสาขา** (มี `branch_id` ด้วย: สต็อก, serial, บัตรสต็อก, กะ, บิล, ใบกำกับ, ใบลดหนี้, ใบเปลี่ยน, เครื่อง POS)
- key ทุกตัวขึ้นต้นด้วย `tenant_id`: unique เช่น `(tenant_id, code)`, FK รวม `(tenant_id, x_id) → (tenant_id, id)` — จึงอ้างแถวของบริษัทอื่นไม่ได้แม้รู้ id
- ตาราง platform (กลุ่ม A) ไม่มี `tenant_id` (หรือใช้ `company_id`) และใช้ผ่าน role an22_platform

1.3 กลุ่มตาราง

กลุ่ม	ชื่อ	จำนวน	ตาราง
A	แพลตฟอร์ม (SaaS)	6	companies, platform_admins, billing_invoices, platform_doc_counters, password_reset_tokens, login_throttle
B	บริษัท สาขา ผู้ใช้	9	company_info, branches, users, user_branches, user_mfa, menu_access, pos_terminals, doc_counters, audit_log
C	สินค้าและข้อมูลหลัก	6	categories, suppliers, payment_methods, products, product_pack_units, product_suppliers
D	ราคาและโปรโมชั่น	2	sku_prices, promotions
E	สต็อกและการโอน	5	branch_stock, stock_movements, serial_numbers, transfers, transfer_lines

กลุ่ม	ชื่อ	จำนวน	ตาราง
F	กะขายและ POS	6	shifts, shift_counts, cash_movements, drawer_opens, shift_adjustments, held_bills
G	เอกสารขาย	10	sales, sale_lines, sale_payments, tax_invoices, credit_notes, credit_note_lines, credit_note_deductions, credit_note_refunds, exchanges, exchange_lines
	รวม	**44**	+ `flyway_schema_history` ของ Flyway (ประวัติ migration)

1.4 ข้อตกลงของ schema

เรื่อง	ข้อตกลง
ชื่อ	ตาราง `dt_` + snake_case (ชื่อใน JSON ของ API เป็น camelCase ไม่มี prefix เช่น `dt_sale_lines.unit_price` → `SaleLine.unitPrice`); constraint / index ขึ้นต้นด้วยชื่อตาราง
คอลัมน์ stamp	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — trigger `set_row_stamp()` ตั้งให้ทุกครั้ง (แอปลืมใส่ไม่ได้, UPDATE ทับ created_* ไม่ได้); `*_by` = `dt_users.id` เมื่อ kind = `user`, `dt_platform_admins.id` เมื่อ `platform`, null เมื่อ `system` (job / seed); ไม่ส่งออก API เว้นบางตารางที่ใช้เป็นเวลาของเอกสาร (บอกไว้ในคอลัมน์)
append-only	ตารางประวัติ / บรรทัดของเอกสาร (บัตรสต็อก, audit, บรรทัดบิล, การชำระ, ใบลดหนี้, ใบเปลี่ยน, ยอดนับกะ ฯลฯ) มีแค่ `created_*` (trigger แบบ `append_only`) และ role ของร้านไม่มีสิทธิ์ UPDATE / DELETE
enum	`text` + `CHECK (x IN (…))` ค่าตรงกับ type union ใน TypeScript — data dictionary แสดงค่าที่ใช้ได้ใต้คำอธิบาย
เงิน / ทุน	เงิน มูลค่า COGS `numeric(14,2)` (ปัด 2 ตำแหน่ง); ทุนต่อหน่วยและทุนเฉลี่ย `numeric(14,4)` (ปัด 4); ราคาขายรวม VAT, ทุนไม่รวม VAT; จำนวนสินค้าเป็น `integer` หน่วยฐาน
เวลา / วันที่	เวลาเป็น `timestamptz`; วันธุรกิจ (ช่วงราคา / โปร, ครบกำหนด) เป็น `date` = วันท้องถิ่น Asia/Bangkok รวมทั้งสองปลาย; `null` ปลายทาง = ไม่สิ้นสุด
ห้ามลบเอกสาร	บิล ใบกำกับ ใบลดหนี้ ใบเปลี่ยน ใบโอน กะ บัตรสต็อก serial — มีแต่เปลี่ยนสถานะ / เพิ่มรายการใหม่; master (SKU, ผู้จำหน่าย, ช่องทางชำระ, สาขา) ที่มีเอกสารอ้าง → ปิดใช้งานแทนลบ
snapshot	เอกสารเก็บค่า ณ ตอนออก (ชื่อ / รหัสสินค้า, ราคา, ชื่อช่องทาง, POS ID, ชื่อผู้ทำ, สถานะ VAT) ไม่ join master ตอนอ่าน
อ้างอิงไม่มี FK	ประวัติอ้าง master ที่ลบได้โดยไม่มี FK (เช่น `dt_sale_lines.product_id`) — ER แสดงเป็นเส้นประ, dictionary ติดป้าย ref
FK	ทุก FK เป็น `ON DELETE RESTRICT` (ไม่มี cascade) และมี index ฝั่งลูก
ค่าที่คำนวณ	ไม่เก็บค่าที่คำนวณได้ (ราคาวันนี้, สถานะคืนสินค้า, ยอดที่คาดของกะ …) — api คำนวณตอนอ่าน (บอกไว้ใน "ค่าที่ไม่ได้เก็บ" ของแต่ละตาราง)
เลขเอกสาร	ตัวนับใน `dt_doc_counters` ต่อ prefix (ไม่ใช้ SEQUENCE) — ออกหลังตรวจกฎผ่านใน transaction เดียวกับเอกสาร เลขต่อเนื่องไม่ข้าม (รูปแบบในหัวข้อ 4.*n* "รูปแบบเลขเอกสาร")

1.5 วิธีอ่านเอกสาร

- บทที่ 2 ER diagram: ภาพรวมก่อน แล้วภาพละกลุ่มพร้อมคอลัมน์ (ใน PDF กดชื่อตารางเพื่อไปบทที่ 3)
- บทที่ 3 ทุกตาราง: ใช้ทำอะไร → วงจรชีวิตของแถว → กฎ → รายคอลัมน์ → constraint / index / สิทธิ์ ที่อ่านจาก DB จริง
- บทที่ 4 งานแต่ละอย่างเขียน / อ่านตารางไหนตามลำดับ · บทที่ 5 package ของ api ใช้ตารางไหนแบบไหน · บทที่ 6 role / RLS / ข้อมูลที่เข้ารหัส
- รหัสตัวเล็กท้ายข้อความ เช่น E12 D3 = รหัสการตัดสินใจใน `FrontEnd-An22/docs/decisions.md` (อ่านประกอบได้ ไม่จำเป็น)
- โครงสร้าง (ชนิด / null / ค่าเริ่ม / key / constraint) สร้างจาก DB ทุกครั้งที่ build จึงตรงกับ migration เสมอ; คำอธิบายเขียนใน `docs/db-manual/descriptions.mjs`

บทที่ 2 ER diagram

ภาพสร้างจาก catalog ของ DB ที่ migrate ถึง V10 — ใน PDF กดชื่อตารางในภาพเพื่อไปหน้าคำอธิบายได้

สัญลักษณ์



FK แบบบังคับ: ฝั่งตีนกา (ลูก) มีได้ 0..n แถวต่อ 1 แถวของฝั่งขีดคู่ (แม่ — คอลัมน์ FK เป็น NOT NULL)



FK ที่ว่างได้ (คอลัมน์ nullable): แถวลูกอาจไม่อ้างแม่

อ้างอิงแบบไม่มี FK (ประวัติ / array / ของชั่วคราว) — ป้าย ref

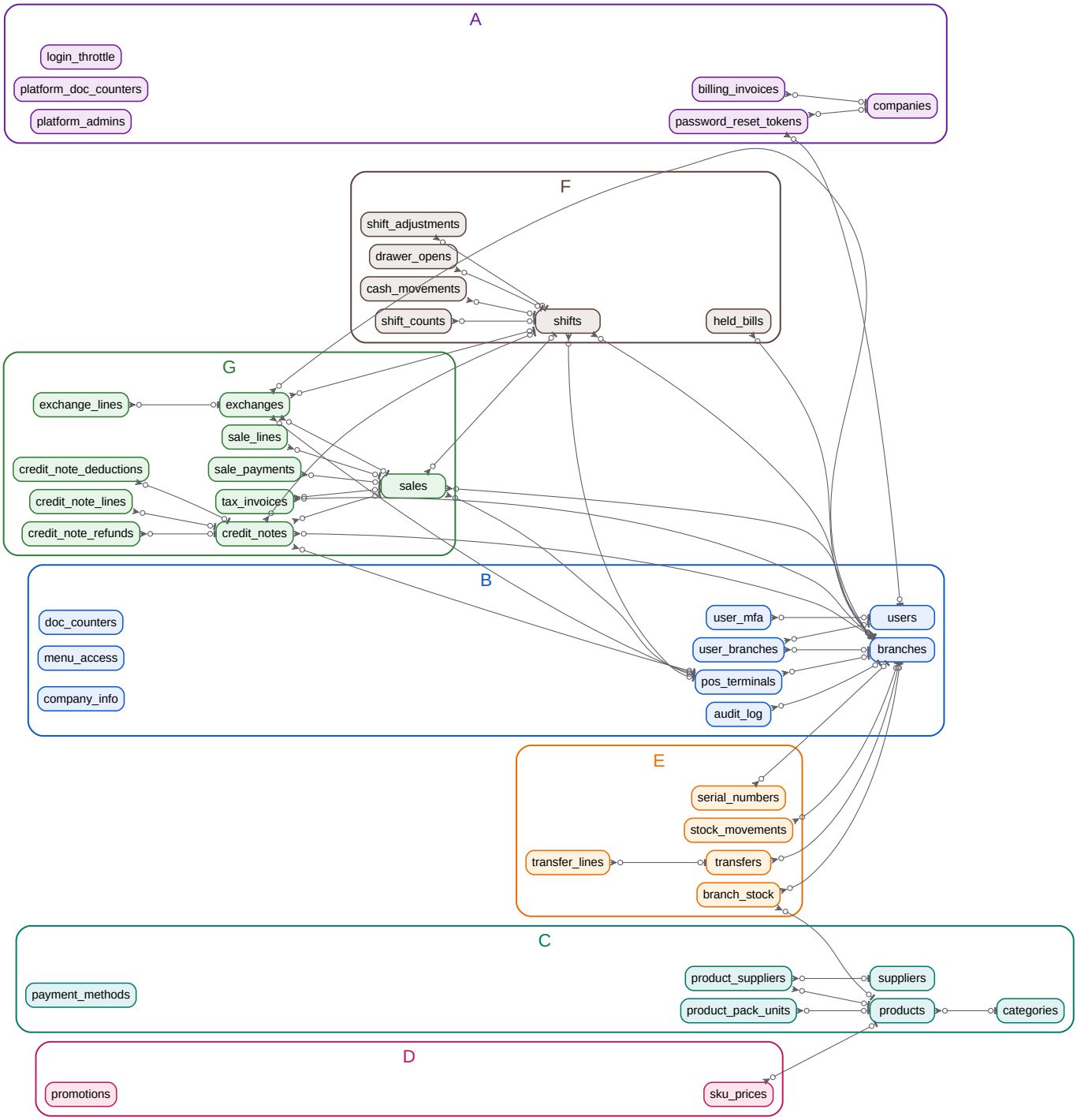
PK · FK · UQ · `?`

ในกรอบตาราง: ชื่อตัวหนา = PK, **FK** = มี FK, **UQ** = อยู่ใน unique, `?` = ว่างได้ (nullable); บรรทัด "+ n stamp columns" = คอลัมน์ stamp ที่ไม่แสดง

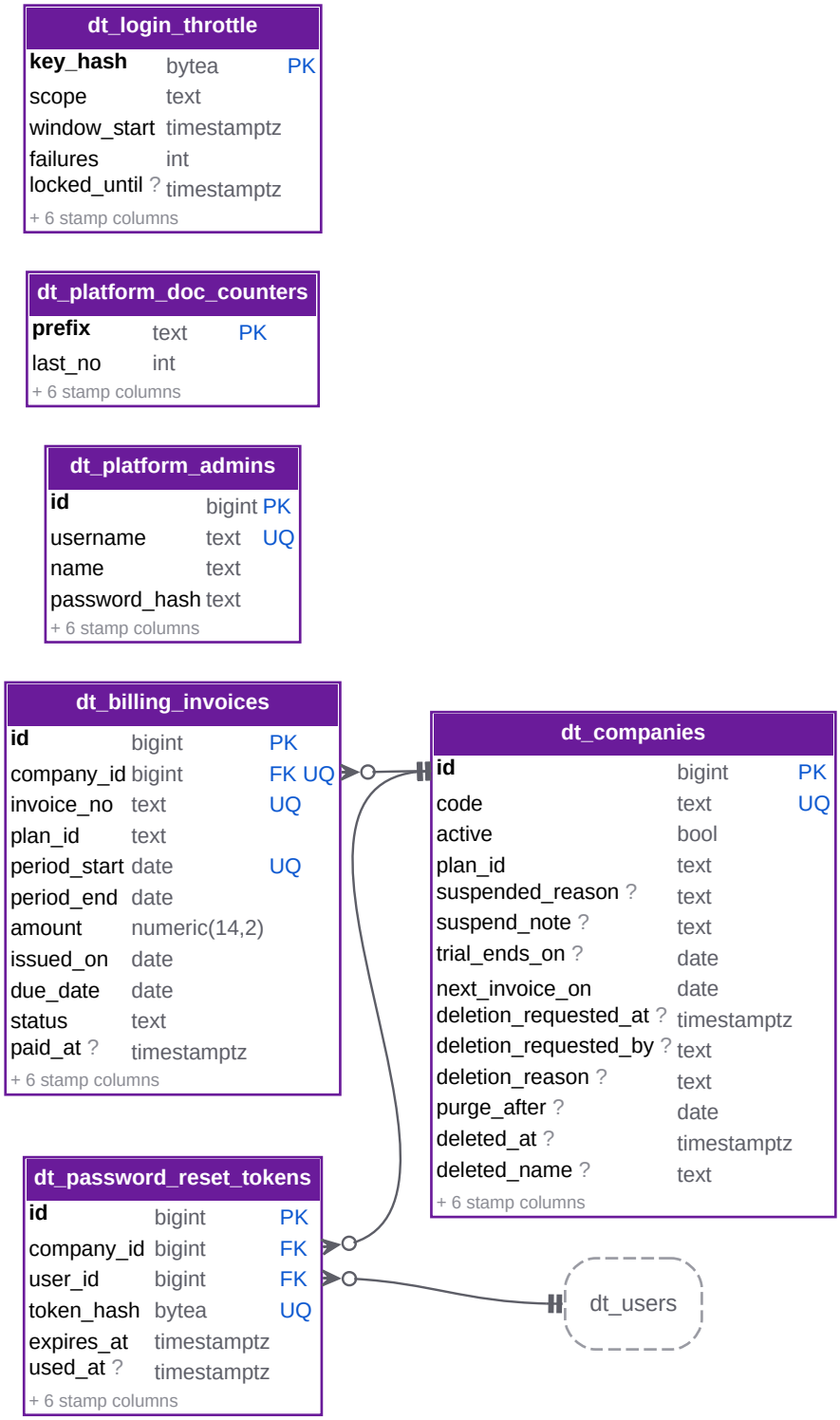


กรอบเส้นประ = ตารางของกลุ่มอื่นที่ถูกอ้าง (ดูคอลัมน์ในภาพของกลุ่มนั้น)

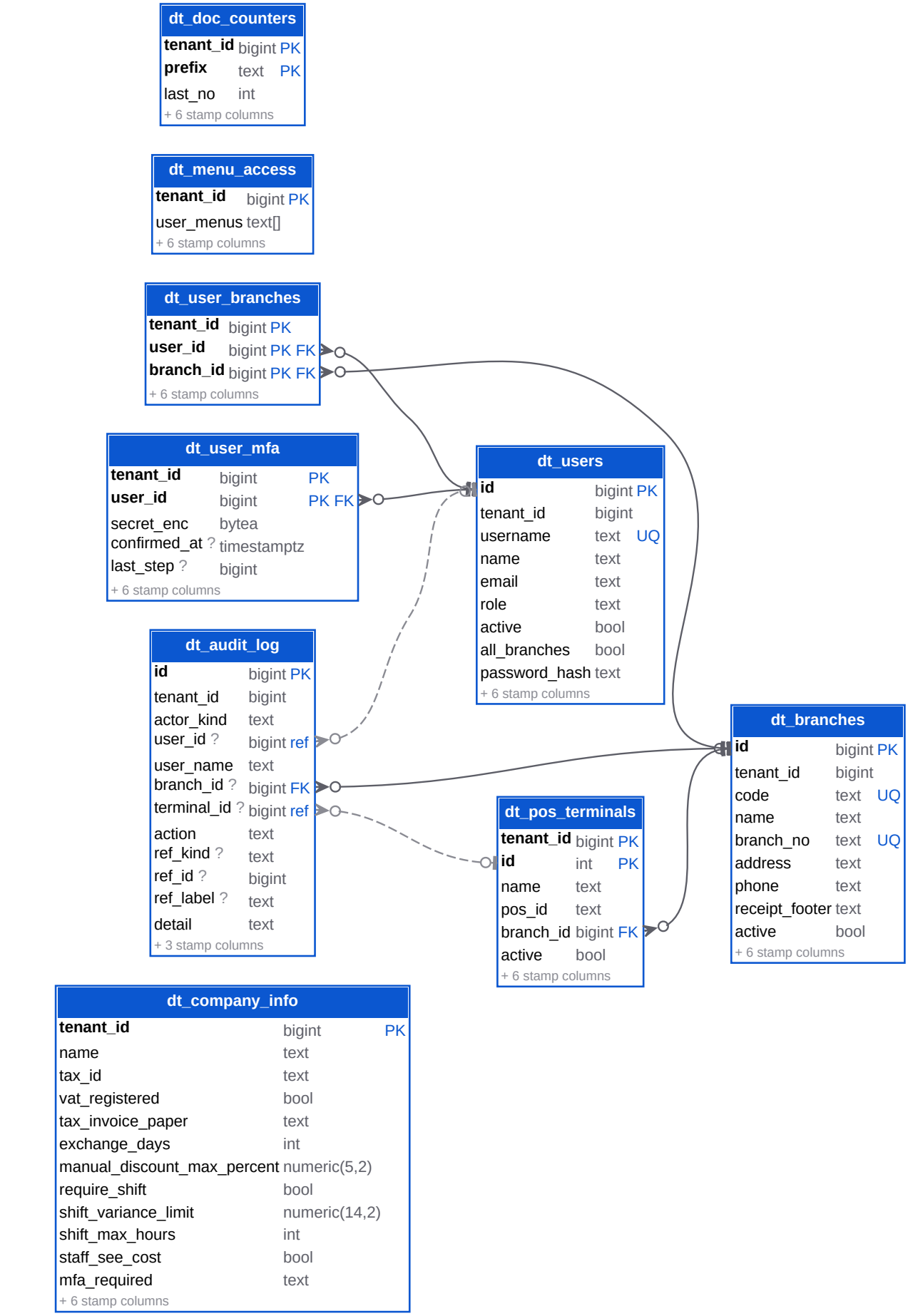
ไม่วาดเส้น `tenant_id → dt_companies` ที่ทุกตารางของร้านมี (ภาพจะอ่านไม่ออก) — FK รวม `(tenant_id, x_id)` วาดจากคอลัมน์ `x_id`



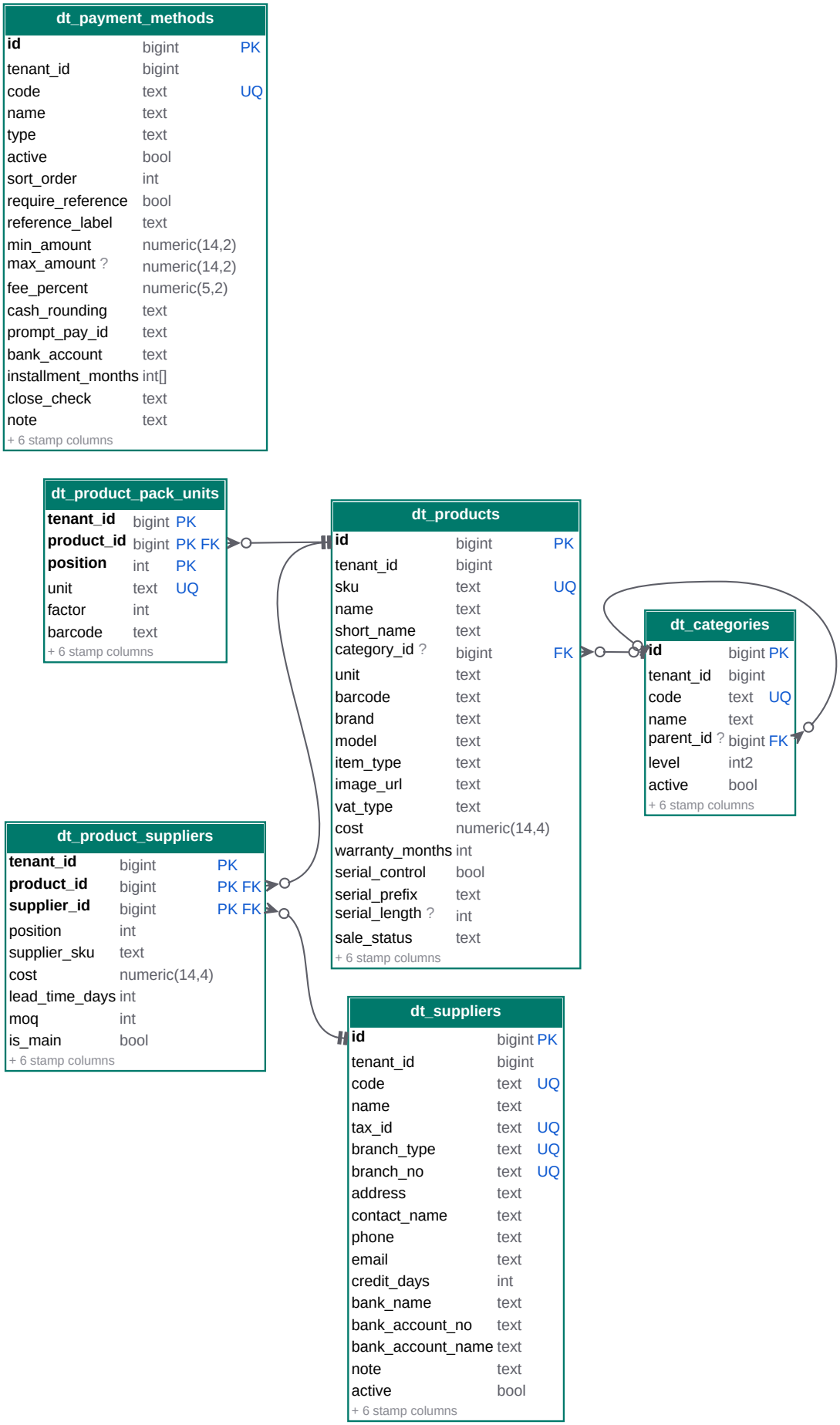
ภาพรวม 44 ตาราง — กรอบสี = กลุ่ม (A–G ตามหัวข้อ 1.3), เส้น = FK (ตีนกาอยู่ฝั่งลูก); ไม่วาดเส้น `tenant_id` → `dt_companies` ที่ทุกตารางมี และอ้างอิงแบบไม่มี FK (ดูในภาพของแต่ละกลุ่ม)



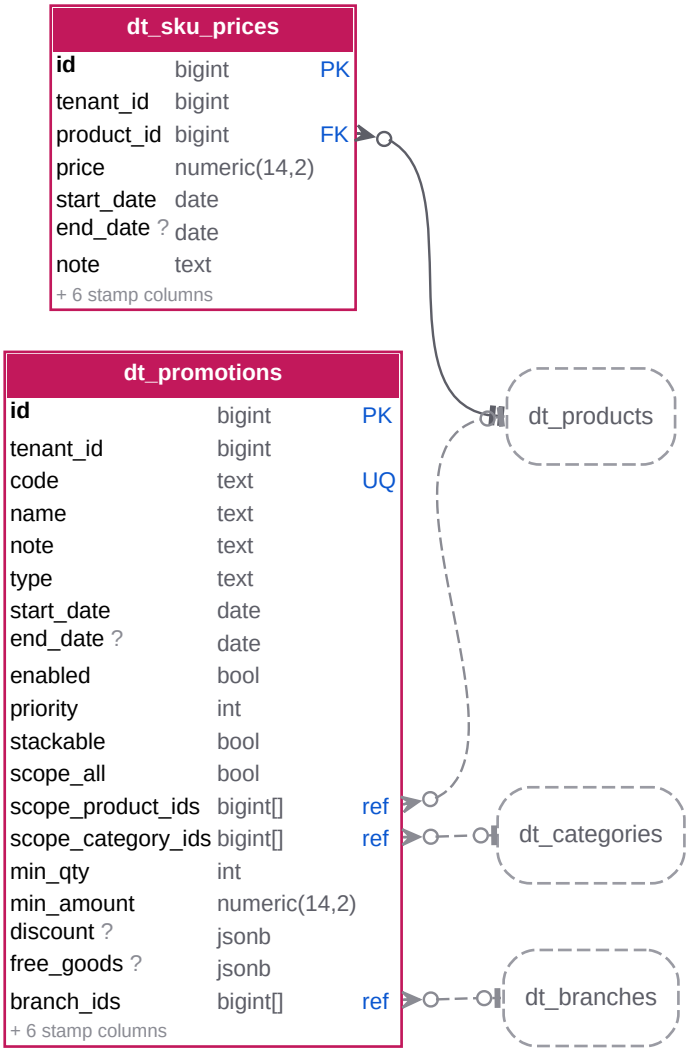
กลุ่ม A แพลตฟอร์ม (SaaS) — บริษัท (tenant) · ผู้ดูแลแพลตฟอร์ม · ใบแจ้งหนี้ค่าบริการ SaaS · ตัวนับเลขเอกสารของแพลตฟอร์ม · ลิงก์ตั้งรหัสผ่านใหม่ของ owner · ตัวนับ login ผิด



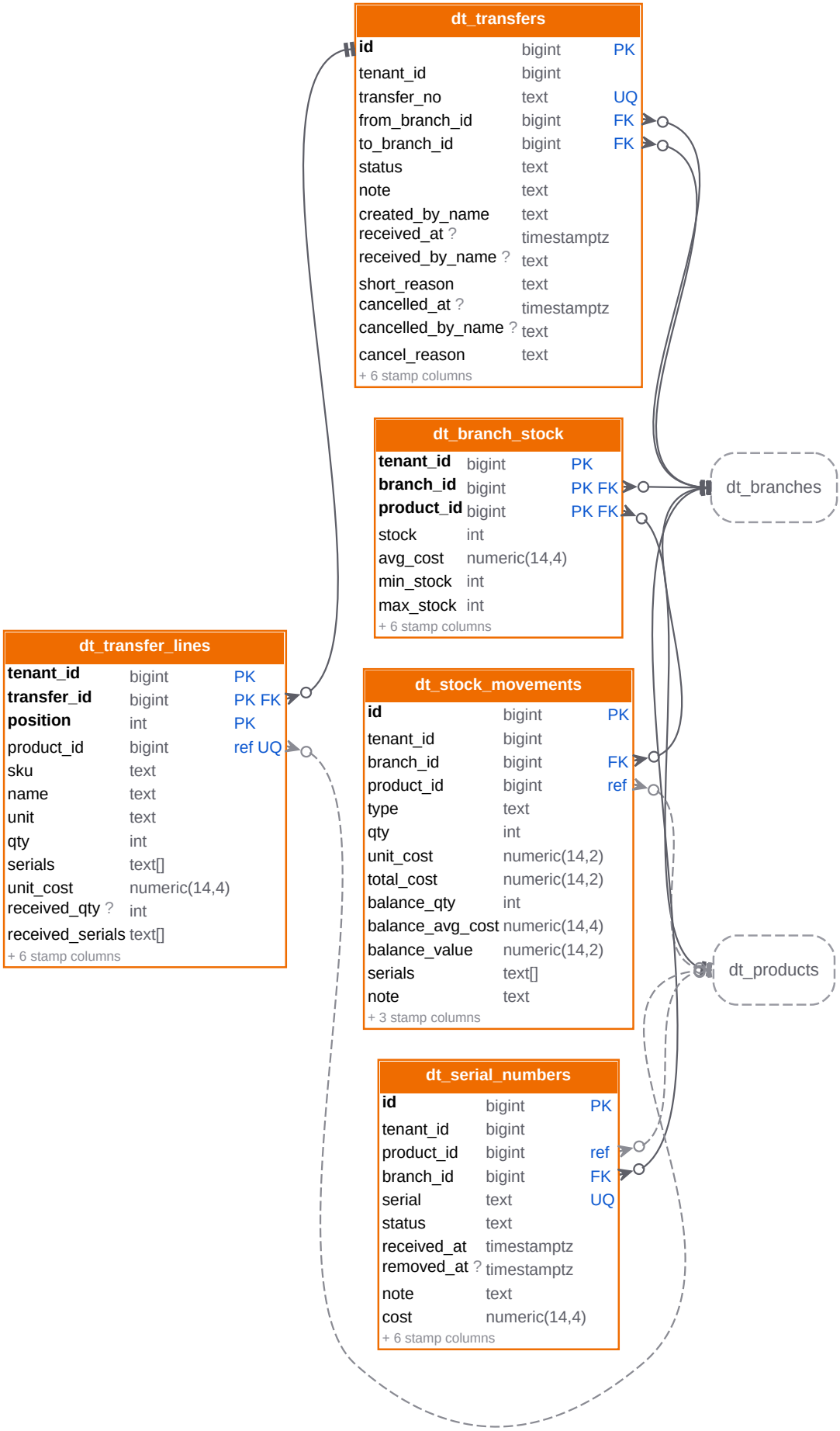
กลุ่ม B บริษัท สาขา ผู้ใช้ — ข้อมูลบริษัท + กติกาที่ทุกสาขาใช้ร่วม · สาขา · ผู้ใช้ของบริษัท · สาขาที่ผู้ใช้เข้าได้ · Microsoft Authenticator ของผู้ใช้ · เมนูที่พนักงาน (role user) เห็น · เครื่อง POS · ตัวนับเลขเอกสารของบริษัท · บันทึกการใช้งาน (audit log)



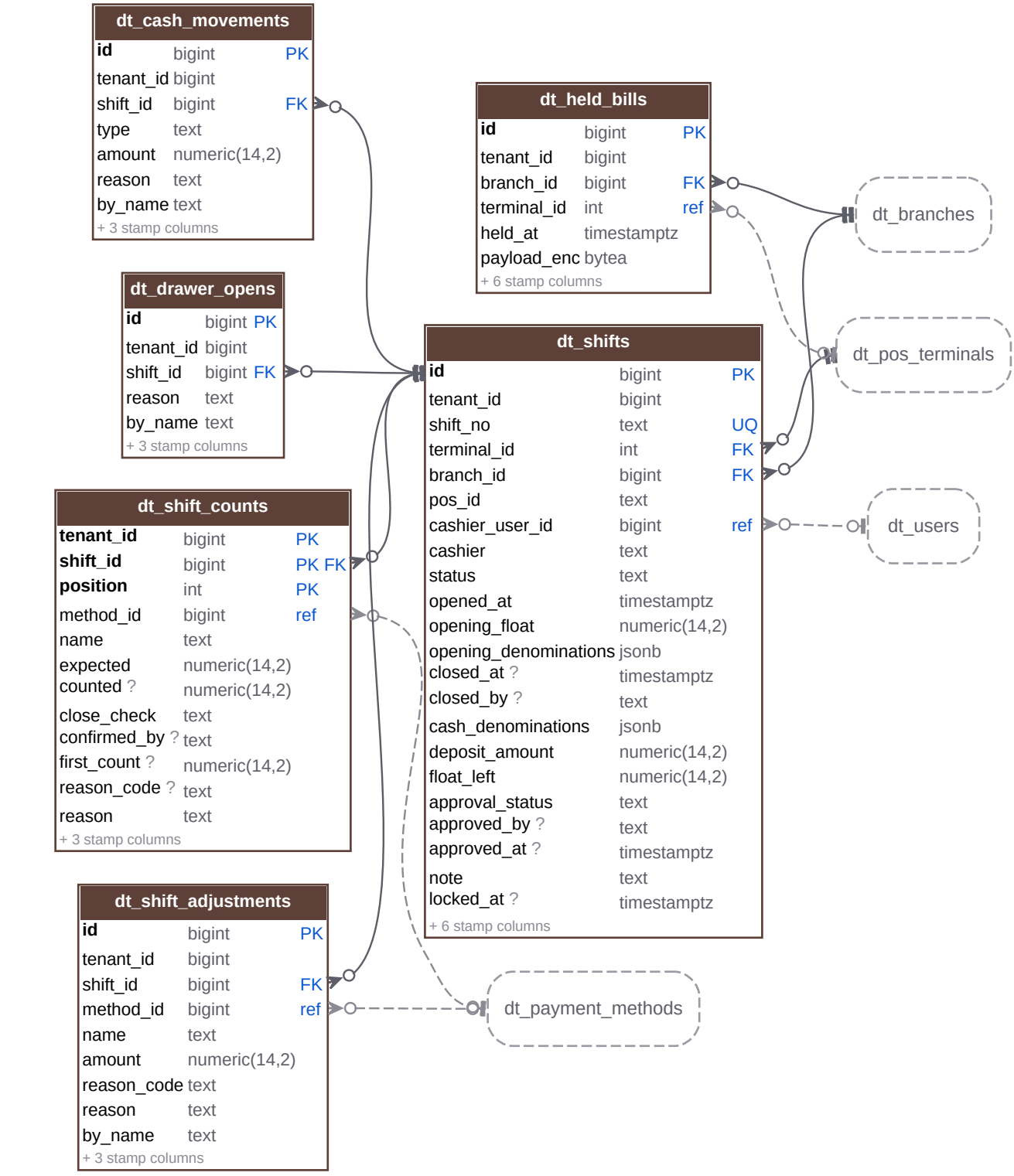
กลุ่ม C สินค้าและข้อมูลหลัก — หมวดหมู่สินค้า 3 ระดับ · ผู้จำหน่าย · ช่องทางชำระเงิน · สินค้า (SKU master) · หน่วยแพ็กของ SKU · ผู้จำหน่ายของ SKU



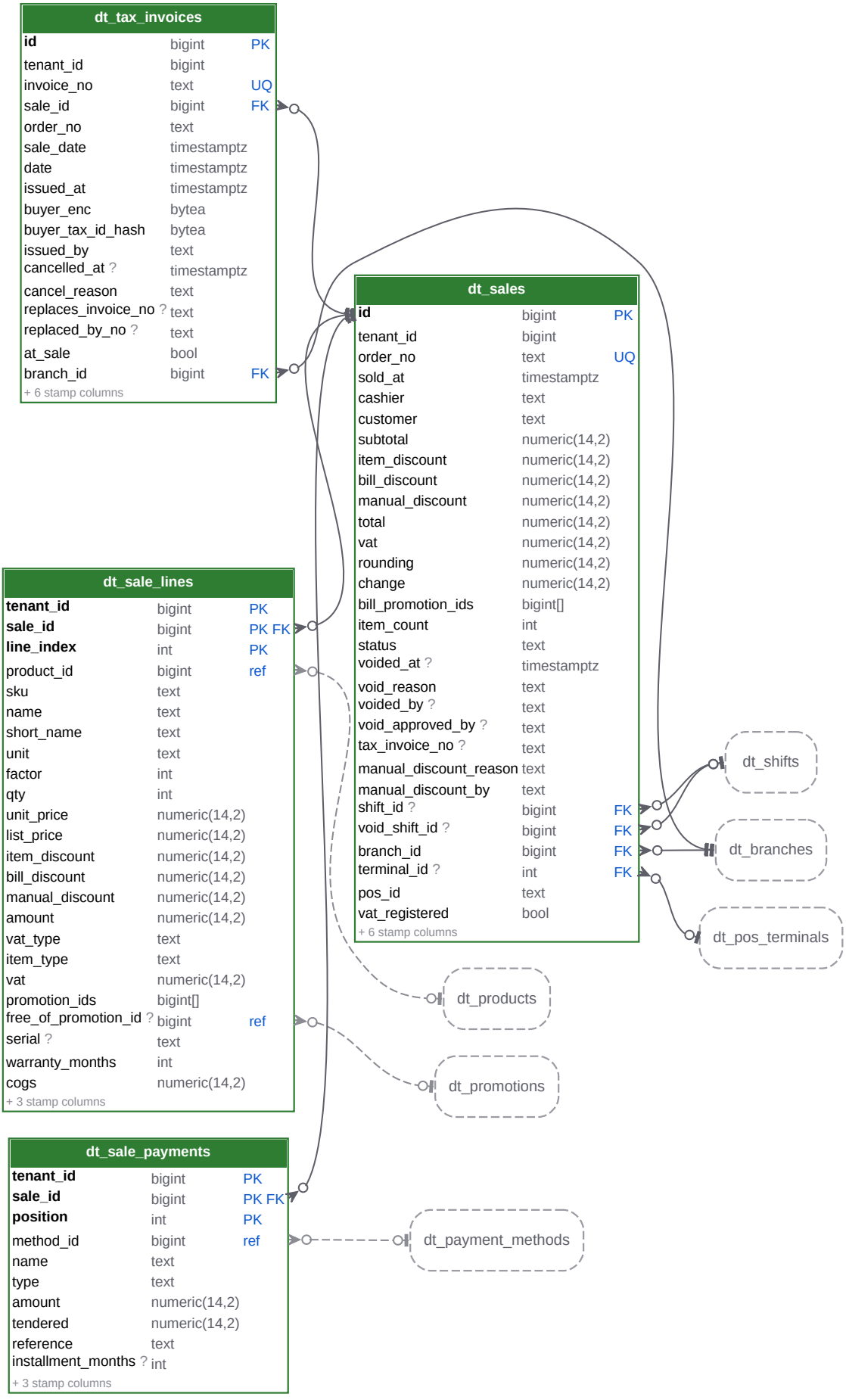
กลุ่ม D ราคาและโปรโมชั่น — ราคาขายตามช่วงวัน · โปรโมชั่น



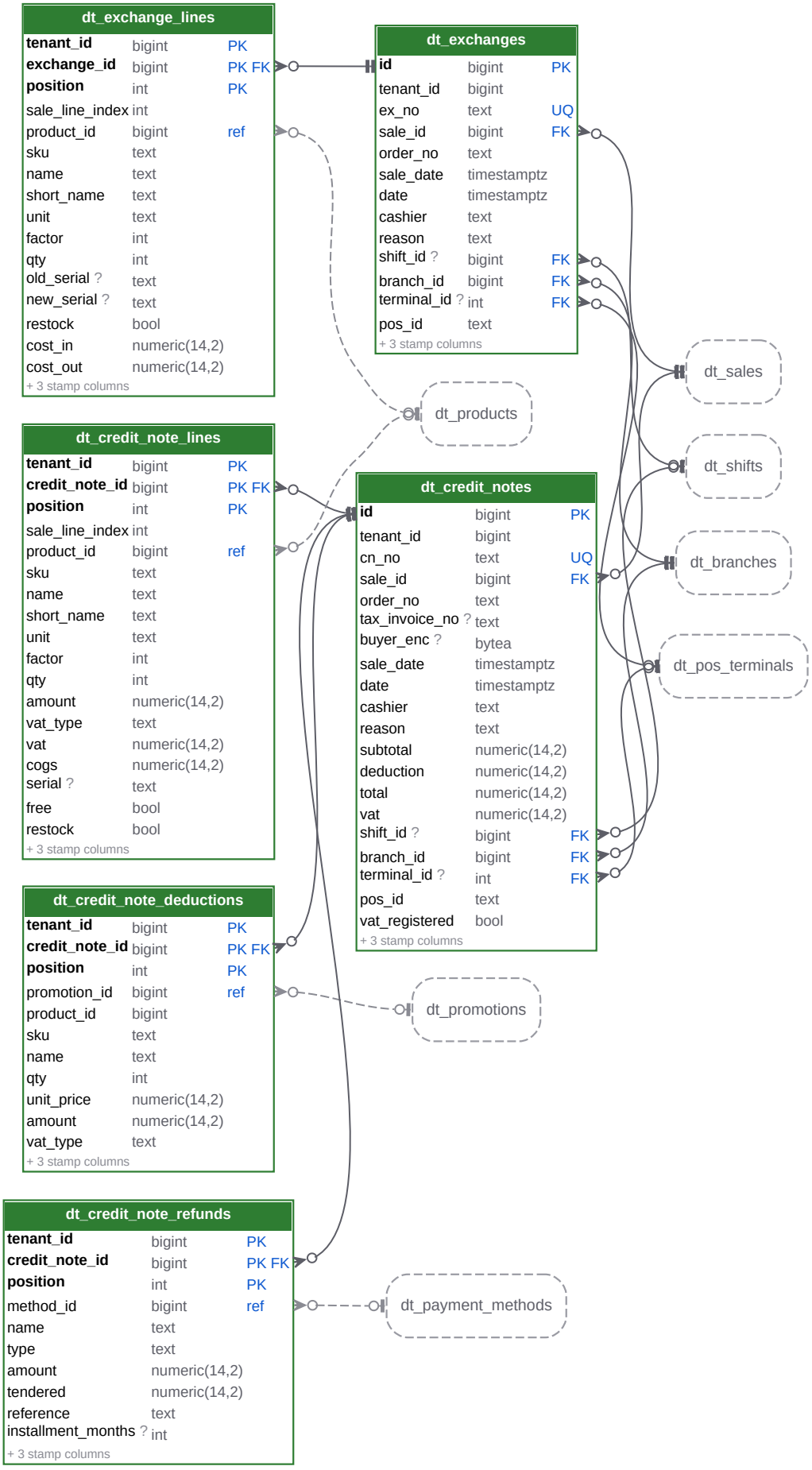
กลุ่ม E สต็อกและการโอน — สต็อกต่อสาขา · บัตรสต็อก (stock movement) · serial รายเครื่อง · ใบโอนสินค้าระหว่างสาขา · บรรทัดของใบโอน



กลุ่ม F กะขายและ POS — กะขาย · ยอดนับตอนปิดกะ · เงินเข้า / ออกลิ้นชักระหว่างกะ · เปิดลิ้นชักโดยไม่มีการขาย · ปรับปรุงยอดกะที่ปิดแล้ว · บิลที่พักไว้ของ POS



กลุ่ม G เอกสารขาย (1/2 บิลและใบกำกับ) — บิลขาย (POS) · บรรทัดบิล · การชำระของบิล · ใบกำกับภาษีเต็มรูป



กลุ่ม G เอกสารขาย (2/2 ใบลดหนี้และใบเปลี่ยน) — ใบลดหนี้ (คืนสินค้า) · บรรทัดใบลดหนี้ · ยอดหักของแถมที่เกินสิทธิ์ · การคืนเงินของใบลดหนี้ · ใบเปลี่ยนสินค้า · บรรทัดใบเปลี่ยน

บทที่ 3 คำอธิบายตารางและ data dictionary

เรียงตามกลุ่ม A–G — แต่ละตาราง: ใช้ทำอะไร, วงจรชีวิตของแถว, กฎสำคัญ, ค่าที่ไม่ได้เก็บ แล้วตารางคอลัมน์และ constraint / index / RLS / สิทธิ์ (สร้างจาก DB)

> **คอลัมน์ของตาราง data dictionary**
> **null** ✓ = ว่างได้ · **ค่าเริ่มต้น** = DEFAULT ของ DB (*identity* = เลขอัตโนมัติ) · **key** PK primary key FK foreign key (ปลายทางอยู่หลัง →) UQ อยู่ใน unique / unique index ref อ้างอิงแบบไม่มี FK (ปลายทางอยู่หลัง ⇢) · **สิทธิ์** S = SELECT, I = INSERT, U = UPDATE, D = DELETE

A แพลตฟอร์ม (SaaS)

ตารางของผู้ให้บริการ ไม่ใช่ข้อมูลของร้าน: ทะเบียนบริษัท (tenant), ผู้ดูแลแพลตฟอร์ม, ใบแจ้งหนี้ค่าบริการ, ลิงก์ตั้งรหัสผ่าน และตัวนับ login ผิด — ส่วนใหญ่ไม่มี `tenant_id` และใช้ผ่าน role `an22_platform` (BYPASSRLS) เท่านั้น; `dt_companies` / `dt_billing_invoices` เปิด RLS ให้ร้านอ่านแถวของตัวเองได้

A `dt_companies` บริษัท (tenant)

ทะเบียนบริษัทที่ใช้ระบบ 1 แถว = 1 ลูกค้าของ SaaS — `id` คือ tenant ที่ทุกตารางของร้านอ้างผ่าน `tenant_id`; เก็บสถานะการใช้งาน แพ็กเกจ รอบเก็บเงิน และคำขอลบ (ชื่อ / เลขภาษีของบริษัทอยู่ที่ `dt_company_info`)

วงจรชีวิตของแถว

- สร้างตอนสมัครใช้งาน (`POST /auth/signup`) พร้อม owner, ข้อมูลบริษัท, สาขาแรก, ช่องทางเงินสด
- แก้: เปลี่ยนแพ็กเกจ, job ค่าบริการ (เลื่อน `next_invoice_on`, ระงับเมื่อค้างชำระ), ผู้ดูแลแพลตฟอร์มระงับ / เปิดใช้, ขอลบ / ยกเลิกคำขอ
- **ไม่ลบ**: purge ลบข้อมูลทุกตารางของบริษัท แต่เหลือแถวนี้ไว้ (`deleted_at` + ชื่อตอนลบ) เพื่อให้รหัสบริษัทไม่ถูกใช้ซ้ำ

กฎสำคัญ

- `code` ไม่ซ้ำทั้งแพลตฟอร์ม และห้ามใช้ซ้ำแม้บริษัทเดิมถูกลบแล้ว E39
- `active = false` = ระงับ: login ไม่ได้, token เดิมได้ 401 — เหตุผลอยู่ใน `suspended_reason`
- RLS: role ของร้าน (an22_app) เห็นเฉพาะแถว `id = app_tenant_id()` และอ่านอย่างเดียว

ถูกอ้างโดย `dt_billing_invoices`, `dt_password_reset_tokens`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสบริษัท = tenant id ที่ทุกตารางของร้านอ้าง
2	**code**	text			UQ	รหัสบริษัทที่พิมพ์ตอน login (ตัวใหญ่ A-Z 0-9) — ไม่ซ้ำทั้งแพลตฟอร์ม ใช้ซ้ำไม่ได้
3	**active**	bool		true		ใช้งานอยู่ (false = ระงับ)
4	**plan_id**	text				แพ็กเกจปัจจุบัน — เพดานจำนวนสาขา / ผู้ใช้ / เครื่อง POS ที่ใช้งาน (ค่าของแต่ละแพ็กเกจอยู่ในโค้ด `PlanRules` ไม่ใช่ตาราง) E34 ค่าที่ใช้ได้: `starter` `standard` `pro`
5	**suspended_reason**	text	✓			เหตุที่ระงับ: `overdue` ค้างชำระเกินกำหนด · `platform` ผู้ดูแลแพลตฟอร์มระงับ · `deletion` ขอลบบริษัท; null ขณะใช้งาน ค่าที่ใช้ได้: `overdue` `platform` `deletion`
6	**suspend_note**	text	✓			หมายเหตุของผู้ดูแลแพลตฟอร์มตอนระงับ
7	**trial_ends_on**	date	✓			วันสุดท้ายของช่วงทดลองใช้ (null = ไม่มี / จบแล้ว)
8	**next_invoice_on**	date				วันที่จะออกใบแจ้งหนี้รอบถัดไป — job รายวันออกใบเมื่อถึงวัน แล้วเลื่อนไปเดือนถัดไป E35

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
9	**deletion_requested_at**	timestamptz	✓			เวลาที่ owner ขอลบบริษัท (null = ไม่มีคำขอ)
10	**deletion_requested_by**	text	✓			ชื่อผู้ขอลบ (snapshot)
11	**deletion_reason**	text	✓			เหตุผลที่ขอลบ
12	**purge_after**	date	✓			ลบถาวรได้ตั้งแต่วันนี้ (= วันขอ + ระยะรอ) — ก่อนถึงวันยกเลิกคำขอได้
13	**deleted_at**	timestamptz	✓			เวลาที่ purge แล้ว — บริษัทนี้ไม่มีข้อมูลเหลือ (login / token เหมือนไม่มีบริษัท)
14	**deleted_name**	text	✓			ชื่อบริษัท ณ ตอน purge (ข้อมูลอื่นถูกลบหมด)
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (code)
- **CHECK** `((code ~ '^[A-Z0-9]+$'::text))`
- **CHECK** `((plan_id = ANY (ARRAY['starter'::text, 'standard'::text, 'pro'::text])))`
- **CHECK** `((suspended_reason = ANY (ARRAY['overdue'::text, 'platform'::text, 'deletion'::text])))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(id = app_tenant_id())` · สิทธิ์ an22_app **S** · an22_platform **SIUD**

A dt_platform_admins ผู้ดูแลแพลตฟอร์ม

บัญชีของผู้ให้บริการ SaaS ที่ login หน้า `/platform` — ไม่ใช่ผู้ใช้ของบริษัทใด จัดการทะเบียนบริษัท ใบแจ้งหนี้ ระงับ / purge E36

วงจรชีวิตของแถว

- คนแรกสร้างด้วยมือตอน init DB (`docs/db-init.md` ขั้น 4) — ยังไม่มีหน้าจอเพิ่ม / ลบ
- แก้ / ลบ: ด้วยมือ (DBA)

กฎสำคัญ

- ไม่อยู่ใต้ RLS และ an22_app ไม่มีสิทธิ์ใด ๆ — ใช้ผ่าน an22_platform เท่านั้น
- `username` ไม่ซ้ำ รูปแบบ a-z 0-9 . _ - ยาว 3–30

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**username**	text			UQ	ชื่อ login ของหน้า /platform (ตัวเล็ก)
3	**name**	text				ชื่อที่แสดง / บันทึกเป็นผู้ทำใน audit ของบริษัท
4	**password_hash**	text				hash ของรหัสผ่าน (BCrypt) — ไม่เคยส่งออก API
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (username)
- **CHECK** `((username ~ '^[a-z0-9._-]{3,30}$'::text))`

trigger: stamp (BEFORE INSERT OR UPDATE) · ไม่อยู่ใต้ RLS (ตาราง platform) · สิทธิ์ an22_app **—** · an22_platform **SIUD**

A dt_billing_invoices ใบแจ้งหนี้ค่าบริการ SaaS

ใบแจ้งหนี้รายเดือนที่แพลตฟอร์มเรียกเก็บจากบริษัท — **ไม่ใช่เอกสารภาษีของร้าน** E35; ใช้แสดงแถบเตือนค้างชำระ และระงับบริษัทที่เกินกำหนด

วงจรชีวิตของแถว

- สร้างโดย job รายวัน (`BillingService.catchUp`, 00:05 และตอน login / me) เมื่อถึง `dt_companies.next_invoice_on` — รันซ้ำได้ผลเดิม (1 ใบต่อบริษัทต่อรอบ)

- แก้: ผู้ดูแลแพลตฟอร์มบันทึกรับชำระ (`open` → `paid`)
- ไม่ลบ — purge บริษัทก็ยังเก็บไว้

กฎสำคัญ

- `invoice_no` = `SUB-YYYYMM-NNNN` ไม่ซ้ำทั้งแพลตฟอร์ม (ตัวนับใน `dt_platform_doc_counters`)
- `paid_at` มีค่าเมื่อและเฉพาะเมื่อ `status = paid`
- ร้านอ่านใบของตัวเองได้ผ่าน RLS (`company_id = app_tenant_id()`)

อ้างถึง `dt_companies`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**company_id**	bigint			FK UQ	บริษัทที่ถูกเรียกเก็บ (ตาราง platform จึงใช้ชื่อ company_id ไม่ใช่ tenant_id) → dt_companies
3	**invoice_no**	text			UQ	เลขใบแจ้งหนี้ `SUB-<ปีเดือน>-<ลำดับ>`
4	**plan_id**	text				แพ็กเกจของรอบนี้ (snapshot ตอนออก) ค่าที่ใช้ได้: `starter` `standard` `pro`
5	**period_start**	date			UQ	วันแรกของรอบบริการ — ไม่ซ้ำต่อบริษัท (กันออกซ้ำ)
6	**period_end**	date				วันสุดท้ายของรอบบริการ
7	**amount**	numeric(14,2)				ยอดเรียกเก็บ (บาท) ตามราคาแพ็กเกจ ณ วันออก
8	**issued_on**	date				วันที่ออกใบ
9	**due_date**	date				วันครบกำหนดชำระ — เกินกำหนด + ระยะผ่อนผัน → บริษัทถูกระงับ (`overdue`)
10	**status**	text				สถานะ: `open` รอชำระ · `paid` ชำระแล้ว · `void` ยกเลิก ค่าที่ใช้ได้: `open` `paid` `void`
11	**paid_at**	timestamptz	✓			เวลาที่บันทึกรับชำระ
	`created_at` , `created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (company_id, period_start)
- **UNIQUE** (invoice_no)
- **FK** (company_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((amount >= (0)::numeric))`
- **CHECK** `((period_end >= period_start))`
- **CHECK** `((invoice_no ~ '^SUB-[0-9]{6}-[0-9]{4,}$'::text))`
- **CHECK** `(((status = 'paid'::text) = (paid_at IS NOT NULL)))`
- **CHECK** `((plan_id = ANY (ARRAY['starter'::text, 'standard'::text, 'pro'::text])))`
- **CHECK** `((status = ANY (ARRAY['open'::text, 'paid'::text, 'void'::text])))`
- **index** `dt_billing_invoices_company_idx` btree (company_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(company_id = app_tenant_id())` · สิทธิ์ an22_app **S** · an22_platform **SIUD**

A `dt_platform_doc_counters` ตัวนับเลขเอกสารของแพลตฟอร์ม

เลขรันของเอกสารระดับแพลตฟอร์ม (ตอนนี้มีแค่ใบแจ้งหนี้ `SUB-YYYYMM-`) — แบบเดียวกับ `dt_doc_counters` ของร้าน

วงจรชีวิตของแถว

- แถวแรกของ prefix สร้างตอนออกเลขแรก แล้ว `last_no + 1` ทุกครั้ง (`INSERT … ON CONFLICT DO UPDATE`)
- ไม่ลบ

กฎสำคัญ

- แถวถูกล็อกจน commit จึงไม่ออกเลขซ้ำแม้มีหลาย instance; rollback แล้วเลขไม่ข้าม

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**prefix**	text			PK	ส่วนหน้าของเลข เช่น SUB-202611-
2	**last_no**	int				เลขล่าสุดที่ออกไปแล้วของ prefix นี้
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (prefix)
- **CHECK** ((last_no >= 0))

trigger: stamp (BEFORE INSERT OR UPDATE) · ไม่อยู่ใต้ RLS (ตาราง platform) · สิทธิ์ an22_app **—** · an22_platform **SIUD**

A dt_password_reset_tokens ลิงก์ตั้งรหัสผ่านใหม่ของ owner

token ของลิงก์ "ลืมรหัสผ่าน" ที่ส่งทางอีเมลให้ owner — ใช้ครั้งเดียว หมดอายุใน 1 ชั่วโมง E41; พนักงานไม่ใช้ช่องทางนี้ (admin ตั้งรหัสให้)

วงจรชีวิตของแถว

- สร้างตอน POST /auth/forgot-password ที่อีเมลตรงกับ owner ที่ใช้งานอยู่
- แก้: ตั้งรหัสสำเร็จ → used_at
- ลบ: job รายวันลบ token ที่หมดอายุเกิน 1 วัน, purge บริษัท

กฎสำคัญ

- เก็บเฉพาะ hash ของ token — ตัว token จริงอยู่ในลิงก์ของอีเมลเท่านั้น
- ไม่อยู่ใต้ RLS (ใช้ก่อน login) — an22_platform เท่านั้น

อ้างถึง dt_companies, dt_users

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — id ใน JSON ของ API
2	**company_id**	bigint			FK	บริษัทของผู้ใช้ (FK รวมกับ user_id → dt_users (tenant_id, id)) → dt_companies
3	**user_id**	bigint			FK	owner ที่ขอ (FK รวม company_id + user_id) → dt_users
4	**token_hash**	bytea			UQ	hash ของ token ในลิงก์ — ไม่ซ้ำ
5	**expires_at**	timestamptz				หมดอายุ (ออก + 1 ชั่วโมง)
6	**used_at**	timestamptz	✓			เวลาที่ใช้ตั้งรหัสแล้ว (null = ยังไม่ใช้) — ใช้แล้วใช้ซ้ำไม่ได้
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (token_hash)
- **FK** (company_id) → dt_companies (id) ON DELETE RESTRICT
- **FK** (company_id, user_id) → dt_users (tenant_id, id) ON DELETE RESTRICT
- **index** dt_password_reset_tokens_user_idx btree (company_id, user_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · ไม่อยู่ใต้ RLS (ตาราง platform) · สิทธิ์ an22_app **—** · an22_platform **SIUD**

A dt_login_throttle ตัวนับ login ผิด

จำกัดการเดารหัสผ่าน: นับ login ผิดต่อบัญชี (บริษัท + username ที่พิมพ์) และต่อ IP ภายในช่วงเวลา — เกินเพดาน (บัญชี 5 / IP 30 ครั้งใน 15 นาที) ล็อก 15 นาที ตอบ 429; ใช้นับ signup / ลืมรหัสผ่านต่อ IP ด้วย

วงจรชีวิตของแถว

- สร้าง / แก้: ทุกครั้งที่ตรวจรหัส (จองสิทธิ์ก่อนตรวจ แล้วนับผิดหรือคืนสิทธิ์ — atomic)
- ลบ: login สำเร็จลบแถวของบัญชี; job รายวันลบแถวที่หมดช่วงเกิน 1 วัน

กฎสำคัญ

- ไม่เก็บ username / IP ตรง ๆ — key เป็น HMAC ด้วยกุญแจ `PII_HASH_KEY`
- ไม่มี `tenant_id` / RLS (นับก่อนรู้ว่าเป็นบริษัทไหน) — an22_platform เท่านั้น

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**key_hash**	bytea			PK	HMAC ของ key (บัญชี = บริษัท + username, หรือ IP) — PK
2	**scope**	text				ชนิดของ key: `account` ต่อบัญชี · `ip` ต่อ IP (ค่า `X-Real-IP` จาก an22-web) ค่าที่ใช้ได้: `account` `ip`
3	**window_start**	timestamptz				เริ่มช่วงนับปัจจุบัน — เลยช่วงแล้วเริ่มนับใหม่
4	**failures**	int				จำนวนครั้งที่ผิดในช่วง (รวมสิทธิ์ที่จองไว้ระหว่างตรวจรหัส)
5	**locked_until**	timestamptz	✓			ล็อกถึงเวลานี้ (null = ไม่ล็อก) — ระหว่างล็อกตอบ 429 โดยไม่ตรวจรหัส
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (key_hash)
- **CHECK** `((failures >= 0))`
- **CHECK** `((scope = ANY (ARRAY['account'::text, 'ip'::text])))`
- **index** `dt_login_throttle_updated_idx` btree (updated_at)

trigger: stamp (BEFORE INSERT OR UPDATE) · ไม่อยู่ใต้ RLS (ตาราง platform) · สิทธิ์ an22_app **—** · an22_platform **SIUD**

B บริษัท สาขา ผู้ใช้

ข้อมูลระดับบริษัทที่ทุกเมนูใช้: ข้อมูลผู้ขายบนเอกสาร + กติกาที่ทุกสาขาใช้ร่วม, สาขา (1 สาขา = 1 คลังสินค้า = 1 สถานประกอบการ), ผู้ใช้และสาขาที่เข้าได้, เมนูของพนักงาน, เครื่อง POS, ตัวนับเลขเอกสาร และบันทึกการใช้งาน

B dt_company_info ข้อมูลบริษัท + กติกาที่ทุกสาขาใช้ร่วม

ข้อมูลผู้ขายที่พิมพ์บนเอกสาร (ชื่อ, เลขผู้เสียภาษี, จด VAT) และค่าตั้งระดับบริษัท (เปลี่ยนสินค้าได้กี่วัน, เพดานส่วนลดพิเศษ, กระดาษใบกำกับ, ค่ากะ, พนักงานเห็นทุน) — 1 แถวต่อบริษัท; หน้าตั้งค่า "บริษัท" (`PUT /settings/company` , company admin) E11

วงจรชีวิตของแถว

- สร้างตอนสมัคร
- แก้: หน้าตั้งค่า (company admin)
- ไม่ลบ (purge เท่านั้น)

กฎสำคัญ

- ข้อมูลร้านบนใบเสร็จ (`StoreInfo` ของ API) **ไม่ได้เก็บเป็นตาราง** — ประกอบจากตารางนี้ + `dt_branches` ของสาขาที่ขอ ทุกครั้งที่อ่าน
- ค่าที่ CHECK ตรงกับกฎ `companyInfoError()` ของหน้าบ้าน (ช่วงของแต่ละค่า)
- `require_shift` ปิดไม่ได้ขณะมีกะเปิดอยู่ (กฎในแอป)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK FK	บริษัท (PK — 1 แถวต่อบริษัท) → `dt_companies.id` → dt_companies
2	**name**	text				ชื่อผู้ขายบนเอกสาร / ชื่อบริษัทใน login
3	**tax_id**	text		`''`		เลขประจำตัวผู้เสียภาษี 13 หลัก ('' = ยังไม่กรอก)
4	**vat_registered**	bool		true		จดทะเบียน VAT — false: ไม่คิด VAT ในบิล, ออกใบกำกับเต็มรูปไม่ได้ (บิลเก็บ snapshot ไว้ที่ `dt_sales.vat_registered`)
5	**tax_invoice_paper**	text		`'A4'`		กระดาษเริ่มต้นของใบกำกับเต็มรูป: `A4` หรือ `80mm` (slip) ค่าที่ใช้ได้: `A4` `80mm`
6	**exchange_days**	int		7		เปลี่ยนสินค้าได้ภายในกี่วันหลังวันขาย (0 = ไม่จำกัด; admin ทำได้ตลอด)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
7	**manual_discount_max_percent**	numeric(5,2)		5		เพดานส่วนลดพิเศษที่พนักงานให้ได้ (% ของราคาหลังโปร; 0 = ห้าม; admin ไม่จำกัด)
8	**require_shift**	bool		true		บังคับเปิดกะก่อนขาย / ยกเลิกบิล / คืนเงิน / เงินเข้าออกลิ้นชัก
9	**shift_variance_limit**	numeric(14,2)		100		ส่วนต่างตอนปิดกะต่อช่องทางที่เกินนี้ (บาท) ต้องให้ admin อนุมัติ
10	**shift_max_hours**	int		12		เปิดกะนานเกินกี่ชั่วโมงให้เตือน (เตือนอย่างเดียว)
11	**staff_see_cost**	bool		false		ให้ role user เห็นต้นทุน / กำไร (ค่าเริ่ม false — API ใส่ null ในทุกช่องทุนของ response) E43
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					
12	**mfa_required**	text		'none'		ใครต้อง login ด้วยรหัส Microsoft Authenticator E55: `none` ไม่บังคับ (ผู้ใช้เลือกเอง) · `admins` owner + admin · `all` ทุกคน — มีผลตั้งแต่ login ครั้งถัดไป ค่าที่ใช้ได้: `none` `admins` `all`	

- **PK** (tenant_id)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `(((exchange_days >= 0) AND (exchange_days <= 365)))`
- **CHECK** `(((manual_discount_max_percent >= (0)::numeric) AND (manual_discount_max_percent <= (100)::numeric)))`
- **CHECK** `((mfa_required = ANY (ARRAY['none'::text, 'admins'::text, 'all'::text])))`
- **CHECK** `(((shift_max_hours >= 1) AND (shift_max_hours <= 48)))`
- **CHECK** `(((shift_variance_limit >= (0)::numeric) AND (shift_variance_limit <= (100000)::numeric)))`
- **CHECK** `((tax_id ~ '^([0-9]{13})?$'::text))`
- **CHECK** `((tax_invoice_paper = ANY (ARRAY['A4'::text, '80mm'::text])))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B dt_branches สาขา

สาขาของบริษัท 1 สาขา = 1 คลังสินค้า = 1 สถานประกอบการบนเอกสารภาษี; เก็บชื่อสถานประกอบการ ที่อยู่ เบอร์โทร ท้ายใบเสร็จ ที่พิมพ์บนเอกสารของสาขานั้น — เอกสาร / สต็อก / กะเกือบทุกตารางอ้างสาขา

วงจรชีวิตของแถว

- สาขาแรกสร้างตอนสมัคร (`branch_no` 00000 สำนักงานใหญ่)
- เพิ่ม / แก้: แท็บสาขาในข้อมูลหลัก (company admin) และข้อมูลสาขาในตั้งค่า (admin ของสาขา)
- **ไม่ลบ** — ปิดใช้งานด้วย `active = false`

กฎสำคัญ

- `code` และ `branch_no` ไม่ซ้ำในบริษัท; `code` ใช้ในเลขเอกสาร (เช่น `POS-HQ-1-…`)
- ต้องเหลือสาขาที่ใช้งาน ≥ 1; ปิดสาขาที่มีกะเปิดอยู่ไม่ได้; จำนวนสาขาที่ใช้งานไม่เกินแพ็กเกจ (กฎในแอป)

ถูกอ้างโดย `dt_audit_log`, `dt_branch_stock`, `dt_credit_notes`, `dt_exchanges`, `dt_held_bills`, `dt_pos_terminals`, `dt_sales`, `dt_serial_numbers`, `dt_shifts`, `dt_stock_movements`, `dt_tax_invoices`, `dt_transfers`, `dt_user_branches`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**code**	text			UQ	รหัสสาขา 2–10 ตัว (A-Z 0-9) — อยู่ในเลขเอกสารของสาขา
4	**name**	text				ชื่อสถานประกอบการ (พิมพ์บนเอกสาร / รายงานภาษีขาย)
5	**branch_no**	text			UQ	เลขสาขาตามกรมสรรพากร 5 หลัก (`00000` = สำนักงานใหญ่)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
6	**address**	text				ที่อยู่ของสถานประกอบการ (พิมพ์บนเอกสาร)
7	**phone**	text		''		เบอร์โทรบนใบเสร็จ
8	**receipt_footer**	text		''		ข้อความท้ายใบเสร็จ (≤ 200 ตัว)
9	**active**	bool		true		ใช้งานอยู่ (false = ปิด — ไม่มีในรายการเลือกสาขา)
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, branch_no)
- **UNIQUE** (tenant_id, code)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((branch_no ~ '^[0-9]{5}$'::text))`
- **CHECK** `((code ~ '^[A-Z0-9]{2,10}$'::text))`
- **CHECK** `(((char_length(name) >= 1) AND (char_length(name) <= 100)))`
- **CHECK** `((char_length(receipt_footer) <= 200))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B dt_users ผู้ใช้ของบริษัท

บัญชีพนักงานที่ login ด้วย รหัสบริษัท + username + รหัสผ่าน; บอกบทบาท (owner / admin / user) และสาขาที่เข้าได้

วงจรชีวิตของแถว

- owner สร้างตอนสมัคร; คนอื่นเพิ่มที่เมนูผู้ใช้ (admin)
- แก้: เมนูผู้ใช้, ตั้งรหัสผ่าน (`PUT /users/:id/password`), โอนความเป็นเจ้าของ, รีเซ็ตรหัสทางอีเมล (owner)
- ลบ: admin ลบได้ — ประวัติ (audit, เอกสาร) เก็บชื่อเป็น snapshot จึงอยู่ต่อ; owner ลบ / ปิด / เปลี่ยน role ไม่ได้

กฎสำคัญ

- `username` ไม่ซ้ำในบริษัท (เก็บตัวเล็ก) · `email` ไม่ซ้ำในบริษัทเมื่อไม่ว่าง (unique index บน `lower(email)`)
- owner 1 คนต่อบริษัท (partial unique index), ต้องมีอีเมลและ `all_branches = true` (CHECK)
- ลดสิทธิ์ / ปิด / ลบตัวเองไม่ได้ และต้องเหลือ admin ที่ใช้งาน ≥ 1 (กฎในแอป)

ถูกอ้างโดย `dt_password_reset_tokens`, `dt_user_branches`, `dt_user_mfa`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**username**	text			UQ	ชื่อ login (a-z 0-9 . _ - ยาว 3–30, ตัวเล็ก) — ไม่ซ้ำในบริษัท
4	**name**	text				ชื่อที่แสดง — ถูก copy เป็นชื่อผู้ทำในเอกสาร (cashier, voided_by ...)
5	**email**	text		''		อีเมล ('' = ไม่มี) — owner บังคับ (ใช้รับลิงก์ตั้งรหัสผ่าน)
6	**role**	text				บทบาท: `owner` เจ้าของ (สิทธิ์ admin ทั้งหมด + บัญชีบริษัท) · `admin` ผู้ดูแล · `user` พนักงาน (แคชเชียร์ / คลัง) ค่าที่ใช้ได้: `owner` `admin` `user`
7	**active**	bool		true		ใช้งานอยู่ (false = login ไม่ได้, token เดิมได้ 401)
8	**all_branches**	bool		false		เข้าได้ทุกสาขา (true แล้ว `dt_user_branches` ว่าง) — admin + ทุกสาขา = company admin
9	**password_hash**	text				hash ของรหัสผ่าน (BCrypt) — ไม่เคยส่งออก API

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
10	**created_at**	timestamptz				วันที่สร้างบัญชี (= User.createdAt) + เป็น stamp ด้วย
	created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, username)
- **UNIQUE index** dt_users_tenant_email_key btree (tenant_id, lower(email)) WHERE (email <> ''::text)
- **UNIQUE index** dt_users_tenant_owner_key btree (tenant_id) WHERE (role = 'owner'::text)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** (((role <> 'owner'::text) OR (all_branches AND (email <> ''::text))))
- **CHECK** ((role = ANY (ARRAY['owner'::text, 'admin'::text, 'user'::text])))
- **CHECK** ((username ~ '^[a-z0-9._-]{3,30}$'::text))

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B dt_user_branches สาขาที่ผู้ใช้เข้าได้

รายการสาขาของผู้ใช้ที่ไม่ได้ all_branches (User.branchIds) — ใช้ตัดสินว่า X-Branch-Id ที่ส่งมาใช้ได้หรือไม่ (ไม่ได้ → 403)

วงจรชีวิตของแถว

- เขียนใหม่ทั้งชุดทุกครั้งที่บันทึกผู้ใช้ (ลบของเดิมแล้ว insert)
- ว่างเมื่อผู้ใช้เป็น all_branches E9

กฎสำคัญ

- PK (tenant, user, branch)

อ้างถึง dt_branches, dt_users

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT
2	**user_id**	bigint			PK FK	ผู้ใช้ → dt_users → dt_users
3	**branch_id**	bigint			PK FK	สาขาที่เข้าได้ → dt_branches → dt_branches
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, user_id, branch_id)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, user_id) → dt_users (tenant_id, id) ON DELETE RESTRICT
- **index** dt_user_branches_branch_idx btree (tenant_id, branch_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B dt_user_mfa Microsoft Authenticator ของผู้ใช้

login สองขั้น E55: secret TOTP (RFC 6238) ของผู้ใช้ที่ผูกแอป Microsoft Authenticator — ผู้ใช้ที่มีแถวที่ยืนยันแล้ว (User.mfaEnabled) ต้องใส่รหัส 6 หลักทุกครั้งหลังรหัสผ่าน ไม่ว่าเข้าทาง /auth/login หรือ /auth/login-mfa

วงจรชีวิตของแถว

- สร้าง (รอยืนยัน confirmed_at null): login ทาง /auth/login-mfa / บริษัทบังคับ แต่ยังไม่ผูก หรือกดเปิดใช้ในหน้าตั้งค่า (POST /me/mfa/setup) — ลองใหม่ได้ secret เดิมจนกว่าจะยืนยัน
- ยืนยัน: รหัสแรกที่ถูก (POST /auth/mfa/verify หรือ /me/mfa/confirm) ตั้ง confirmed_at; ทุกครั้งที่ใช้รหัสตั้ง last_step

- ลบ: ผู้ใช้ปิดเอง (`POST /me/mfa/disable` + รหัสผ่าน, ถ้าบริษัทไม่บังคับ), admin ล้างให้เมื่อมือถือหาย (`DELETE /users/:id/mfa`), ลบผู้ใช้

กฎสำคัญ

- 1 แถวต่อผู้ใช้ (PK tenant + user)
- รับรหัสของช่วง 30 วินาทีปัจจุบัน ±1 ช่วง และต้องมากกว่า `last_step` (UPDATE … WHERE last_step < :step — รหัสเดิมใช้ซ้ำไม่ได้ แม้ส่งพร้อมกัน)
- ไม่อยู่ใน `GET /company/export` (`CompanyExportDao.SKIP`) — secret ไม่เคยออกนอก api หลังยืนยัน

อ้างถึง `dt_users`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**user_id**	bigint			PK FK	ผู้ใช้ → `dt_users` → dt_users
3	**secret_enc**	bytea				secret base32 (20 byte) เข้ารหัส AES-GCM ด้วย `PiiCipher` (AAD `dt_user_mfa.secret:<tenant>`)
4	**confirmed_at**	timestamptz	✓			เวลาที่ยืนยันด้วยรหัสแรก — null = ผูกยังไม่เสร็จ (ยังไม่นับว่าเปิดใช้)
5	**last_step**	bigint	✓			ช่วง 30 วินาที (unix time / 30) ของรหัสที่ใช้ล่าสุด — กันใช้รหัสซ้ำ
	`created_at` , `created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, user_id)
- **FK** (tenant_id, user_id) → dt_users (tenant_id, id) ON DELETE RESTRICT

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B `dt_menu_access` เมนูที่พนักงาน (role user) เห็น

admin เลือกเมนูที่ role user เห็นในหน้า `/users/menu-access` — ซ่อนแค่เมนู สิทธิ์ข้อมูลยังคุมที่ API (403)

วงจรชีวิตของแถว

- **ไม่มีแถว = ค่าเริ่มต้น** (แดชบอร์ด / ขายหน้าร้าน / การขาย / ตั้งค่า)
- บันทึกครั้งแรกสร้างแถว ต่อไปแก้แถวเดิม (upsert)

กฎสำคัญ

- 1 แถวต่อบริษัท
- เมนูที่เป็นของ admin เท่านั้นใส่ไม่ได้ (`menuAccessError()`)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK FK	บริษัท (PK) → `dt_companies.id` → dt_companies
2	**user_menus**	text[]				path ของเมนูที่ role user เห็น เช่น `{dashboard,pos,sales,settings}`
	`created_at` , `created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B `dt_pos_terminals` เครื่อง POS

ทะเบียนเครื่อง POS (= ลิ้นชักเงิน) ของบริษัท พร้อม POS ID ที่จดกับกรมสรรพากร — เครื่องตั้งครั้งเดียวโดย admin ในตั้งค่า "เครื่องนี้" แล้วส่งเลขเครื่องมาใน header `X-Terminal-Id`; บิล / กะ / เลขบิลรันต่อเครื่อง

วงจรชีวิตของแถว

- เพิ่ม / แก้ / ปิด: แท็บเครื่อง POS ในข้อมูลหลัก (admin)
- ลบได้เฉพาะเครื่องที่ยังไม่มีกะ / เอกสารอ้าง (FK RESTRICT + ตรวจในแอป)

กฎสำคัญ

- `id` = เลขเครื่องที่ผู้ใช้กำหนด 1–999 ไม่ซ้ำในบริษัท เปลี่ยนไม่ได้ (PK `(tenant_id, id)` ไม่ใช่ identity)
- `pos_id` ไม่ซ้ำในบริษัทเมื่อไม่ว่าง; ร้านจด VAT ที่เครื่องใช้งานไม่มี POS ID → เตือนเท่านั้น
- เครื่องอยู่สาขาเดียว — ขาย / เปิดกะได้เฉพาะเมื่อ `X-Branch-Id` ตรงสาขาของเครื่อง; ปิดเครื่องที่มีกะเปิดไม่ได้

อ้างถึง `dt_branches` · ถูกอ้างโดย `dt_credit_notes`, `dt_exchanges`, `dt_sales`, `dt_shifts`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
2	**id**	int			PK	เลขเครื่อง (ผู้ใช้กำหนด 1–999) — อยู่ในเลขบิล `POS-<สาขา>-<เครื่อง>-…`
3	**name**	text				ชื่อเครื่อง เช่น "เคาน์เตอร์ 1"
4	**pos_id**	text		''		หมายเลขเครื่อง POS ที่กรมสรรพากรอนุมัติ ('' = ยังไม่มี) — copy ลงบิล / กะ / ใบลดหนี้ตอนออก
5	**branch_id**	bigint			FK	สาขาที่เครื่องตั้งอยู่ → dt_branches
6	**active**	bool		true		ใช้งานอยู่ (เครื่องที่ปิดขาย / เปิดกะไม่ได้)
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, id)
- **UNIQUE index** `dt_pos_terminals_tenant_pos_id_key` btree (tenant_id, pos_id) WHERE (pos_id <> ''::text)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `(((id >= 1) AND (id <= 999)))`
- **CHECK** `(((char_length(name) >= 1) AND (char_length(name) <= 40)))`
- **CHECK** `((char_length(pos_id) <= 30))`
- **index** `dt_pos_terminals_branch_idx` btree (tenant_id, branch_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

B `dt_doc_counters` ตัวนับเลขเอกสารของบริษัท

เลขรันของเอกสารทุกชนิด (บิล, ใบกำกับ, ใบลดหนี้, ใบเปลี่ยน, กะ, ใบโอน) แยกตาม prefix ซึ่งรวมสาขา / เครื่อง / วันไว้แล้ว — ตารางรูปแบบเลขอยู่ในบทที่ 4

วงจรชีวิตของแถว

- prefix ใหม่ (เช่นวันใหม่) สร้างแถวตอนออกเลขแรก แล้ว `last_no + 1` (`DocNumbers.next`, `INSERT … ON CONFLICT DO UPDATE … RETURNING`)
- ไม่ลบ (purge เท่านั้น)

กฎสำคัญ

- ออกเลขหลังตรวจกฎผ่านหมดแล้ว ใน transaction เดียวกับเอกสาร: แถวถูกล็อกจน commit → ไม่ซ้ำแม้หลาย instance / ผ่าน PgBouncer และ rollback แล้วเลขไม่ข้าม (ไม่ใช้ SEQUENCE)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies
2	**prefix**	text			PK	ส่วนหน้าของเลข เช่น POS-HQ-1-20261108-, CN-BN-20261108-, SH-HQ-20261108-
3	**last_no**	int				เลขล่าสุดที่ออกแล้ว
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, prefix)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** ((last_no >= 0))

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SIU** · an22_platform **SIUD**

B dt_audit_log **บันทึกการใช้งาน (audit log)**

เหตุการณ์สำคัญของบริษัท (login, ยกเลิกบิล, ส่วนลดพิเศษ, ตัดสต็อก, แก้ราคา / โปร / ผู้ใช้ / ตั้งค่า, ปิดกะแทน ฯลฯ) — หน้า ประวัติการใช้งาน (GET /audit-log, admin) E40

วงจรชีวิตของแถว

- เพิ่มใน transaction เดียวกับงานนั้นเมื่อสำเร็จ (Audit.record); login ผิดบันทึกแยก transaction เพื่อให้อยู่แม้ request ล้ม
- **append-only**: an22_app มีแค่ INSERT + SELECT — ไม่มีการแก้ / ลบ; ลบพร้อมบริษัทตอน purge เท่านั้น

กฎสำคัญ

- branch_id null = เหตุการณ์ระดับบริษัท (เห็นเฉพาะ owner / company admin); branch admin เห็นเฉพาะสาขาของตัวเอง
- ref_kind กับ ref_label มีหรือไม่มีพร้อมกัน; มี ref_id ได้เมื่อมี ref_kind

อ้างถึง dt_branches

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — id ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies
3	**actor_kind**	text				ผู้ทำเป็นใคร: user ผู้ใช้ร้าน · platform ผู้ดูแลแพลตฟอร์ม · system ระบบ / job ค่าที่ใช้ได้: user platform system
4	**user_id**	bigint	✓		ref	ผู้ใช้ที่ทำ (null = platform / ระบบ / login ผิดที่ไม่รู้ผู้ใช้) ⇢ dt_users.id (ไม่มี FK — ประวัติอยู่ต่อแม้ลบผู้ใช้)
5	**user_name**	text				ชื่อผู้ทำ (snapshot) หรือ username ที่พิมพ์ตอน login ผิด
6	**branch_id**	bigint	✓		FK	สาขาของเหตุการณ์ (null = ระดับบริษัท) → dt_branches
7	**terminal_id**	bigint	✓		ref	เครื่อง POS ที่ทำ (ถ้ามี) ⇢ dt_pos_terminals.id (ไม่มี FK — เครื่องลบได้)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
8	**action**	text				ชนิดเหตุการณ์ — ป้ายภาษาไทยของแต่ละค่าอยู่ที่ AUDIT_ACTION_LABEL ของหน้าบ้าน ค่าที่ใช้ได้: login login_failed sale_void manual_discount credit_note tax_invoice_reissue stock_out serial_remove transfer_cancel price_change promotion_change user_change user_password owner_transfer plan_change signup company_suspend company_reactivate invoice_paid company_export deletion_request deletion_cancel password_reset menu_access company_settings branch_settings branch_change terminal_change shift_close_other shift_approve shift_adjust mfa_enable mfa_disable mfa_reset
9	**ref_kind**	text	✓			ชนิดของสิ่งที่ถูกกระทำ (null = ไม่มี เช่น login) ค่าที่ใช้ได้: sale credit_note tax_invoice product transfer price promotion user branch terminal shift company
10	**ref_id**	bigint	✓			id ของสิ่งนั้น (ไม่มี FK — ประวัติอยู่ต่อแม้ถูกลบ)
11	**ref_label**	text	✓			ป้ายที่อ่านได้ เช่น เลขบิล / รหัส SKU (snapshot)
12	**detail**	text		''		รายละเอียดเพิ่ม เช่น ยอดส่วนลด + เหตุผล
13	**created_at**	timestamptz				เวลาเกิดเหตุการณ์ (= AuditEvent.at) — มี index (tenant, created_at) สำหรับค้นตามช่วงวัน
	created_by , created_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ created_*					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((action = ANY (ARRAY['login'::text, 'login_failed'::text, 'sale_void'::text, 'manual_discount'::text, 'credit_note'::text, 'tax_invoice_reissue'::text, 'stock_out'::text, 'serial_remove'::text, 'transfer_cancel'::text, 'price_change'::text, 'promotion_change'::text, 'user_change'::text, 'user_password'::text, 'owner_transfer'::text, 'plan_change'::text, 'signup'::text, 'company_suspend'::text, 'company_reactivate'::text, 'invoice_paid'::text, 'company_export'::text, 'deletion_request'::text, 'deletion_cancel'::text, 'password_reset'::text, 'menu_access'::text, 'company_settings'::text, 'branch_settings'::text, 'branch_change'::text, 'terminal_change'::text, 'shift_close_other'::text, 'shift_approve'::text, 'shift_adjust'::text, 'mfa_enable'::text, 'mfa_disable'::text, 'mfa_reset'::text])))`
- **CHECK** `((actor_kind = ANY (ARRAY['user'::text, 'platform'::text, 'system'::text])))`
- **CHECK** `((((ref_kind IS NULL) = (ref_label IS NULL)) AND ((ref_kind IS NOT NULL) OR (ref_id IS NULL))))`
- **CHECK** `((ref_kind = ANY (ARRAY['sale'::text, 'credit_note'::text, 'tax_invoice'::text, 'product'::text, 'transfer'::text, 'price'::text, 'promotion'::text, 'user'::text, 'branch'::text, 'terminal'::text, 'shift'::text, 'company'::text])))`
- **index** `dt_audit_log_branch_idx` btree (tenant_id, branch_id)
- **index** `dt_audit_log_tenant_at_idx` btree (tenant_id, created_at)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

C สินค้าและข้อมูลหลัก

ข้อมูลหลักระดับบริษัท (ใช้ร่วมทุกสาขา): หมวดหมู่ 3 ระดับ, ผู้จำหน่าย, ช่องทางชำระเงิน (ปุ่มของ POS) และ SKU พร้อมหน่วยแพ็ก / ผู้จำหน่ายของ SKU — สต็อกต่อสาขาอยู่ในกลุ่ม E ราคาขายอยู่ในกลุ่ม D

C dt_categories หมวดหมู่สินค้า 3 ระดับ

ต้นไม้หมวดหมู่ แผนก > หมวด > หมวดย่อย — SKU ผูกกับหมวดระดับล่างสุดที่ใช้งาน; โปรโมชั่นระบุหมวดได้ (ครอบคลุมทุก SKU ข้างล่าง)

วงจรชีวิตของแถว

- เพิ่ม / แก้ / ปิด: แท็บหมวดหมู่ในข้อมูลหลัก (admin)
- ลบได้เฉพาะหมวดที่ไม่มีหมวดลูกและไม่มี SKU; หมวดที่มี SKU เพิ่มหมวดลูกไม่ได้

กฎสำคัญ

- `level` 1 ↔ ไม่มี `parent_id` (CHECK); ลูกอยู่ระดับ parent + 1 (กฎในแอป)
- `code` ไม่ซ้ำในบริษัท

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- `productCount` (จำนวน SKU ในหมวด) นับตอนอ่าน
- `Product.categoryPath` เช่น "ไอที > คอมพิวเตอร์ > โน้ตบุ๊ก" ประกอบตอนอ่าน

ถูกอ้างโดย `dt_products`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**code**	text			UQ	รหัสหมวด (A-Z 0-9 -) ไม่ซ้ำในบริษัท
4	**name**	text				ชื่อหมวด
5	**parent_id**	bigint	✓		FK	หมวดแม่ (null = ระดับ 1 แผนก) → self FK → dt_categories
6	**level**	int2				ระดับ 1 แผนก · 2 หมวด · 3 หมวดย่อย
7	**active**	bool		true		ใช้งานอยู่ (SKU ใหม่ผูกได้เฉพาะหมวดล่างสุดที่ใช้งาน)
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, code)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, parent_id) → dt_categories (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((code ~ '^[A-Z0-9-]+$'::text))`
- **CHECK** `(((level >= 1) AND (level <= 3)))`
- **CHECK** `((btrim(name) <> ''::text))`
- **CHECK** `(((parent_id IS NULL) = (level = 1)))`
- **index** `dt_categories_parent_idx` btree (tenant_id, parent_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

C `dt_suppliers` ผู้จำหน่าย

ผู้จำหน่ายที่ร้านซื้อสินค้า — ใช้ผูกกับ SKU (ทุนซื้อ, lead time, MOQ) และจัดกลุ่มใบแนะนำสั่งซื้อ

วงจรชีวิตของแถว

- เพิ่ม / แก้: แท็บผู้จำหน่าย (admin) — PUT แทนทั้งแถว
- ลบไม่ได้ขณะผูกกับ SKU (FK จาก `dt_product_suppliers`) → ปิดใช้งานแทน

กฎสำคัญ

- `code` ไม่ซ้ำ; (เลขภาษี, สำนักงานใหญ่ / สาขา, เลขสาขา) ไม่ซ้ำในบริษัท
- เลขภาษี 13 หลักต้องผ่าน checksum (`isValidThaiTaxId()` ในแอป); สำนักงานใหญ่ → `branch_no` ว่าง, สาขา → 5 หลัก (CHECK)

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- `productCount` นับตอนอ่าน

ถูกอ้างโดย `dt_product_suppliers`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**code**	text			UQ	รหัสผู้จำหน่าย (A-Z 0-9 -)
4	**name**	text				ชื่อผู้จำหน่าย
5	**tax_id**	text			UQ	เลขประจำตัวผู้เสียภาษี 13 หลัก
6	**branch_type**	text			UQ	`head` สำนักงานใหญ่ · `branch` สาขา ค่าที่ใช้ได้: `head` `branch`
7	**branch_no**	text		`''`	UQ	เลขสาขา 5 หลักเมื่อเป็นสาขา ('' เมื่อสำนักงานใหญ่)
8	**address**	text		`''`		ที่อยู่
9	**contact_name**	text		`''`		ผู้ติดต่อ
10	**phone**	text		`''`		เบอร์โทร
11	**email**	text		`''`		อีเมล
12	**credit_days**	int		`30`		เครดิตกี่วัน (0 = เงินสด)
13	**bank_name**	text		`''`		ธนาคารสำหรับโอนจ่าย
14	**bank_account_no**	text		`''`		เลขบัญชี
15	**bank_account_name**	text		`''`		ชื่อบัญชี
16	**note**	text		`''`		หมายเหตุ
17	**active**	bool		`true`		ใช้งานอยู่
	`created_at` , `created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, code)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, tax_id, branch_type, branch_no)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((((branch_type = 'head'::text) AND (branch_no = ''::text)) OR ((branch_type = 'branch'::text) AND (branch_no ~ '^[0-9]{5}$'::text))))`
- **CHECK** `((branch_type = ANY (ARRAY['head'::text, 'branch'::text])))`
- **CHECK** `((code ~ '^[A-Z0-9-]+$'::text))`
- **CHECK** `((credit_days >= 0))`
- **CHECK** `((btrim(name) <> ''::text))`
- **CHECK** `((tax_id ~ '^[0-9]{13}$'::text))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

C dt_payment_methods ช่องทางชำระเงิน

ปุ่มรับเงินของ POS (เงินสด, บัตร, QR PromptPay, โอน, e-wallet, ผ่อน, อื่น ๆ) พร้อมเงื่อนไขของแต่ละช่องทาง — เรียงปุ่มด้วย `sort_order`

วงจรชีวิตของแถว

- ช่องทางเงินสดแรกสร้างตอนสมัคร; เพิ่ม / แก้ / ปิดที่แท็บช่องทางชำระ (admin) — PUT แทนทั้งแถว, เรียงด้วย `PUT /payment-methods/order`
- ลบได้เฉพาะที่ยังไม่มีการขายใช้ (ตรวจกับ `dt_sale_payments`) → ปิดใช้งานแทน

กฎสำคัญ

- ต้องมีช่องทางเงินสดที่ใช้งาน ≥ 1 เสมอ (ปิด / เปลี่ยนชนิด / ลบตัวสุดท้ายไม่ได้)
- ค่าที่ชนิดนั้นไม่ใช้ถูกล้างก่อนบันทึก (`withPaymentTypeRules()`): ปัดเศษ = เงินสดเท่านั้น, PromptPay = qr, บัญชี = qr / โอน, เดือนผ่อน = ผ่อน
- การขายเก็บชื่อ / ชนิดของช่องทางเป็น snapshot ใน `dt_sale_payments`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**code**	text			UQ	รหัสช่องทาง (A-Z 0-9 -, ≤ 20)
4	**name**	text				ข้อความบนปุ่ม POS / ใบเสร็จ (≤ 20 ตัว)
5	**type**	text				ชนิด: `cash` เงินสด · `card` บัตร · `qr` QR / PromptPay · `transfer` โอน · `e_wallet` · `installment` ผ่อน · `other` ค่าที่ใช้ได้: `cash` `card` `qr` `transfer` `e_wallet` `installment` `other`
6	**active**	bool		true		แสดงเป็นปุ่มใน POS
7	**sort_order**	int				ลำดับปุ่ม (น้อยอยู่ก่อน)
8	**require_reference**	bool		false		แคชเชียร์ต้องกรอกเลขอ้างอิง (เลขอนุมัติ / เลขอ้างอิงการโอน)
9	**reference_label**	text		''		ป้ายของช่องเลขอ้างอิง เช่น "เลขอนุมัติ"
10	**min_amount**	numeric(14,2)		0		ยอดขั้นต่ำต่อบรรทัดการชำระ (0 = ไม่มี)
11	**max_amount**	numeric(14,2)	✓			ยอดสูงสุดต่อบรรทัด (null = ไม่จำกัด)
12	**fee_percent**	numeric(5,2)		0		ค่าธรรมเนียม (MDR) % ที่ร้านเสีย — ใช้คิดยอดรับสุทธิในรายงาน (เงินสด 0)
13	**cash_rounding**	text		'none'		ปัดเศษเงินสด: `none` · `0.25` ปัดเป็นสลึง · `1` ปัดเป็นบาท ค่าที่ใช้ได้: `none` `0.25` `1`
14	**prompt_pay_id**	text		''		PromptPay ID (เบอร์ 10 หลัก หรือเลขภาษี 13 หลัก) — ชนิด qr เท่านั้น ('' = ไม่มี)
15	**bank_account**	text		''		บัญชีรับเงิน (qr / โอน) สำหรับกระทบยอด
16	**installment_months**	int[]		'{}'		จำนวนเดือนผ่อนที่เลือกได้ เช่น `{3,6,10}` — ชนิดผ่อนเท่านั้น
17	**close_check**	text				ตรวจตอนปิดกะ: `count` นับยอด · `confirm` ติ๊กว่าตรวจแล้ว · `none` ไม่ตรวจ (เงินสด = count เสมอ) ค่าที่ใช้ได้: `count` `confirm` `none`
18	**note**	text		''		หมายเหตุ
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, code)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((cash_rounding = ANY (ARRAY['none'::text, '0.25'::text, '1'::text])))`
- **CHECK** `(((max_amount > (0)::numeric) AND (max_amount >= min_amount)))`
- **CHECK** `((close_check = ANY (ARRAY['count'::text, 'confirm'::text, 'none'::text])))`
- **CHECK** `((code ~ '^[A-Z0-9-]{1,20}$'::text))`
- **CHECK** `(((fee_percent >= (0)::numeric) AND (fee_percent <= (10)::numeric)))`
- **CHECK** `((min_amount >= (0)::numeric))`
- **CHECK** `(((char_length(name) >= 1) AND (char_length(name) <= 20)))`
- **CHECK** `((type = ANY (ARRAY['cash'::text, 'card'::text, 'qr'::text, 'transfer'::text, 'e_wallet'::text, 'installment'::text, 'other'::text])))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

C `dt_products` สินค้า (SKU master)

ข้อมูลหลักของ SKU ระดับบริษัท ใช้ร่วมกันโดยเมนู SKU (master), คลังสินค้า (สต็อก) และราคา — สินค้าปกติ / คุม serial / บริการ (ไม่มีสต็อก); สต็อก + ทุนเฉลี่ยต่อสาขาอยู่ที่ `dt_branch_stock` ราคาขายอยู่ที่ `dt_sku_prices`

วงจรชีวิตของแถว

- เพิ่ม / แก้: ฟอร์ม SKU หรือนำเข้า Excel (`POST` / `PUT /products`)
- ลบได้เมื่อหมดสต็อกทุกสาขา ไม่มีประวัติการขาย ไม่มีใบโอนค้าง และไม่อยู่ในโปรที่ยังไม่หมดอายุ — ประวัติ movement / serial / เอกสารอยู่ต่อ (ไม่มี FK)

กฎสำคัญ

- `sku` ไม่ซ้ำในบริษัทแบบไม่สนตัวพิมพ์ (unique index บน `upper(sku)`)
- **บาร์โค้ดทุกตัว (ของ SKU + ของหน่วยแพ็ก) ไม่ซ้ำทั้งบริษัท** — ตรวจในแอปใต้ล็อกระดับบริษัท (ไม่มี unique ใน DB เพราะอยู่ 2 ตาราง)
- บริการไม่คุม serial (CHECK) และไม่มีสต็อก / แพ็ก / จุดสั่งซื้อ (`withServiceRules()`); `serial_control` / `item_type` เปลี่ยนไม่ได้ขณะมีสต็อกที่สาขาใดหรือมีใบโอนค้าง
- หมวดต้องเป็นหมวดล่างสุดที่ใช้งาน

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- `stock` / `avgCost` / `minStock` / `maxStock` เติมจาก `dt_branch_stock` ของสาขาใน `X-Branch-Id` (ไม่มีแถว = 0)
- `currentPrice` = ราคาจากช่วงราคาที่ครอบวันนี้ (null = ยังไม่กำหนดราคา)
- `categoryPath` , `packUnits` (จาก `dt_product_pack_units`), `suppliers` (จาก `dt_product_suppliers`)

อ้างถึง `dt_categories` · ถูกอ้างโดย `dt_branch_stock` , `dt_product_pack_units` , `dt_product_suppliers` , `dt_sku_prices`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**sku**	text			UQ	รหัส SKU (ไม่ซ้ำแบบไม่สนตัวพิมพ์)
4	**name**	text				ชื่อสินค้า
5	**short_name**	text		`''`		ชื่อบนใบเสร็จ / ปุ่ม POS (≤ 20 ตัว; ว่าง = ใช้ name)
6	**category_id**	bigint	✓		FK	หมวดย่อย (null = ยังไม่ระบุ) → dt_categories
7	**unit**	text		`'ชิ้น'`		หน่วยฐาน (นับสต็อกด้วยหน่วยนี้) เช่น ชิ้น
8	**barcode**	text		`''`		บาร์โค้ดของหน่วยฐาน ('' = ไม่มี)
9	**brand**	text		`''`		ยี่ห้อ
10	**model**	text		`''`		รุ่น
11	**item_type**	text		`'stock'`		`stock` สินค้ามีสต็อก · `service` บริการ (ค่าติดตั้ง ค่าแรง — ไม่มีสต็อก) ค่าที่ใช้ได้: `stock` `service`
12	**image_url**	text		`''`		รูปย่อเป็น data URL ขนาดไม่เกิน 400px ('' = ไม่มี)
13	**vat_type**	text		`'vat7'`		ภาษี: `vat7` ราคารวม VAT 7% · `exempt` ยกเว้น VAT ค่าที่ใช้ได้: `vat7` `exempt`
14	**cost**	numeric(14,4)		`0`		**ทุนมาตรฐาน** ต่อหน่วยฐาน ไม่รวม VAT — ค่าเริ่มตอนรับเข้า / ประมาณการเมื่อไม่มีสต็อก; บริการ = ต้นทุนต่อการขาย (COGS); **ห้ามใช้คิดมูลค่าสต็อก** (ใช้ avg_cost)
15	**warranty_months**	int		`0`		ประกันกี่เดือน (0 = ไม่มี) — copy ลงบรรทัดบิล

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
16	**serial_control**	bool		false		คุม serial รายเครื่อง (สต็อก = จำนวน serial ที่ in_stock; มีหน่วยเดียว ไม่มีแพ็ก)
17	**serial_prefix**	text		''		รูปแบบ serial: ต้องขึ้นต้นด้วย ('' = ไม่บังคับ)
18	**serial_length**	int	✓			รูปแบบ serial: ความยาว (null = ไม่บังคับ)
19	**sale_status**	text		'active'		สถานะการขาย: `active` ปกติ · `no_sale` ห้ามขาย (ซื้อได้) · `no_purchase` ห้ามซื้อ (ขายของที่เหลือ) · `discontinued` เลิกจำหน่าย (ซ่อนจากคลัง / ราคา) ค่าที่ใช้ได้: `active` `no_sale` `no_purchase` `discontinued`
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE index** `dt_products_tenant_sku_key` btree (tenant_id, upper(sku))
- **FK** (tenant_id, category_id) → dt_categories (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((cost >= (0)::numeric))`
- **CHECK** `((item_type = ANY (ARRAY['stock'::text, 'service'::text])))`
- **CHECK** `((sale_status = ANY (ARRAY['active'::text, 'no_sale'::text, 'no_purchase'::text, 'discontinued'::text])))`
- **CHECK** `((serial_length > 0))`
- **CHECK** `(((item_type = 'stock'::text) OR (NOT serial_control)))`
- **CHECK** `((char_length(short_name) <= 20))`
- **CHECK** `((btrim(sku) <> ''::text))`
- **CHECK** `((vat_type = ANY (ARRAY['vat7'::text, 'exempt'::text])))`
- **CHECK** `((warranty_months >= 0))`
- **index** `dt_products_category_idx` btree (tenant_id, category_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

C `dt_product_pack_units` หน่วยแพ็กของ SKU

หน่วยใหญ่ที่ขาย / ซื้อได้นอกจากหน่วยฐาน พร้อมบาร์โค้ดของตัวเอง เช่น 1 กล่อง = 4 ชิ้น (`Product.packUnits`)

วงจรชีวิตของแถว

- เขียนใหม่ทั้งชุดทุกครั้งที่บันทึก SKU (ลบของเดิมแล้ว insert ตามลำดับ)
- ลบพร้อม SKU

กฎสำคัญ

- SKU คุม serial ไม่มีแถว (มีหน่วยเดียว)
- ชื่อหน่วยไม่ซ้ำใน SKU; บาร์โค้ดไม่ซ้ำกับบาร์โค้ดใด ๆ ของบริษัท (ตรวจในแอป)

อ้างถึง `dt_products`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**product_id**	bigint			PK FK	SKU → dt_products
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**unit**	text			UQ	ชื่อหน่วย เช่น กล่อง ลัง
5	**factor**	int				จำนวนหน่วยฐานต่อ 1 หน่วยนี้ (> 1)
6	**barcode**	text		''		บาร์โค้ดของหน่วยนี้ ('' = ไม่มี)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, product_id, position)
- **UNIQUE** (tenant_id, product_id, unit)
- **FK** (tenant_id, product_id) → dt_products (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((factor > 1))`
- **CHECK** `((btrim(unit) <> ''::text))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

C dt_product_suppliers ผู้จำหน่ายของ SKU

เงื่อนไขการซื้อของ SKU จากผู้จำหน่ายแต่ละราย (`Product.suppliers`) — รายหลักใช้กับใบแนะนำสั่งซื้อและเป็นทุนเริ่มต้นตอนนำเข้าการรับสินค้า

วงจรชีวิตของแถว

- เขียนใหม่ทั้งชุดทุกครั้งที่บันทึก SKU
- ลบพร้อม SKU

กฎสำคัญ

- ผู้จำหน่ายซ้ำใน SKU ไม่ได้ (PK)
- เมื่อมีแถว ต้องมีรายหลัก (`is_main`) 1 รายพอดี (partial unique index)

อ้างถึง `dt_products`, `dt_suppliers`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**product_id**	bigint			PK FK	SKU → dt_products
3	**supplier_id**	bigint			PK FK	ผู้จำหน่าย → dt_suppliers
4	**position**	int				ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
5	**supplier_sku**	text		''		รหัสสินค้าของผู้จำหน่าย (ใช้ในใบสั่งซื้อ)
6	**cost**	numeric(14,4)		0		ทุนซื้อต่อหน่วยฐาน ไม่รวม VAT
7	**lead_time_days**	int		0		สั่งแล้วได้ของในกี่วัน
8	**moq**	int		0		สั่งขั้นต่ำ (หน่วยฐาน; 0 = ไม่มี)
9	**is_main**	bool		false		ผู้จำหน่ายหลักของ SKU
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, product_id, supplier_id)
- **UNIQUE index** `dt_product_suppliers_main_key` btree (tenant_id, product_id) WHERE is_main
- **FK** (tenant_id, product_id) → dt_products (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, supplier_id) → dt_suppliers (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((cost >= (0)::numeric))`
- **CHECK** `((lead_time_days >= 0))`
- **CHECK** `((moq >= 0))`
- **index** `dt_product_suppliers_supplier_idx` btree (tenant_id, supplier_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

D ราคาและโปรโมชั่น

ราคาขายเป็นช่วงวันที่ (ราคาเดียวทั้งบริษัท) และโปรโมชั่นที่ POS ใช้คิดราคา — กฎการคิดราคาอยู่ใน `PosPricing` (port จาก `priceCart()` ของหน้าบ้าน) ไม่อยู่ใน DB

D `dt_sku_prices` ราคาขายตามช่วงวัน

ราคาขาย (รวม VAT) ของ SKU เป็นช่วงวันที่ ราคาเดียวทั้งบริษัท D2 — ตั้งราคาอนาคตล่วงหน้าได้; `Product.currentPrice` = ช่วงที่ครอบวันนี้

วงจรชีวิตของแถว

- เพิ่ม / แก้ / ลบ: เมนูราคา (admin)
- ลบพร้อม SKU

กฎสำคัญ

- **ช่วงของ SKU เดียวกันห้ามทับกัน** — `EXCLUDE USING gist` บน `daterange(start_date, end_date, '[]')` (extension btree_gist) เป็นตาข่ายชั้นสุดท้าย แอปตรวจก่อนด้วย `findOverlap()` เพื่อข้อความภาษาไทย
- วันที่เป็นวันท้องถิ่น (Asia/Bangkok) รวมทั้งสองปลาย; `end_date` null = ไม่สิ้นสุด

อ้างถึง `dt_products`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**product_id**	bigint			FK	SKU → dt_products
4	**price**	numeric(14,2)				ราคาขายต่อหน่วยฐาน รวม VAT (บาท) — หน่วยแพ็กขาย = ราคา × factor
5	**start_date**	date				วันเริ่ม (รวมวันนี้)
6	**end_date**	date	✓			วันสุดท้าย (รวมวันนี้; null = ไม่สิ้นสุด)
7	**note**	text		''		หมายเหตุ เช่น เหตุผลที่ปรับราคา
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **EXCLUDE** USING gist (tenant_id WITH =, product_id WITH =, daterange(start_date, end_date, '[]'::text) WITH &&)
- **FK** (tenant_id, product_id) → dt_products (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((end_date >= start_date))`
- **CHECK** `((price >= (0)::numeric))`
- **index** `dt_sku_prices_no_overlap` gist (tenant_id, product_id, daterange(start_date, end_date, '[]'::text))
- **index** `dt_sku_prices_product_idx` btree (tenant_id, product_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

D `dt_promotions` โปรโมชั่น

โปรที่ POS ใช้คิดราคาอัตโนมัติ 3 แบบ: `item_discount` ลดต่อชิ้น · `bill_discount` ลดท้ายบิล · `free_goods` ของแถม — มีช่วงวัน, ลำดับความสำคัญ, รวมกับโปรอื่นได้หรือไม่ และจำกัดสาขาได้

วงจรชีวิตของแถว

- เพิ่ม / แก้ / ลบ: เมนูโปรโมชั่น (admin; จำกัดสาขา = company admin)
- **เริ่มแล้วแก้ได้แค่** เปิด / ปิด, เลื่อนวันจบให้เร็วขึ้น (≥ วันนี้) และหมายเหตุ; เริ่มแล้วลบไม่ได้ (ปิดแทน)

กฎสำคัญ

- ชนิด `free_goods` ↔ มี `free_goods` และไม่มี `discount` (CHECK)
- scope / ของแถม / สาขาเก็บเป็น array / jsonb **ไม่มี FK**: โปรที่หมดอายุเป็นประวัติ (ลบ SKU / หมวดที่โปรหมดอายุอ้างได้) — SKU / หมวดที่โปรยังไม่หมดอายุอ้างอยู่ลบไม่ได้ (ตรวจในแอป)
- โปรทับช่วงกันได้; กฎการคิด (priority สูงก่อน, ไม่ stackable หยุดตัวที่ต่ำกว่า, ลดสินค้าก่อนลดท้ายบิล) อยู่ใน `PosPricing`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**code**	text			UQ	รหัสโปร (A-Z 0-9 -, ≤ 30) ไม่ซ้ำในบริษัท
4	**name**	text				ชื่อโปร (แสดงในบิล)
5	**note**	text		`''`		หมายเหตุ
6	**type**	text				ชนิด: `item_discount` ลดราคาสินค้า · `bill_discount` ลดท้ายบิล · `free_goods` ของแถม ค่าที่ใช้ได้: `item_discount` `bill_discount` `free_goods`
7	**start_date**	date				วันเริ่ม (รวม)
8	**end_date**	date	✓			วันจบ (รวม; null = ไม่สิ้นสุด)
9	**enabled**	bool		`true`		เปิดใช้ (false = ปิดด้วยมือ ไม่ว่าวันที่จะเป็นอย่างไร)
10	**priority**	int		`0`		ลำดับความสำคัญ 0–999 (สูงคิดก่อน)
11	**stackable**	bool		`false`		รวมกับโปรอื่นบนสินค้า / บิลเดียวกันได้
12	**scope_all**	bool		`false`		ใช้กับสินค้าทุกตัว (ลดท้ายบิล = true เสมอ)
13	**scope_product_ids**	bigint[]		`'{}'`	ref	SKU ที่ร่วมโปร (array ของ `dt_products.id`) ⇢ dt_products.id (ไม่มี FK — array)
14	**scope_category_ids**	bigint[]		`'{}'`	ref	หมวดที่ร่วมโปร (array ของ `dt_categories.id` — ครอบคลุมทุก SKU ข้างล่าง) ⇢ dt_categories.id (ไม่มี FK — array)
15	**min_qty**	int		1		ลดสินค้า: ขั้นต่ำต่อบรรทัด · ของแถม: ซื้อกี่ชิ้นต่อ 1 ชุด (0 = ใช้ min_amount)
16	**min_amount**	numeric(14,2)		`0`		ลดท้ายบิล: ยอดบิลขั้นต่ำ · ของแถม: ยอดซื้อสินค้าที่ร่วมโปรต่อ 1 ชุด (0 = ใช้ min_qty)
17	**discount**	jsonb	✓			ลดสินค้า / ท้ายบิล: `{ kind: percent | amount, value, maxDiscount }` (ลดสินค้าคิดต่อชิ้น; % มีเพดานบาทได้) — null สำหรับของแถม
18	**free_goods**	jsonb	✓			ของแถม: `{ items: [{ productId, qty }], repeat, maxSets }` — repeat = ทุกครั้งที่ครบเงื่อนไขได้อีกชุด, maxSets = เพดานชุดต่อบิล
19	**branch_ids**	bigint[]		`'{}'`	ref	สาขาที่ใช้ได้ (array ของ `dt_branches.id`; ว่าง = ทุกสาขา) ⇢ dt_branches.id (ไม่มี FK — array)
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, code)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((end_date >= start_date))`
- **CHECK** `((code ~ '^[A-Z0-9-]{1,30}$'::text))`
- **CHECK** `((((type = 'free_goods'::text) = (free_goods IS NOT NULL)) AND ((discount IS NULL) OR (free_goods IS NULL))))`

- **CHECK** `((min_amount >= (0)::numeric))`
- **CHECK** `((min_qty >= 0))`
- **CHECK** `((btrim(name) <> ''::text))`
- **CHECK** `(((priority >= 0) AND (priority <= 999)))`
- **CHECK** `((type = ANY (ARRAY['item_discount'::text, 'bill_discount'::text, 'free_goods'::text])))`

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

E สต็อกและการโอน

สต็อกต่อสาขา (ยอดคงเหลือ + ทุนเฉลี่ย), บัตรสต็อก (ทุกการเดินของสินค้า), serial รายเครื่อง และใบโอนระหว่างสาขา — สต็อกเปลี่ยนผ่าน `StockLedger` ของ api เท่านั้น

E dt_branch_stock สต็อกต่อสาขา

ยอดคงเหลือ ทุนเฉลี่ย และจุดสั่งซื้อของ SKU ที่แต่ละสาขา (1 สาขา = 1 คลัง, D1) — API เติมค่าของสาขาใน `X-Branch-Id` ลงใน `Product`

วงจรชีวิตของแถว

- สร้างเมื่อมีการเดินสต็อกครั้งแรกของ (สาขา, SKU) หรือบันทึกจุดสั่งซื้อ (`INSERT … ON CONFLICT DO NOTHING`) — **ไม่มีแถว = ทุกค่าเป็น 0** E13
- แก้: ทุกการเดินสต็อกผ่าน `StockLedger` (ล็อกแถว FOR UPDATE → คิดทุน → update → เขียน movement) และฟอร์ม SKU / Excel (จุดสั่งซื้อของสาขาปัจจุบัน)
- ลบพร้อม SKU (ยอดเป็น 0 แล้ว)

กฎสำคัญ

- **ทุน**: สินค้าปกติ = เฉลี่ยเคลื่อนที่ (`movingAverage()` ทุกครั้งที่รับเข้า ตัดออกที่ทุนเฉลี่ยเดิม); serial = เฉลี่ยทุนของเครื่องที่ in_stock
- สต็อกติดลบไม่ได้ (CHECK `stock >= 0` + ตรวจ "สต็อกคงเหลือไม่พอ" ในแอป)
- serial SKU: `stock` = จำนวน serial ที่ in_stock ของสาขา

อ้างถึง `dt_branches` , `dt_products`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**branch_id**	bigint			PK FK	สาขา (คลัง) → dt_branches
3	**product_id**	bigint			PK FK	SKU → dt_products
4	**stock**	int		0		คงเหลือ (หน่วยฐาน)
5	**avg_cost**	numeric(14,4)		0		ทุนเฉลี่ยต่อหน่วยฐาน ไม่รวม VAT (ทศนิยม 4 ตำแหน่ง) — ใช้คิดมูลค่าสต็อกและ COGS
6	**min_stock**	int		0		จุดสั่งซื้อ: คงเหลือ ≤ นี้ = ควรสั่ง (0 = ไม่ตั้ง)
7	**max_stock**	int		0		ระดับเป้าหมายตอนสั่ง (0 = 2 × จุดสั่งซื้อ; ต้อง ≥ จุดสั่งซื้อ)
	`created_at` , `created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, branch_id, product_id)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, product_id) → dt_products (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((avg_cost >= (0)::numeric))`
- **CHECK** `((max_stock >= 0))`
- **CHECK** `((min_stock >= 0))`
- **CHECK** `((stock >= 0))`

- **index** `dt_branch_stock_product_idx` btree (tenant_id, product_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

E `dt_stock_movements` บัตรสต็อก (stock movement)

ทุกการเดินของสินค้า 1 แถวต่อ 1 รายการ พร้อมยอดคงเหลือหลังรายการของ (สาขา, SKU) — หน้าบัตรสต็อก `/inventory/stock-card/:productId` (ยกมา + รับ − จ่าย = คงเหลือ)

วงจรชีวิตของแถว

- เพิ่มโดย `StockLedger` เท่านั้น: รับเข้า, ตัดออก, ขาย, ยกเลิกบิล, ใบลดหนี้, ใบเปลี่ยน, โอนออก / โอนเข้า, รับ / ตัด serial
- **append-only** — ไม่แก้ ไม่ลบ (an22_app มีแค่ INSERT + SELECT)

กฎสำคัญ

- บัตรสต็อกเป็นของสาขาเดียว ไม่รวมข้ามสาขา (ทุนเฉลี่ยของแต่ละสาขาต่างกัน)
- `opening` = ยอดยกมา ไม่ใช่การรับเข้า
- ไม่มี FK ไป SKU — ลบ SKU ที่หมดสต็อกได้ ประวัติอยู่ต่อ

อ้างถึง `dt_branches`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	`bigint`		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	`bigint`			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**branch_id**	`bigint`			FK	สาขาที่สต็อกเดิน → dt_branches
4	**product_id**	`bigint`			ref	SKU ⇢ dt_products.id (ไม่มี FK — ลบ SKU ที่หมดสต็อกได้ ประวัติอยู่ต่อ)
5	**type**	`text`				ชนิด: `opening` ยอดยกมา · `receive` รับเข้า (รวมรับคืนจากบิล) · `issue` จ่ายออก (ขาย / ตัด / ของขาดตอนรับโอน) · `transfer_out` โอนออก · `transfer_in` รับโอน ค่าที่ใช้ได้: `opening` `receive` `issue` `transfer_out` `transfer_in`
6	**qty**	`int`				จำนวน (หน่วยฐาน): + เข้า / − ออก
7	**unit_cost**	`numeric(14,2)`				ทุนต่อหน่วยของรายการนี้ ไม่รวม VAT
8	**total_cost**	`numeric(14,2)`				qty × unit_cost (serial = ผลรวมทุนของเครื่องที่เดิน)
9	**balance_qty**	`int`				คงเหลือหลังรายการ
10	**balance_avg_cost**	`numeric(14,4)`				ทุนเฉลี่ยหลังรายการ (ทศนิยม 4)
11	**balance_value**	`numeric(14,2)`				มูลค่าคงเหลือหลังรายการ
12	**serials**	`text[]`		`'{}'`		serial ที่เดิน (เฉพาะ SKU serial)
13	**note**	`text`		`''`		เหตุผล / อ้างอิง เช่น เลขบิล เลขใบโอน เลขใบส่งของ
14	**created_at**	`timestamptz`				วันเวลาของรายการ (= `StockMovement.date`) — index (สาขา, SKU, เวลา) สำหรับบัตรสต็อกตามช่วงวัน
	`created_by`, `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((type = ANY (ARRAY['opening'::text, 'receive'::text, 'issue'::text, 'transfer_out'::text, 'transfer_in'::text])))`
- **index** `dt_stock_movements_card_idx` btree (tenant_id, branch_id, product_id, created_at, id)

- **index** `dt_stock_movements_product_idx` btree (tenant_id, product_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

E `dt_serial_numbers` serial รายเครื่อง

สินค้าคุม serial 1 เครื่อง = 1 แถวทั้งบริษัท พร้อมทุนของเครื่อง (specific identification) และสาขาที่อยู่ — ประวัติของเครื่องอ่านจาก `dt_stock_movements.serials`

วงจรชีวิตของแถว

- สร้างตอนรับ serial เข้าครั้งแรก
- แก้สถานะ: ขาย → sold, ตัดออก → damaged / other, โอน → in_transit แล้วย้ายสาขาเมื่อรับ, คืน / ยกเลิกบิล → in_stock
- **ไม่ลบ** — รับเครื่องเดิมกลับเข้า (เช่นคืนผู้จำหน่ายแล้วซื้อกลับ) ใช้แถวเดิม เปลี่ยนสถานะ / สาขา / ทุน / SKU ใหม่

กฎสำคัญ

- `serial` ไม่ซ้ำในบริษัท (A-Z 0-9 -)
- ขาย / ตัด / แถม / เปลี่ยนได้เฉพาะเครื่องที่ in_stock ในสาขาของ request (อยู่สาขาอื่น → 400 บอกสาขา)
- in_transit = กำลังโอน ขายไม่ได้ทั้งสองสาขา `branch_id` ยังเป็นต้นทางจนรับ

อ้างถึง `dt_branches`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**product_id**	bigint			ref	SKU ของเครื่อง (ล่าสุด) ⇢ dt_products.id (ไม่มี FK — ประวัติ)
4	**branch_id**	bigint			FK	สาขาที่เครื่องอยู่ (in_stock) หรืออยู่ล่าสุด → dt_branches
5	**serial**	text			UQ	หมายเลขเครื่อง (ตัวใหญ่) ไม่ซ้ำในบริษัท
6	**status**	text				สถานะ: `in_stock` ในคลัง · `in_transit` ระหว่างโอน · `sold` ขายแล้ว · `damaged` เสีย / ตัดจำหน่าย · `other` ตัดออกด้วยเหตุอื่น ค่าที่ใช้ได้: `in_stock` `in_transit` `sold` `damaged` `other`
7	**received_at**	timestamptz				เวลารับเข้าล่าสุด
8	**removed_at**	timestamptz	✓			เวลาที่ออกจากคลัง (null = ยังอยู่)
9	**note**	text		''		หมายเหตุล่าสุด เช่น "รับคืนเข้าคลัง"
10	**cost**	numeric(14,4)				ทุนของเครื่องนี้ ไม่รวม VAT (ทศนิยม 4) — เป็น COGS เมื่อขาย
	`created_at`, `created_by`, `created_by_kind`, `updated_at`, `updated_by`, `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, serial)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((cost >= (0)::numeric))`
- **CHECK** `((serial ~ '^[A-Z0-9-]+$'::text))`
- **CHECK** `((status = ANY (ARRAY['in_stock'::text, 'in_transit'::text, 'sold'::text, 'damaged'::text, 'other'::text])))`
- **index** `dt_serial_numbers_branch_idx` btree (tenant_id, branch_id)
- **index** `dt_serial_numbers_product_idx` btree (tenant_id, product_id, branch_id, status)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

E `dt_transfers` ใบโอนสินค้าระหว่างสาขา

หัวใบโอน: สร้างที่สาขาต้นทาง (ของออกทันที) แล้วสาขาปลายทางรับ 1 ครั้ง หรือ admin ต้นทางยกเลิกขณะยังไม่รับ E26–E29

วงจรชีวิตของแถว

- สร้าง: ผู้ที่ทำงานที่สาขาต้นทาง — เลข `TR-<สาขาต้นทาง>-YYYYMMDD-NNNN`
- แก้: รับ (`in_transit` → `received`) หรือยกเลิก (`in_transit` → `cancelled`) — แถวถูกล็อกก่อนทำ
- **ไม่ลบ** (an22_app ไม่มี DELETE)

กฎสำคัญ

- ต้นทาง ≠ ปลายทาง; สถานะ received ↔ มีเวลา + ผู้รับ, cancelled ↔ มีเวลา + ผู้ยกเลิก + เหตุผล (CHECK)
- SKU ที่อยู่ในใบโอนค้างลบไม่ได้ / เปลี่ยนการคุม serial / ชนิดไม่ได้

อ้างถึง `dt_branches` · ถูกอ้างโดย `dt_transfer_lines`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**transfer_no**	text			UQ	เลขใบโอน (ไม่ซ้ำในบริษัท)
4	**from_branch_id**	bigint			FK	สาขาต้นทาง (ของออก) → dt_branches
5	**to_branch_id**	bigint			FK	สาขาปลายทาง (รับของ) → dt_branches
6	**status**	text				`in_transit` ระหว่างทาง · `received` รับแล้ว · `cancelled` ยกเลิก (ของกลับต้นทางที่ทุนเดิม) ค่าที่ใช้ได้: `in_transit` `received` `cancelled`
7	**note**	text		''		หมายเหตุของผู้สร้าง
8	**created_by_name**	text				ชื่อผู้สร้าง (snapshot — `Transfer.createdBy`)
9	**received_at**	timestamptz	✓			เวลารับ
10	**received_by_name**	text	✓			ชื่อผู้รับ (snapshot)
11	**short_reason**	text		''		เหตุผลที่ของมาไม่ครบ ('' = ครบ) — ส่วนที่ขาดถูกตัดเป็น issue ที่ปลายทาง
12	**cancelled_at**	timestamptz	✓			เวลายกเลิก
13	**cancelled_by_name**	text	✓			ชื่อผู้ยกเลิก (snapshot)
14	**cancel_reason**	text		''		เหตุผลที่ยกเลิก
15	**created_at**	timestamptz				เวลาสร้างใบ (= `Transfer.createdAt`)
	`created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, transfer_no)
- **FK** (tenant_id, from_branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **FK** (tenant_id, to_branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((from_branch_id <> to_branch_id))`
- **CHECK** `(((status = 'cancelled'::text) = ((cancelled_at IS NOT NULL) AND (cancelled_by_name IS NOT NULL) AND (btrim(cancel_reason) <> ''::text))))`
- **CHECK** `(((status = 'received'::text) = ((received_at IS NOT NULL) AND (received_by_name IS NOT NULL))))`
- **CHECK** `((status = ANY (ARRAY['in_transit'::text, 'received'::text, 'cancelled'::text])))`
- **index** `dt_transfers_from_branch_idx` btree (tenant_id, from_branch_id)
- **index** `dt_transfers_to_branch_idx` btree (tenant_id, to_branch_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIU** · an22_platform **SIUD**

E `dt_transfer_lines` บรรทัดของใบโอน

สินค้าแต่ละ SKU ในใบโอน: จำนวน / serial ที่ส่ง ทุนโอน และจำนวน / serial ที่รับจริง

วงจรชีวิตของแถว

- สร้างพร้อมหัวใบ
- แก้ตอนรับ (`received_qty` , `received_serials`)
- ไม่ลบ

กฎสำคัญ

- 1 SKU ต่อ 1 บรรทัดในใบ (unique)
- `received_qty` อยู่ระหว่าง 0 ถึง qty (CHECK)

อ้างถึง `dt_transfers`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**transfer_id**	bigint			PK FK	ใบโอน → dt_transfers
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**product_id**	bigint			ref UQ	SKU ⇢ dt_products.id (ไม่มี FK — ชื่อสินค้าเป็น snapshot)
5	**sku**	text				snapshot รหัส SKU ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
6	**name**	text				snapshot ชื่อสินค้า ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
7	**unit**	text				snapshot หน่วยฐาน ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
8	**qty**	int				จำนวนที่ส่ง (หน่วยฐาน)
9	**serials**	text[]		'{}'		serial ที่ส่ง (SKU serial: qty = จำนวน serial)
10	**unit_cost**	numeric(14,4)				ทุนโอนต่อหน่วยฐาน ไม่รวม VAT (ปกติ = ทุนเฉลี่ยต้นทางขณะโอน; serial = เฉลี่ยทุนของเครื่อง) — ปลายทางรับเข้าด้วยทุนนี้
11	**received_qty**	int	✓			จำนวนที่รับจริง (null = ยังไม่รับ)
12	**received_serials**	text[]		'{}'		serial ที่รับจริง
	`created_at` , `created_by` , `created_by_kind` , `updated_at` , `updated_by` , `updated_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (tenant_id, transfer_id, position)
- **UNIQUE** (tenant_id, transfer_id, product_id)
- **FK** (tenant_id, transfer_id) → dt_transfers (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `(((received_qty >= 0) AND (received_qty <= qty)))`
- **CHECK** `(("position" >= 0))`
- **CHECK** `((qty > 0))`
- **CHECK** `((unit_cost >= (0)::numeric))`
- **index** `dt_transfer_lines_product_idx` btree (tenant_id, product_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIU** · an22_platform **SIUD**

F กะขายและ POS

กะขาย (1 เครื่อง POS / ลิ้นชัก + 1 แคชเชียร์) กับทุกอย่างที่นับเงินในกะ: ยอดนับตอนปิด, เงินเข้า / ออกลิ้นชัก, เปิดลิ้นชักโดยไม่ขาย, ปรับปรุงยอดหลังปิด และบิลที่พักไว้ของเครื่อง

F `dt_shifts` กะขาย

กะของเครื่อง POS 1 เครื่อง + แคชเชียร์ 1 คน ตั้งแต่เปิด (เงินทอนตั้งต้น) ถึงปิด (นับเงินแบบไม่เห็นยอด) — เงินทุกบาทนับในกะที่มันเกิด (บิล / void / คืนเงิน / เงินเข้าออก)

วงจรชีวิตของแถว

- สร้าง: เปิดกะที่ POS (`POST /shifts/open`) — เลข `SH-<สาขา>-<วันเปิด>-NN`
- แก้: ล็อก / ปลดล็อกหน้าจอ, ปิดกะ (status, ยอดฝาก, เงินคงลิ้นชัก, สถานะอนุมัติ), admin อนุมัติภายหลัง
- **ไม่ลบ**; ปรับยอดหลังปิดเป็นแถวใหม่ใน `dt_shift_adjustments` (กะเดิมไม่เปลี่ยน)

กฎสำคัญ

- เปิดได้ 1 กะต่อเครื่อง และ 1 กะต่อคน (partial unique index `WHERE status = 'open'` — ตาข่ายเมื่อเปิดพร้อมกัน)
- closed ↔ มีเวลาปิด + ผู้ปิด; ล็อกได้เฉพาะกะที่เปิด (CHECK)
- กะเป็นของวันที่เปิด; เปิดนานเกิน `shift_max_hours` เตือนเท่านั้น

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- `counts` จาก `dt_shift_counts`
- `adjustmentTotal` = ผลรวม `dt_shift_adjustments`
- ยอดที่ระบบคาดต่อช่องทาง (`expectedByMethod()`) คิดจากบิล / void / ใบลดหนี้ / เงินเข้าออกของกะทุกครั้งที่อ่าน

อ้างถึง `dt_branches` , `dt_pos_terminals` · ถูกอ้างโดย `dt_cash_movements` , `dt_credit_notes` , `dt_drawer_opens` , `dt_exchanges` , `dt_sales` , `dt_shift_adjustments` , `dt_shift_counts`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**shift_no**	text			UQ	เลขกะ (ไม่ซ้ำในบริษัท)
4	**terminal_id**	int			FK	เครื่อง POS ของกะ → dt_pos_terminals
5	**branch_id**	bigint			FK	สาขาของเครื่องตอนเปิดกะ E24 → dt_branches
6	**pos_id**	text		`''`		POS ID ของเครื่องตอนเปิด (snapshot — พิมพ์บน Z report)
7	**cashier_user_id**	bigint			ref	ผู้เปิด = เจ้าของกะ / ผู้รับผิดชอบลิ้นชัก (ใช้ตรวจสิทธิ์; ไม่มี FK) ⇢ dt_users.id (ไม่มี FK)
8	**cashier**	text				ชื่อแคชเชียร์ (snapshot สำหรับแสดง / รายงาน)
9	**status**	text				`open` เปิดอยู่ · `closed` ปิดแล้ว ค่าที่ใช้ได้: `open` `closed`
10	**opened_at**	timestamptz		`now()`		เวลาเปิด — กะเป็นของวันท้องถิ่นนี้
11	**opening_float**	numeric(14,2)				เงินทอนตั้งต้นในลิ้นชัก (บาท)
12	**opening_denominations**	jsonb		`'[]'`		รายการแบงก์ / เหรียญตอนเปิด `[{ value, qty }]` (jsonb)
13	**closed_at**	timestamptz	✓			เวลาปิด
14	**closed_by**	text	✓			ผู้ปิด: แคชเชียร์ หรือ admin ที่ปิดแทน (ชื่อ)
15	**cash_denominations**	jsonb		`'[]'`		รายการแบงก์ / เหรียญที่นับตอนปิด (jsonb)
16	**deposit_amount**	numeric(14,2)		`0`		เงินสดที่นำออกจากลิ้นชักตอนปิด (เข้าตู้เซฟ / ธนาคาร)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
17	**float_left**	numeric(14,2)		0		เงินที่เหลือในลิ้นชัก = เงินทอนตั้งต้นที่เสนอให้กะถัดไป
18	**approval_status**	text		'not_needed'		not_needed ไม่ต้องอนุมัติ · pending รออนุมัติ (ส่วนต่างเกินเพดาน) · approved อนุมัติแล้ว ค่าที่ใช้ได้: not_needed pending approved
19	**approved_by**	text	✓			ผู้อนุมัติ (ชื่อ)
20	**approved_at**	timestamptz	✓			เวลาอนุมัติ
21	**note**	text		''		หมายเหตุตอนปิด
22	**locked_at**	timestamptz	✓			หน้าจอ POS ของกะนี้ถูกล็อกตั้งแต่ (null = ไม่ล็อก) — ล็อกอยู่ขายไม่ได้จนแคชเชียร์ / admin ปลดล็อก E48
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, shift_no)
- **UNIQUE index** dt_shifts_open_cashier_key btree (tenant_id, cashier_user_id) WHERE (status = 'open'::text)
- **UNIQUE index** dt_shifts_open_terminal_key btree (tenant_id, terminal_id) WHERE (status = 'open'::text)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **FK** (tenant_id, terminal_id) → dt_pos_terminals (tenant_id, id) ON DELETE RESTRICT
- **CHECK** ((approval_status = ANY (ARRAY['not_needed'::text, 'pending'::text, 'approved'::text])))
- **CHECK** (((status = 'closed'::text) = ((closed_at IS NOT NULL) AND (closed_by IS NOT NULL))))
- **CHECK** ((deposit_amount >= (0)::numeric))
- **CHECK** (((status = 'open'::text) OR (locked_at IS NULL)))
- **CHECK** ((opening_float >= (0)::numeric))
- **CHECK** ((status = ANY (ARRAY['open'::text, 'closed'::text])))
- **index** dt_shifts_branch_idx btree (tenant_id, branch_id, opened_at)
- **index** dt_shifts_terminal_idx btree (tenant_id, terminal_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SIU** · an22_platform **SIUD**

F dt_shift_counts ยอดนับตอนปิดกะ

ผลการตรวจแต่ละช่องทางตอนปิดกะ (นับ / ยืนยัน / ไม่ตรวจ) เทียบกับยอดที่ระบบคาด — ใช้หาส่วนต่าง เหตุผล และการอนุมัติ

วงจรชีวิตของแถว

- เพิ่มครั้งเดียวตอนปิดกะ เรียงตามลำดับของ expectedByMethod()
- **append-only**

กฎสำคัญ

- ส่วนต่างไม่เป็น 0 ต้องมีเหตุผล; นับใหม่ได้ 1 ครั้ง (ยอดแรกเก็บใน first_count)

อ้างถึง dt_shifts

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT
2	**shift_id**	bigint			PK FK	กะ → dt_shifts
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**method_id**	bigint			ref	ช่องทางชำระ (ทุกช่องทางเงินสดรวมเป็นบรรทัดลิ้นชักเดียว) ⇢ dt_payment_methods.id (ไม่มี FK — ชื่อช่องทางเป็น snapshot)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
5	**name**	text				snapshot ชื่อช่องทาง ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
6	**expected**	numeric(14,2)				ยอดที่ระบบคาด ณ ตอนปิด (snapshot)
7	**counted**	numeric(14,2)	✓			ยอดที่นับได้ครั้งสุดท้าย (null = ชนิด confirm / none — ไม่มีส่วนต่าง); ช่องทางไม่ใช่เงินสดติดลบได้เมื่อคืนเงินมากกว่ารับ
8	**close_check**	text				วิธีตรวจของช่องทางตอนปิด (snapshot): `count` · `confirm` · `none` ค่าที่ใช้ได้: `count` `confirm` `none`
9	**confirmed_by**	text	✓			ผู้ติ๊ก "ตรวจแล้ว" (ชนิด confirm)
10	**first_count**	numeric(14,2)	✓			ยอดนับครั้งแรก เมื่อนับใหม่หลังเห็นส่วนต่าง
11	**reason_code**	text	✓			เหตุของส่วนต่าง: `wrong_change` ทอนผิด · `wrong_received` รับเงินผิด · `wrong_method` กดช่องทางผิด · `unknown` ไม่ทราบ · `other` ค่าที่ใช้ได้: `wrong_change` `wrong_received` `wrong_method` `unknown` `other`
12	**reason**	text		''		คำอธิบายเพิ่ม
	`created_at`, `created_by`, `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (tenant_id, shift_id, position)
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((close_check = ANY (ARRAY['count'::text, 'confirm'::text, 'none'::text])))`
- **CHECK** `(("position" >= 0))`
- **CHECK** `((reason_code = ANY (ARRAY['wrong_change'::text, 'wrong_received'::text, 'wrong_method'::text, 'unknown'::text, 'other'::text])))`

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

F `dt_cash_movements` เงินเข้า / ออกลิ้นชักระหว่างกะ

ใส่เงินเพิ่ม / นำเงินออกจากลิ้นชักโดยไม่ใช่การขาย เช่น แลกเงินทอน จ่ายค่าส่งของ — นับในยอดเงินสดที่คาดของกะ

วงจรชีวิตของแถว

- เพิ่มที่ POS (แคชเชียร์ของกะ หรือ admin)
- **append-only**

กฎสำคัญ

- ต้องมีเหตุผล; ยอด > 0 (CHECK) — ทิศทางอยู่ใน `type`
- เงินออกเกินเงินสดที่คาดในลิ้นชักไม่ได้

อ้างถึง `dt_shifts`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**shift_id**	bigint			FK	กะ → dt_shifts
4	**type**	text				`in` ใส่เงินเข้า · `out` นำเงินออก ค่าที่ใช้ได้: `in` `out`
5	**amount**	numeric(14,2)				จำนวนเงิน (บาท, > 0)
6	**reason**	text				เหตุผล

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
7	**by_name**	text				ผู้ทำ (ชื่อ — CashMovement.by)
8	**created_at**	timestamptz				เวลาที่ทำ (= CashMovement.at)
	created_by, created_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ created_*					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** ((amount > (0)::numeric))
- **CHECK** ((btrim(reason) <> ''::text))
- **CHECK** ((type = ANY (ARRAY['in'::text, 'out'::text])))
- **index** dt_cash_movements_shift_idx btree (tenant_id, shift_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

F dt_drawer_opens เปิดลิ้นชักโดยไม่มีการขาย

บันทึกทุกครั้งที่เปิดลิ้นชักโดยไม่ขาย (ไม่มีเงินเข้าออก) เพื่อการควบคุม — นับจำนวนใน X / Z report และรายงานส่วนต่าง S7

วงจรชีวิตของแถว

- เพิ่มที่ POS (เมนูกะ → เปิดลิ้นชัก)
- **append-only**

กฎสำคัญ

- ต้องมีเหตุผล; ไม่กระทบยอดเงินที่คาด

อ้างถึง dt_shifts

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — id ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies
3	**shift_id**	bigint			FK	กะ → dt_shifts
4	**reason**	text				เหตุผลที่เปิด
5	**by_name**	text				ผู้เปิด (ชื่อ)
6	**created_at**	timestamptz				เวลาที่เปิด (= DrawerOpen.at)
	created_by, created_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ created_*					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** ((btrim(reason) <> ''::text))
- **index** dt_drawer_opens_shift_idx btree (tenant_id, shift_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

F dt_shift_adjustments ปรับปรุงยอดกะที่ปิดแล้ว

admin แก้ยอดของกะที่ปิดแล้ว (นับผิด, เจอเงิน, ยอดบัตรเข้าช้า) เป็นรายการแยก — ยอดนับเดิม Z report เดิม และสถานะอนุมัติไม่เปลี่ยน S8

วงจรชีวิตของแถว

- เพิ่มที่หน้ากะ /sales/shifts/:id (admin)

- **append-only** — ไม่แก้ / ลบ

กฎสำคัญ

- ปรับได้เฉพาะช่องทางที่ถูกนับในกะนั้น; ยอด ≠0 (CHECK)
- ส่วนต่างหลังปรับ = `adjustedVariance()`

อ้างถึง `dt_shifts`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**shift_id**	bigint			FK	กะที่ปิดแล้ว → dt_shifts
4	**method_id**	bigint			ref	ช่องทางที่ปรับ ⇢ dt_payment_methods.id (ไม่มี FK)
5	**name**	text				snapshot ชื่อช่องทาง ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
6	**amount**	numeric(14,2)				+ มากกว่าที่นับ (เช่นเจอเงิน) / − น้อยกว่า
7	**reason_code**	text				เหตุ: `count_error` นับผิด · `found_cash` เจอเงิน · `late_settlement` ยอดเข้าช้า · `other` ค่าที่ใช้ได้: `count_error` `found_cash` `late_settlement` `other`
8	**reason**	text		`''`		คำอธิบาย
9	**by_name**	text				ผู้ปรับ (ชื่อ)
10	**created_at**	timestamptz				เวลาที่ปรับ (= `ShiftAdjustment.at`)
	`created_by`, `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **CHECK** `((amount <> (0)::numeric))`
- **CHECK** `((reason_code = ANY (ARRAY['count_error'::text, 'found_cash'::text, 'late_settlement'::text, 'other'::text])))`
- **index** `dt_shift_adjustments_shift_idx` btree (tenant_id, shift_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

F `dt_held_bills` บิลที่พักไว้ของ POS

ตะกร้าที่แคชเชียร์กด "พักบิล" ไว้เรียกคืนทีหลัง ต่อสาขา + เครื่อง — เก็บที่ server (ไม่ใช่ localStorage) เพราะอาจมีเลขภาษี / ที่อยู่ผู้ซื้อ E48

วงจรชีวิตของแถว

- เพิ่มตอนพักบิล
- ลบเมื่อเรียกคืน / ทิ้ง (เป็นของชั่วคราว — ตารางเดียวในกลุ่มนี้ที่ลบแถวได้)

กฎสำคัญ

- ทั้งก้อนเข้ารหัส (AES-256-GCM, AAD `dt_held_bills.payload:<tenant>`) — DB อ่านเนื้อหาไม่ได้
- ไม่มี FK ไปเครื่อง POS (ของชั่วคราว)

อ้างถึง `dt_branches`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies
3	**branch_id**	bigint			FK	สาขาที่พัก → dt_branches
4	**terminal_id**	int			ref	เครื่องที่พัก (รายการบิลพักแสดงเฉพาะของเครื่องนี้) ⇢ dt_pos_terminals.id (ไม่มี FK — ของชั่วคราว)
5	**held_at**	timestamptz				เวลาที่พัก
6	**payload_enc**	bytea				ตะกร้าทั้งก้อน (รายการ, ผู้ซื้อ, ส่วนลด) แบบเข้ารหัส
	created_at, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					
7	**created_by**	bigint	✓			ผู้พักบิล (stamp — HeldBill.createdBy)

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **index** dt_held_bills_branch_idx btree (tenant_id, branch_id, terminal_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SIUD** · an22_platform **SIUD**

G เอกสารขาย

บิลขาย POS กับเอกสารที่อ้างบิล: ใบกำกับภาษีเต็มรูป, ใบลดหนี้ (คืนสินค้า + คืนเงิน) และใบเปลี่ยนสินค้า — **ทุกเอกสารห้ามลบ** บิลไม่เปลี่ยนหลังขาย (มีแต่สถานะ void); ชื่อสินค้า / ราคา / ช่องทางชำระในเอกสารเป็น snapshot

G dt_sales บิลขาย (POS)

หัวบิลขายหน้าร้าน 1 แถว = 1 ใบเสร็จ / ใบกำกับอย่างย่อ — ยอดรวม ส่วนลด VAT การปัดเศษ เงินทอน สถานะ void และ snapshot ของสาขา / เครื่อง / กะ / สถานะ VAT ตอนขาย

วงจรชีวิตของแถว

- สร้าง: POST /sales หลังตรวจ 8 ขั้น (บทที่ 4) — เลข POS-<สาขา>-<เครื่อง>-YYYYMMDD-NNNN
- แก้ได้เฉพาะ: ยกเลิกบิลวันเดียวกัน (status = cancelled + void_*), ผูกเลขใบกำกับเต็มรูป (tax_invoice_no) — **ยอด / บรรทัดไม่เปลี่ยนตลอดไป**
- คืนของวันถัดไปเป็นใบลดหนี้ เปลี่ยนสินค้าเป็นใบเปลี่ยน (บิลเดิมไม่เปลี่ยน); **ไม่ลบ**

กฎสำคัญ

- void / ใบลดหนี้ / ใบเปลี่ยนทำได้เฉพาะที่สาขาที่ขาย D3
- voided_at มีค่าได้เฉพาะ status = cancelled (CHECK)
- ร้านไม่จด VAT ตอนขาย (vat_registered = false) → VAT ทุกบรรทัด = 0 ออกใบกำกับเต็มรูปไม่ได้ และไม่อยู่ในรายงานภาษีขาย

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- lines / payments จากตารางลูก
- returnStatus (none / partial / full), creditedTotal , creditNoteNos จาก dt_credit_notes
- exchangeNos จาก dt_exchanges

อ้างถึง dt_branches , dt_shifts , dt_pos_terminals · ถูกอ้างโดย dt_credit_notes , dt_exchanges , dt_sale_lines , dt_sale_payments , dt_tax_invoices

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — id ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
3	**order_no**	text			UQ	เลขบิล (ไม่ซ้ำในบริษัท) — เลขเก่าก่อนมีสาขา / เครื่องไม่เปลี่ยน
4	**sold_at**	timestamptz		now()		วันเวลาขาย (= Sale.date) — จุดความรับผิดทางภาษี
5	**cashier**	text		''		ชื่อแคชเชียร์ (snapshot)
6	**customer**	text		''		ชื่อลูกค้าที่พิมพ์ ('' = ลูกค้าทั่วไป)
7	**subtotal**	numeric(14,2)				Σ ราคาต่อหน่วย × จำนวน ก่อนส่วนลด
8	**item_discount**	numeric(14,2)		0		ส่วนลดโปรระดับสินค้ารวม
9	**bill_discount**	numeric(14,2)		0		ส่วนลดโปรท้ายบิล
10	**manual_discount**	numeric(14,2)		0		ส่วนลดพิเศษรวม (ต่อบรรทัด + ท้ายบิล) กี่ให้กี่ POS หลังโปรทั้งหมด
11	**total**	numeric(14,2)				ยอดสุทธิรวม VAT (= Σ amount ของบรรทัด)
12	**vat**	numeric(14,2)		0		VAT ที่รวมอยู่ใน total
13	**rounding**	numeric(14,2)		0		ปัดเศษเงินสด (+ ลูกค้าจ่ายมากกว่า total / − น้อยกว่า)
14	**change**	numeric(14,2)		0		เงินทอน
15	**bill_promotion_ids**	bigint[]		'{}'		โปรท้ายบิลที่ใช้ (array ของ dt_promotions.id)
16	**item_count**	int		0		จำนวนชิ้นรวม (หน่วยฐาน รวมของแถม)
17	**status**	text				pending (ยังไม่ใช้) · paid ชำระแล้ว · cancelled ยกเลิก (void) ค่าที่ใช้ได้: pending paid cancelled
18	**voided_at**	timestamptz	✓			เวลายกเลิก
19	**void_reason**	text		''		เหตุผลที่ยกเลิก
20	**voided_by**	text	✓			ผู้ยกเลิก (ชื่อ)
21	**void_approved_by**	text	✓			admin ที่อนุมัติให้พนักงานยกเลิก (null = admin ยกเลิกเอง)
22	**tax_invoice_no**	text	✓			เลขใบกำกับเต็มรูปที่ใช้ได้ของบิล (null = ไม่มี) — server ตั้งเมื่อออก / ออกใหม่
23	**manual_discount_reason**	text		''		เหตุผลของส่วนลดพิเศษ (1 ข้อต่อบิล; '' = ไม่มี)
24	**manual_discount_by**	text		''		ผู้ให้ส่วนลดพิเศษ (ชื่อ)
25	**shift_id**	bigint	✓		FK	กะที่ขาย (null = ไม่มีกะ: ก่อนมีกะ หรือปิด require_shift) → dt_shifts
26	**void_shift_id**	bigint	✓		FK	กะที่ยกเลิก — เงินออกจากลิ้นชักของกะนั้น → dt_shifts
27	**branch_id**	bigint			FK	สาขาที่ขาย E12 → dt_branches
28	**terminal_id**	int	✓		FK	เครื่อง POS ที่ขาย → dt_pos_terminals
29	**pos_id**	text		''		POS ID ของเครื่องตอนขาย (snapshot — พิมพ์บนใบเสร็จ)
30	**vat_registered**	bool		true		ร้านจด VAT ตอนขาย (snapshot) B20
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, order_no)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **FK** (tenant_id, terminal_id) → dt_pos_terminals (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, void_shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((status = ANY (ARRAY['pending'::text, 'paid'::text, 'cancelled'::text])))`
- **CHECK** `(((voided_at IS NULL) OR (status = 'cancelled'::text)))`

- **index** `dt_sales_branch_idx` btree (tenant_id, branch_id, sold_at)
- **index** `dt_sales_shift_idx` btree (tenant_id, shift_id)
- **index** `dt_sales_terminal_idx` btree (tenant_id, terminal_id)
- **index** `dt_sales_void_shift_idx` btree (tenant_id, void_shift_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SIU** · an22_platform **SIUD**

G `dt_sale_lines` บรรทัดบิล

สินค้าแต่ละบรรทัดของบิล: snapshot ของสินค้า ราคา ส่วนลดแยกชนิด VAT serial และต้นทุนขาย (COGS) ณ ตอนขาย

วงจรชีวิตของแถว

- สร้างพร้อมบิล
- **append-only** — ไม่แก้ ไม่ลบ

กฎสำคัญ

- อ้างด้วย `line_index` (ใบลดหนี้ / ใบเปลี่ยนอ้าง `sale_line_index`) — ลำดับต้องคงที่
- `amount` = unit_price × qty − item_discount − bill_discount − manual_discount
- serial SKU: 1 บรรทัดต่อ 1 serial (qty = 1); ของแถม = บรรทัดราคา 0 ที่ยังตัดสต็อกและมี COGS
- เครื่องที่ลูกค้าถืออยู่หลังใบเปลี่ยน = `currentSerial()` ไม่ใช่ `serial` ของบรรทัด

อ้างถึง `dt_sales`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**sale_id**	bigint			PK FK	บิล → dt_sales
3	**line_index**	int			PK	ลำดับบรรทัด เริ่ม 0 (คงที่)
4	**product_id**	bigint			ref	SKU ⇢ dt_products.id (ไม่มี FK — snapshot)
5	**sku**	text				snapshot รหัส SKU ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
6	**name**	text				snapshot ชื่อสินค้า ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
7	**short_name**	text				ชื่อบนใบเสร็จ (snapshot; ว่างใช้ name)
8	**unit**	text				หน่วยที่ขาย (หน่วยฐานหรือหน่วยแพ็ก)
9	**factor**	int				หน่วยฐานต่อหน่วยที่ขาย (1 = หน่วยฐาน) — สต็อกเดิน qty × factor
10	**qty**	int				จำนวน (หน่วยที่ขาย)
11	**unit_price**	numeric(14,2)				ราคาต่อหน่วยก่อนส่วนลด (ของแถม = 0)
12	**list_price**	numeric(14,2)				ราคาขายต่อหน่วย ณ วันขายแม้เป็นของแถม — ใช้หักของแถมที่ไม่ได้สิทธิ์ในใบลดหนี้
13	**item_discount**	numeric(14,2)				ส่วนลดโปรสินค้าทั้งบรรทัด
14	**bill_discount**	numeric(14,2)				ส่วนแบ่งของส่วนลดท้ายบิล
15	**manual_discount**	numeric(14,2)				ส่วนลดพิเศษ: ของบรรทัด + ส่วนแบ่งของท้ายบิล
16	**amount**	numeric(14,2)				ยอดสุทธิของบรรทัด รวม VAT
17	**vat_type**	text				`vat7` · `exempt` (snapshot) ค่าที่ใช้ได้: `vat7` `exempt`
18	**item_type**	text				`stock` · `service` (snapshot — บริการไม่เดินสต็อก) ค่าที่ใช้ได้: `stock` `service`
19	**vat**	numeric(14,2)				VAT ที่รวมอยู่ใน amount

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
20	**promotion_ids**	bigint[]		'{}'		โปรสินค้าที่ใช้กับบรรทัดนี้ (บรรทัดของแถม = โปรของแถม)
21	**free_of_promotion_id**	bigint	✓		ref	บรรทัดของแถมของโปรนี้ (null = ไม่ใช่ของแถม) — อ่านด้วย `isFreeLine()` ⇢ dt_promotions.id (ไม่มี FK)
22	**serial**	text	✓			serial ที่ขาย (SKU serial เท่านั้น)
23	**warranty_months**	int		0		ประกันกี่เดือน (snapshot) — นับจากวันขายแม้เปลี่ยนเครื่อง
24	**cogs**	numeric(14,2)				ต้นทุนขายทั้งบรรทัด ไม่รวม VAT (serial = ทุนเครื่อง, ปกติ = ทุนเฉลี่ยขณะขาย, บริการ = ทุนมาตรฐาน)
	`created_at`, `created_by`, `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (tenant_id, sale_id, line_index)
- **FK** (tenant_id, sale_id) → dt_sales (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((factor >= 1))`
- **CHECK** `((item_type = ANY (ARRAY['stock'::text, 'service'::text])))`
- **CHECK** `((line_index >= 0))`
- **CHECK** `((qty >= 1))`
- **CHECK** `((vat_type = ANY (ARRAY['vat7'::text, 'exempt'::text])))`
- **index** `dt_sale_lines_product_idx` btree (tenant_id, product_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G dt_sale_payments การชำระของบิล

บิลแบ่งจ่ายได้หลายช่องทาง 1 แถวต่อ 1 บรรทัดการชำระ

วงจรชีวิตของแถว

- สร้างพร้อมบิล
- **append-only**

กฎสำคัญ

- มีแต่เงินสดที่ tendered เกิน amount ได้ (ส่วนเกิน = เงินทอน) และปัดเศษเฉพาะส่วนเงินสด
- **ห้ามเก็บเลขบัตรเต็ม** — reference เป็นเลขอนุมัติ / 4 หลักท้าย (เลขบัตรเต็มถูกปฏิเสธ 400 ก่อนถึง DB, PCI DSS)

อ้างถึง `dt_sales`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**sale_id**	bigint			PK FK	บิล → dt_sales
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**method_id**	bigint			ref	ช่องทางชำระ ⇢ dt_payment_methods.id (ไม่มี FK — snapshot)
5	**name**	text				snapshot ชื่อช่องทาง ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
6	**type**	text				ชนิดของช่องทาง (snapshot)
7	**amount**	numeric(14,2)				ยอดที่บรรทัดนี้ชำระ (เงินสด: หลังปัดเศษ ไม่รวมเงินทอน)
8	**tendered**	numeric(14,2)				เงินที่ลูกค้ายื่น (เงินสด ≥ amount; อื่น = amount)
9	**reference**	text		''		เลขอ้างอิง เช่น เลขอนุมัติบัตร
10	**installment_months**	int	✓			จำนวนเดือนผ่อน (ชนิดผ่อนเท่านั้น)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
	`created_at` , `created_by` , `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (tenant_id, sale_id, position)
- **FK** (tenant_id, sale_id) → dt_sales (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `(("position" >= 0))`
- **index** `dt_sale_payments_method_idx` btree (tenant_id, method_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G dt_tax_invoices ใบกำกับภาษีเต็มรูป

ใบกำกับเต็มรูปที่ออกตามคำขอแทนใบกำกับอย่างย่อ — ออกพร้อมการขาย หรือภายหลัง (ลงวันที่ / เลขตามวันขาย); ข้อมูลผู้ซื้อเข้ารหัส

วงจรชีวิตของแถว

- สร้าง: พร้อมบิล (`at_sale`) หรือหน้า `/sales/:id/tax-invoice` — เลข `INV-<สาขาของบิล>-<วันขาย>-NNNN`
- แก้: ยกเลิกเมื่อ void บิล หรือ admin ยกเลิกแล้วออกใหม่ (ข้อมูลผู้ซื้อผิด) — ใบเดิมได้ `cancelled_at` + `replaced_by_no` , ใบใหม่ได้ `replaces_invoice_no`
- **ไม่ลบ ไม่แก้ข้อมูลผู้ซื้อ**

กฎสำคัญ

- บิลมีใบที่ใช้ได้ (ยังไม่ยกเลิก) ได้ 1 ใบ (partial unique `WHERE cancelled_at IS NULL`)
- ออกได้เฉพาะบิลที่ชำระแล้วของร้านที่จด VAT ตอนขาย; ยอดเงินมาจากบิลเสมอ

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- ยอดเงิน / VAT ของใบคิดจากบิล (`invoiceTotals()`) ไม่ได้เก็บซ้ำ

อ้างถึง `dt_branches` , `dt_sales`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — `id` ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT → dt_companies
3	**invoice_no**	text			UQ	เลขใบกำกับ (ไม่ซ้ำในบริษัท)
4	**sale_id**	bigint			FK	บิลที่ใบนี้แทนใบกำกับอย่างย่อ → dt_sales
5	**order_no**	text				เลขบิล (snapshot)
6	**sale_date**	timestamptz				วันเวลาขายของบิล
7	**date**	timestamptz				วันที่ของเอกสาร = เวลาขาย (จุดความรับผิด) แม้ออกทีหลัง
8	**issued_at**	timestamptz				เวลาที่ออกจริง (ตรวจสอบย้อนหลัง)
9	**buyer_enc**	bytea				ข้อมูลผู้ซื้อทั้งก้อน (ชื่อ, ชนิดเลข, เลขผู้เสียภาษี / passport, สำนักงานใหญ่ / สาขา, ที่อยู่) แบบเข้ารหัส AES-256-GCM
10	**buyer_tax_id_hash**	bytea				HMAC ของเลขผู้เสียภาษี / passport — ค้นผู้ซื้อเดิมได้โดยไม่ถอดรหัส
11	**issued_by**	text				ผู้ออก (ชื่อ)
12	**cancelled_at**	timestamptz	✓			เวลายกเลิก (null = ใช้ได้)
13	**cancel_reason**	text		`''`		เหตุผลที่ยกเลิก ('' ขณะใช้ได้)
14	**replaces_invoice_no**	text	✓			ใบที่ถูกยกเลิกซึ่งใบนี้ออกแทน (null = ใบแรก)
15	**replaced_by_no**	text	✓			ใบที่ออกแทนใบนี้ (เมื่อใบนี้ถูกยกเลิกเพื่อออกใหม่)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
16	**at_sale**	bool				ออกพร้อมการขาย (true) หรือออกทีหลังแทนใบอย่างย่อ (false) — มีผลกับรายงานภาษีขาย
17	**branch_id**	bigint			FK	สาขาของบิล = สถานประกอบการบนใบ และรายงานภาษีขายที่ใบนี้อยู่ E25 → dt_branches
	created_at, created_by, created_by_kind, updated_at, updated_by, updated_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4)					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, invoice_no)
- **UNIQUE index** dt_tax_invoices_valid_key btree (tenant_id, sale_id) WHERE (cancelled_at IS NULL)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, sale_id) → dt_sales (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **index** dt_tax_invoices_branch_idx btree (tenant_id, branch_id, date)
- **index** dt_tax_invoices_buyer_idx btree (tenant_id, buyer_tax_id_hash)
- **index** dt_tax_invoices_sale_idx btree (tenant_id, sale_id)

trigger: stamp (BEFORE INSERT OR UPDATE) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SIU** · an22_platform **SIUD**

G dt_credit_notes ใบลดหนี้ (คืนสินค้า)

คืนสินค้าของบิลที่ชำระแล้วตั้งแต่วันถัดจากวันขาย (วันเดียวกันใช้ void) — คืนเงิน, รับของกลับเข้าคลังหรือตัดเสีย และกลับรายการ VAT; บิลเดิมไม่เปลี่ยน

วงจรชีวิตของแถว

- สร้าง: หน้า /sales/:id/credit-note (admin) — เลข CN-<สาขา>-YYYYMMDD-NNNN
- **append-only ทั้งใบ** — ออกแล้วไม่แก้ ไม่ลบ

กฎสำคัญ

- คืนต่อบรรทัด = ส่วนของ amount ที่ลูกค้าจ่ายจริง; ของแถมคิดสิทธิ์ใหม่จากของที่ลูกค้าเก็บไว้ (เกินสิทธิ์ → หักใน dt_credit_note_deductions)
- คืนเงินเป็นเงินสด หรือผ่านช่องทางที่บิลจ่าย ไม่เกินที่ช่องทางนั้นรับ − ที่คืนไปแล้ว
- ผู้ซื้อ = snapshot ของใบกำกับที่ใช้ได้ตอนออก เข้ารหัสแบบเดียวกับใบกำกับ

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- บิลได้ returnStatus / creditedTotal / creditNoteNos จากใบลดหนี้ทั้งหมดของบิล

อ้างถึง dt_branches, dt_sales, dt_shifts, dt_pos_terminals · ถูกอ้างโดย dt_credit_note_deductions, dt_credit_note_lines, dt_credit_note_refunds

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — id ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies
3	**cn_no**	text			UQ	เลขใบลดหนี้ (ไม่ซ้ำในบริษัท)
4	**sale_id**	bigint			FK	บิลที่คืน → dt_sales
5	**order_no**	text				เลขบิล (snapshot)
6	**tax_invoice_no**	text	✓			เลขใบกำกับเต็มรูปที่ใช้ได้ตอนออก (null = มีแต่ใบกำกับอย่างย่อ) — ไม่เปลี่ยนแม้ใบกำกับถูกออกใหม่ทีหลัง
7	**buyer_enc**	bytea	✓			ผู้ซื้อของใบกำกับนั้น (เข้ารหัส; null = ไม่มีใบกำกับเต็มรูป)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
8	**sale_date**	timestamptz				วันเวลาขายของบิล
9	**date**	timestamptz		now()		วันเวลาออกใบลดหนี้
10	**cashier**	text				ผู้ออก (ชื่อ)
11	**reason**	text				เหตุผลที่คืน
12	**subtotal**	numeric(14,2)				Σ ยอดคืนของบรรทัด
13	**deduction**	numeric(14,2)				Σ ยอดหักของแถมที่เกินสิทธิ์
14	**total**	numeric(14,2)				ยอดคืนเงิน = subtotal − deduction (≥ 0)
15	**vat**	numeric(14,2)				VAT ที่กลับรายการ
16	**shift_id**	bigint	✓		FK	กะที่จ่ายเงินคืน (null = ไม่มีกะ) → dt_shifts
17	**branch_id**	bigint			FK	สาขาที่ออก = รับของคืนเข้าคลังสาขานี้ (= สาขาของบิล) → dt_branches
18	**terminal_id**	int	✓		FK	เครื่องที่ออก → dt_pos_terminals
19	**pos_id**	text		''		POS ID ของเครื่อง (snapshot)
20	**vat_registered**	bool		true		copy จากบิล — false = ขายตอนไม่จด VAT จึงไม่มี VAT คืน
	created_at, created_by, created_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ created_*					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, cn_no)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, sale_id) → dt_sales (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **FK** (tenant_id, terminal_id) → dt_pos_terminals (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((btrim(reason) <> ''::text))`
- **CHECK** `((total >= (0)::numeric))`
- **index** `dt_credit_notes_branch_idx` btree (tenant_id, branch_id, date)
- **index** `dt_credit_notes_sale_idx` btree (tenant_id, sale_id)
- **index** `dt_credit_notes_shift_idx` btree (tenant_id, shift_id)
- **index** `dt_credit_notes_terminal_idx` btree (tenant_id, terminal_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G dt_credit_note_lines บรรทัดใบลดหนี้

สินค้าที่คืนต่อบรรทัดบิล: จำนวน ยอดคืน VAT ต้นทุน และรับเข้าคลังหรือตัดเสีย

วงจรชีวิตของแถว

- สร้างพร้อมใบลดหนี้
- **append-only**

กฎสำคัญ

- รับเข้าคลังที่ COGS ของการขาย (serial = ทุนเครื่อง); ไม่รับเข้า = ตัดเสีย (serial → damaged); บริการไม่เดินสต็อก

อ้างถึง `dt_credit_notes`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
2	**credit_note_id**	bigint			PK FK	ใบลดหนี้ → dt_credit_notes
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**sale_line_index**	int				บรรทัดของบิลที่คืน (`dt_sale_lines.line_index` ของบิลเดียวกัน)
5	**product_id**	bigint			ref	SKU ⇢ dt_products.id (ไม่มี FK)
6	**sku**	text				snapshot รหัส SKU ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
7	**name**	text				snapshot ชื่อสินค้า ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
8	**short_name**	text				snapshot ชื่อบนใบเสร็จ ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
9	**unit**	text				หน่วยที่ขาย
10	**factor**	int				หน่วยฐานต่อหน่วยที่ขาย
11	**qty**	int				จำนวนที่คืน (หน่วยที่ขาย)
12	**amount**	numeric(14,2)				ยอดคืนของบรรทัด รวม VAT
13	**vat_type**	text				`vat7` · `exempt` ค่าที่ใช้ได้: `vat7` `exempt`
14	**vat**	numeric(14,2)				VAT ใน amount
15	**cogs**	numeric(14,2)				ต้นทุนของหน่วยที่คืน (ส่วนของ COGS ตอนขาย)
16	**serial**	text	✓			serial ที่คืน (= เครื่องที่ลูกค้าถืออยู่ หลังใบเปลี่ยน)
17	**free**	bool				เป็นของแถม
18	**restock**	bool				รับกลับเข้าคลัง (false = ตัดเสีย; บริการ false เสมอ)
	`created_at`, `created_by`, `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (tenant_id, credit_note_id, position)
- **FK** (tenant_id, credit_note_id) → dt_credit_notes (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `(("position" >= 0))`
- **CHECK** `((qty > 0))`
- **CHECK** `((sale_line_index >= 0))`
- **CHECK** `((vat_type = ANY (ARRAY['vat7'::text, 'exempt'::text])))`

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G `dt_credit_note_deductions` ยอดหักของแถมที่เกินสิทธิ์

ของแถมที่ลูกค้าเก็บไว้แต่ของที่เหลือไม่ได้สิทธิ์แล้ว — หักจากยอดคืนที่ราคาขาย ณ วันขาย (`list_price`)

วงจรชีวิตของแถว

- สร้างพร้อมใบลดหนี้
- **append-only**

กฎสำคัญ

- หักครั้งเดียว — ใบลดหนี้ถัดไปของบิลเดียวกันไม่หักซ้ำ

อ้างถึง `dt_credit_notes`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**credit_note_id**	bigint			PK FK	ใบลดหนี้ → dt_credit_notes
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**promotion_id**	bigint			ref	โปรของแถม ⇢ dt_promotions.id (ไม่มี FK)
5	**product_id**	bigint				SKU ของแถม
6	**sku**	text				snapshot รหัส SKU ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
7	**name**	text				snapshot ชื่อสินค้า ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
8	**qty**	int				จำนวนที่หัก (หน่วยฐาน)
9	**unit_price**	numeric(14,2)				ราคาต่อหน่วยที่หัก (ราคาขาย ณ วันขาย)
10	**amount**	numeric(14,2)				ยอดหัก
11	**vat_type**	text				`vat7` · `exempt` ค่าที่ใช้ได้: `vat7` `exempt`
	`created_at` , `created_by` , `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (tenant_id, credit_note_id, position)
- **FK** (tenant_id, credit_note_id) → dt_credit_notes (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `(("position" >= 0))`
- **CHECK** `((qty > 0))`
- **CHECK** `((vat_type = ANY (ARRAY['vat7'::text, 'exempt'::text])))`

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G dt_credit_note_refunds การคืนเงินของใบลดหนี้

คืนเงินแต่ละช่องทาง (รูปแบบเดียวกับ `dt_sale_payments`)

วงจรชีวิตของแถว

- สร้างพร้อมใบลดหนี้
- **append-only**

กฎสำคัญ

- ไม่เกินที่ช่องทางนั้นรับในบิล – ที่คืนไปแล้ว; ห้ามเก็บเลขบัตรเต็ม

อ้างถึง `dt_credit_notes`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**credit_note_id**	bigint			PK FK	ใบลดหนี้ → dt_credit_notes
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**method_id**	bigint			ref	ช่องทางที่คืน ⇢ dt_payment_methods.id (ไม่มี FK)
5	**name**	text				snapshot ชื่อช่องทาง ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
6	**type**	text				ชนิดของช่องทาง (snapshot)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
7	**amount**	numeric(14,2)				ยอดคืน
8	**tendered**	numeric(14,2)				ยอดที่จ่ายจริง (= amount)
9	**reference**	text		''		เลขอ้างอิง
10	**installment_months**	int	✓			เดือนผ่อน (ถ้ามี)
	created_at , created_by , created_by_kind — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ created_*					

- **PK** (tenant_id, credit_note_id, position)
- **FK** (tenant_id, credit_note_id) → dt_credit_notes (tenant_id, id) ON DELETE RESTRICT
- **CHECK** (("position" >= 0))
- **index** dt_credit_note_refunds_method_idx btree (tenant_id, method_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: (tenant_id = app_tenant_id()) · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G dt_exchanges ใบเปลี่ยนสินค้า

เปลี่ยนสินค้าในบิลเป็นตัวใหม่ของ **SKU และหน่วยเดียวกัน** — ราคาเท่าเดิม จึงไม่มีเงิน ไม่มีใบลดหนี้ / ใบกำกับใหม่ มีแต่สต็อกเดิน; บิลเดิมไม่เปลี่ยน

วงจรชีวิตของแถว

- สร้าง: หน้า /sales/:id/exchange (พนักงานภายใน exchange_days , admin ได้ตลอด) — เลข EX-<สาขา>-YYYYMMDD-NNNN
- **append-only ทั้งใบ**

กฎสำคัญ

- ตัวใหม่ต้องมีในคลังก่อน และถูกตัดออกก่อน แล้วตัวเก่าจึงกลับเข้าคลัง (หรือตัดเสีย)
- ทำได้เฉพาะที่สาขาที่ขาย; ไม่ต้องมีกะ

ค่าที่ไม่ได้เก็บ (API คำนวณตอนอ่าน)

- บิลได้ exchangeNos

อ้างถึง dt_branches , dt_sales , dt_shifts , dt_pos_terminals · ถูกอ้างโดย dt_exchange_lines

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**id**	bigint		*identity*	PK	รหัสแถว (สร้างอัตโนมัติ, identity) — id ใน JSON ของ API
2	**tenant_id**	bigint			FK	บริษัทเจ้าของแถว → dt_companies.id — RLS กรองด้วยคอลัมน์นี้; แอปใส่ app_tenant_id() ตอน INSERT → dt_companies
3	**ex_no**	text			UQ	เลขใบเปลี่ยน (ไม่ซ้ำในบริษัท)
4	**sale_id**	bigint			FK	บิลที่เปลี่ยน → dt_sales
5	**order_no**	text				เลขบิล (snapshot)
6	**sale_date**	timestamptz				วันเวลาขายของบิล
7	**date**	timestamptz		now()		วันเวลาที่เปลี่ยน
8	**cashier**	text				ผู้ทำ (ชื่อ)
9	**reason**	text				เหตุผลที่เปลี่ยน
10	**shift_id**	bigint	✓		FK	กะที่เปิดอยู่ของเครื่องตอนนั้น (ถ้ามี) — แสดงในรายงานกะ (ไม่มีเงิน) → dt_shifts
11	**branch_id**	bigint			FK	สาขาที่ทำ = สต็อกเดินที่สาขานี้ → dt_branches
12	**terminal_id**	int	✓		FK	เครื่องที่ทำ → dt_pos_terminals
13	**pos_id**	text		''		POS ID ของเครื่อง (snapshot)

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
	`created_at` , `created_by` , `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (id)
- **UNIQUE** (tenant_id, id)
- **UNIQUE** (tenant_id, ex_no)
- **FK** (tenant_id, branch_id) → dt_branches (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, sale_id) → dt_sales (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id, shift_id) → dt_shifts (tenant_id, id) ON DELETE RESTRICT
- **FK** (tenant_id) → dt_companies (id) ON DELETE RESTRICT
- **FK** (tenant_id, terminal_id) → dt_pos_terminals (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `((btrim(reason) <> ''::text))`
- **index** `dt_exchanges_branch_idx` btree (tenant_id, branch_id)
- **index** `dt_exchanges_sale_idx` btree (tenant_id, sale_id)
- **index** `dt_exchanges_shift_idx` btree (tenant_id, shift_id)
- **index** `dt_exchanges_terminal_idx` btree (tenant_id, terminal_id)

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

G `dt_exchange_lines` บรรทัดใบเปลี่ยน

แต่ละบรรทัดบิลที่เปลี่ยน: serial เก่า / ใหม่ ทุนที่รับกลับ / ตัดออก และรับเข้าคลังหรือตัดเสีย

วงจรชีวิตของแถว

- สร้างพร้อมใบเปลี่ยน
- **append-only**

กฎสำคัญ

- ทุนของบิล (COGS) ไม่เปลี่ยน — ความต่างของทุนอยู่ใน cost_in / cost_out

อ้างถึง `dt_exchanges`

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
1	**tenant_id**	bigint			PK	บริษัทเจ้าของแถว → `dt_companies.id` — RLS กรองด้วยคอลัมน์นี้; แอปใส่ `app_tenant_id()` ตอน INSERT
2	**exchange_id**	bigint			PK FK	ใบเปลี่ยน → dt_exchanges
3	**position**	int			PK	ลำดับของบรรทัดในเอกสาร เริ่ม 0 — API ตอบเรียงตามคอลัมน์นี้
4	**sale_line_index**	int				บรรทัดของบิลที่เปลี่ยน
5	**product_id**	bigint			ref	SKU ⇢ dt_products.id (ไม่มี FK)
6	**sku**	text				snapshot รหัส SKU ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
7	**name**	text				snapshot ชื่อสินค้า ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
8	**short_name**	text				snapshot ชื่อบนใบเสร็จ ณ ตอนออกเอกสาร (ไม่ join master ตอนอ่าน — แก้ master ทีหลังเอกสารไม่เปลี่ยน)
9	**unit**	text				หน่วยที่ขาย
10	**factor**	int				หน่วยฐานต่อหน่วยที่ขาย
11	**qty**	int				จำนวนที่เปลี่ยน (serial = 1)
12	**old_serial**	text	✓			serial ที่ลูกค้านำมาคืน
13	**new_serial**	text	✓			serial ที่ให้ไปแทน

#	คอลัมน์	ชนิด	null	ค่าเริ่มต้น	key	คำอธิบาย
14	**restock**	bool				ตัวเก่ากลับเข้าคลัง (false = ตัดเสีย, serial → damaged)
15	**cost_in**	numeric(14,2)				ทุนของตัวที่รับกลับ (serial = ทุนเครื่อง; ปกติ = ส่วนของ COGS ตอนขาย)
16	**cost_out**	numeric(14,2)				ทุนของตัวที่ให้ไป (serial = ทุนเครื่อง; ปกติ = ทุนเฉลี่ยขณะเปลี่ยน)
	`created_at` , `created_by` , `created_by_kind` — คอลัมน์ stamp มาตรฐาน (trigger ตั้งให้, ดูหัวข้อ 1.4) · ตาราง append-only จึงมีแค่ `created_*`					

- **PK** (tenant_id, exchange_id, position)
- **FK** (tenant_id, exchange_id) → dt_exchanges (tenant_id, id) ON DELETE RESTRICT
- **CHECK** `(("position" >= 0))`
- **CHECK** `((qty > 0))`
- **CHECK** `((sale_line_index >= 0))`

trigger: stamp แบบ append-only (BEFORE INSERT) · RLS + FORCE: `(tenant_id = app_tenant_id())` · สิทธิ์ an22_app **SI** · an22_platform **SIUD**

บทที่ 4 ขั้นตอนธุรกิจ → ตาราง

งานหลักของระบบเขียน / อ่านตารางไหนตามลำดับ — เขียนจาก service ของ an22-api; ทุกขั้นของงานหนึ่งอยู่ใน transaction เดียวกัน (ไม่สำเร็จ = ไม่มีอะไรถูกบันทึก) เว้นแต่บอกไว้

กติกาที่ทุกงานใช้

- ตรวจกฎทั้งหมดก่อน แล้วจึงล็อก / ออกเลข / เขียน (ถือล็อกสั้นที่สุด); งานที่นับทั้งบริษัท (เช่น SKU ซ้ำ บาร์โค้ดซ้ำ เปิดกะ) ล็อกระดับบริษัทด้วย `pg_advisory_xact_lock` ก่อนอ่าน
- สต็อกเปลี่ยนผ่าน `StockLedger` เท่านั้น: ล็อกแถว `dt_branch_stock` (FOR UPDATE) → คิดทุน → update → เขียน `dt_stock_movements` ; เอกสารหลายบรรทัดเรียงตาม productId (กัน deadlock)
- audit บันทึกใน transaction เดียวกับงาน (สำเร็จเท่านั้น) · ทุกงานของร้านได้ `app.tenant_id / app.user_id` ของผู้ใช้ → RLS + stamp

ทำ: เพิ่ม INSERT · แก้ UPDATE (หรือ upsert) · ลบ DELETE · อ่าน SELECT · ล็อก SELECT … FOR UPDATE

สมัครใช้งาน (บริษัทใหม่)

`POST /auth/signup` — ทำด้วย role an22_platform เพราะยังไม่มี tenant; ทั้งหมดใน transaction เดียว

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_login_throttle`	แก้	นับการสมัครต่อ IP (เกิน 10 ครั้งใน 15 นาที → 429) — transaction แยกก่อนเริ่ม
2	`dt_companies`	อ่าน	รหัสบริษัทต้องไม่เคยถูกใช้ (รวมบริษัทที่ลบแล้ว)
3	`dt_companies`	เพิ่ม	บริษัทใหม่ แพ็กเกจที่เลือก ช่วงทดลองใช้ + วันออกใบแจ้งหนี้แรก
4	`dt_users`	เพิ่ม	owner (อีเมลบังคับ, ทุกสาขา)
5	`dt_company_info`	เพิ่ม	ชื่อ เลขภาษี สถานะ VAT + ค่าเริ่มต้นของกติกา
6	`dt_branches`	เพิ่ม	สาขาแรก (สำนักงานใหญ่ 00000)
7	`dt_payment_methods`	เพิ่ม	ช่องทางเงินสด 1 ช่องทาง (ต้องมีเสมอ)
8	`dt_audit_log`	เพิ่ม	`signup`

เมนูของพนักงานไม่มีแถวใน `dt_menu_access` = ค่าเริ่มต้น; เครื่อง POS / หมวด / สินค้าเริ่มว่าง

เข้าสู่ระบบ และจำกัด login ผิด

`POST /auth/login { companyCode, username, password }` — token ที่ api ออกถูกเก็บใน session ของ an22-web (browser ได้แค่ cookie)

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_login_throttle`	แก้	จองสิทธิ์ตรวจรหัสของบัญชีและของ IP (ล็อกอยู่ → 429 ไม่ตรวจรหัส) — transaction สั้นแยก
2	`dt_companies` `dt_users`	อ่าน	หาบริษัทจากรหัส แล้วผู้ใช้จาก username — ไม่เจอก็ยังตรวจกับ hash หลอก (เวลาตอบเท่ากัน)
3	`dt_login_throttle`	แก้	ผิด → นับผิด (ครบเพดานตั้ง locked_until); ถูก → ลบแถวของบัญชี
4	`dt_audit_log`	เพิ่ม	`login` หรือ `login_failed` (บันทึกแม้ request ล้ม)
5	`dt_user_mfa`	แก้	ต้องใช้รหัส Authenticator (E55: `/auth/login-mfa` , ผู้ใช้เปิดใช้แล้ว หรือบริษัทบังคับ) → ยังไม่ออก token: ตอบ challenge (JWT aud=mfa 5 นาที) + secret ใหม่ที่รอยืนยันถ้ายังไม่ผูก; `POST /auth/mfa/verify` ตรวจรหัส ตั้ง `confirmed_at` / `last_step` แล้วจึง audit `login` + ออก token (ผิด = `login_failed` นับในตัวนับเดียวกับรหัสผ่าน)

ขั้น	ตาราง	ทำ	รายละเอียด
6	`dt_billing_invoices`	เพิ่ม	`BillingService.catchUp` ออกใบแจ้งหนี้ที่ถึงรอบ / ระงับบริษัทที่ค้างเกินกำหนด (ดูหัวข้อ billing)
7	`dt_branches` `dt_pos_terminals` `dt_company_info`	อ่าน	สาขาที่เข้าได้ + ข้อมูลของ session (`GET /me` อ่านชุดเดียวกันทุกครั้งที่โหลดหน้า)

ทุก request ของร้านหลัง login: `ShopSessionFilter` อ่าน `dt_users` + `dt_companies` จาก DB ทุกครั้ง (สิทธิ์ที่เปลี่ยนมีผลทันที) แล้วตั้ง `app.tenant_id` / `app.user_id` ให้ทุก transaction

รับสินค้าเข้าคลัง

สินค้าปกติ `PUT /products/:id/stock` (delta > 0) · serial `POST /products/:id/serials` — ที่สาขาใน `X-Branch-Id`

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_branch_stock`	ล็อก	สร้างแถวถ้ายังไม่มี (ON CONFLICT DO NOTHING) แล้ว SELECT ... FOR UPDATE — ล็อก (สาขา, SKU) จน commit
2	`dt_serial_numbers`	เพิ่ม	serial: เครื่องใหม่ in_stock พร้อมทุน — เครื่องที่เคยมี (ขาย / ตัด / อยู่สาขาอื่น) ใช้แถวเดิม (U) เปลี่ยนสถานะ / สาขา / ทุน
3	`dt_branch_stock`	แก้	ปกติ: stock + qty, avg_cost = `movingAverage()` ; serial: นับ / เฉลี่ย serial ที่ in_stock
4	`dt_stock_movements`	เพิ่ม	`receive` + ยอดคงเหลือหลังรายการ (serial: รายการเดียวพร้อม serials ทั้งชุด)

นำเข้าการรับสินค้าจาก Excel = เรียก endpoint เดียวกันทีละแถว / กลุ่ม (ไม่มี endpoint เฉพาะ)

serial ที่อยู่ล่าสุดที่สาขาอื่น: API ตอบ `fromOtherBranch` และหน้าจอแนะนำให้ใช้ใบโอนแทน

ตัดสต็อกออก (admin)

สินค้าปกติ `PUT /products/:id/stock` (delta < 0) · serial `POST /products/:id/serials/remove`

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_branch_stock`	ล็อก	ล็อก (สาขา, SKU)
2	`dt_serial_numbers`	แก้	serial: status → damaged / other / sold + removed_at + หมายเหตุ (ไม่ลบแถว)
3	`dt_branch_stock`	แก้	stock − qty (ไม่พอ → 400 "สต็อกคงเหลือไม่พอ"), ทุนเฉลี่ยคงเดิม
4	`dt_stock_movements`	เพิ่ม	`issue` ที่ทุนเฉลี่ย (serial = ทุนของเครื่อง)
5	`dt_audit_log`	เพิ่ม	`stock_out` / `serial_remove`

โอนสินค้าระหว่างสาขา

สร้าง `POST /transfers` ที่สาขาต้นทาง · รับ `POST /transfers/:id/receive` ที่ปลายทาง · ยกเลิก `POST /transfers/:id/cancel` (admin ต้นทาง)

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_serial_numbers`	อ่าน	**สร้าง** — serial ที่ส่งต้อง in_stock ที่ต้นทาง
2	`dt_doc_counters`	แก้	เลข `TR-<ต้นทาง>-YYYYMMDD-NNNN`
3	`dt_branch_stock`	ล็อก	ล็อก (ต้นทาง, SKU) ทีละ SKU เรียงตาม productId (กัน deadlock)
4	`dt_serial_numbers`	แก้	serial → in_transit (branch_id ยังเป็นต้นทาง)
5	`dt_branch_stock` `dt_stock_movements`	เพิ่ม	ต้นทาง: stock − qty, movement `transfer_out` ที่ทุนเฉลี่ย (serial = ทุนเครื่อง)

ขั้น	ตาราง	ทำ	รายละเอียด
6	`dt_transfers` `dt_transfer_lines`	เพิ่ม	หัวใบ in_transit + บรรทัดพร้อมทุนโอน `unit_cost`
7	`dt_transfers`	ล็อก	**รับ** — ล็อกหัวใบ (รับได้ครั้งเดียว)

8	`dt_branch_stock`
	`dt_stock_movements`
	`dt_serial_numbers`
		แก้	ปลายทาง: `transfer_in` ทุกชิ้นที่ส่งที่ทุนโอน (moving average) / serial → in_stock ที่ปลายทาง แล้วตัดส่วนที่ขาดเป็น `issue` ที่ปลายทาง (serial ที่ขาด → damaged)

9	`dt_transfers` `dt_transfer_lines`	แก้	status received, ผู้รับ, เหตุผลที่ขาด, received_qty / received_serials

10	`dt_transfers`
	`dt_branch_stock`
	`dt_stock_movements`
		แก้	**ยกเลิก** — ของกลับต้นทางที่ทุนเดิม (`transfer_in`), status cancelled + เหตุผล, audit `transfer_cancel`

เปิดกะ

`POST /shifts/open` ที่เครื่องที่ลงทะเบียน (`X-Terminal-Id`)

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_pos_terminals`	อ่าน	เครื่องต้องใช้งานอยู่และอยู่สาขาเดียวกับ `X-Branch-Id`
2	`dt_shifts`	อ่าน	เครื่องนี้ / คนนี้ต้องไม่มีกะเปิดอยู่ (ล็อกระดับบริษัทก่อนตรวจ; partial unique index เป็นตาข่าย)
3	`dt_doc_counters`	แก้	เลข `SH-<สาขา>-<วันเปิด>-NN`
4	`dt_shifts`	เพิ่ม	กะ open + เงินทอนตั้งต้น + snapshot POS ID

ขาย 1 บิล

`POST /sales` (`SaleService.checkout`) — ตรวจ 8 ขั้นก่อน แล้วจึงออกเลข / เดินสต็อก / บันทึก (ถือล็อกสั้นที่สุด)

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_pos_terminals` `dt_shifts`	อ่าน	ตรวจ 1–2: เครื่องลงทะเบียน ใช้งาน อยู่สาขานี้; หน้าจอไม่ถูกล็อก; มีกะเปิดของเครื่องและเป็นของคนขาย (เมื่อ require_shift)

2	`dt_products`
	`dt_branch_stock`
	`dt_sku_prices`
	`dt_promotions`
	`dt_categories`
		อ่าน	ตรวจ 3–4: คิดตะกร้าใหม่ที่ server (`PosPricing` : ราคาวันนี้ + โปร + ส่วนลดพิเศษ) — ไม่เชื่อราคาจากหน้าจอ

3	`dt_serial_numbers`	อ่าน	ตรวจ 5: serial ต้อง in_stock ที่สาขานี้
4	`dt_payment_methods`	อ่าน	ตรวจ 6–8: ยอดตรงกับ `expectedTotal` ที่หน้าจอเห็น (ไม่ตรง → 400 ราคาเปลี่ยน), การชำระถูกกติกา, ข้อมูลผู้ซื้อ (ถ้าขอใบกำกับ)
5	`dt_doc_counters`	แก้	เลข `POS-<สาขา>-<เครื่อง>-YYYYMMDD-NNNN`

6	`dt_branch_stock`
	`dt_serial_numbers`
	`dt_stock_movements`
		แก้	ตัดสต็อกทีละบรรทัดเรียงตาม productId: ล็อก (สาขา, SKU) → `issue` → COGS (serial: เครื่อง → sold; บริการไม่เดินสต็อก)

7	`dt_sales`
	`dt_sale_lines`
	`dt_sale_payments`
		เพิ่ม	หัวบิล paid + บรรทัด (snapshot + COGS) + การชำระ

8	`dt_tax_invoices` `dt_sales`	เพิ่ม	ถ้ามีผู้ซื้อ: ออกใบกำกับเต็มรูป `at_sale` (เลข INV-) แล้วตั้ง `dt_sales.tax_invoice_no`

ขั้น	ตาราง	ทำ	รายละเอียด
9	`dt_audit_log`	เพิ่ม	`manual_discount` เมื่อมีส่วนลดพิเศษ

ใบเตือน "ขายต่ำกว่าทุน" สำหรับคนที่ไม่เห็นทุน: `POST /sales/cost-check` อ่านอย่างเดียว ไม่บันทึก

ยกเลิกบิล (void วันเดียวกัน)

`POST /sales/:id/void` — พนักงานต้องมี admin มาพิมพ์รหัสอนุมัติที่เครื่อง; ทำได้ที่สาขาที่ขาย

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_sales`	ล็อก	ล็อกบิล (SELECT … FOR UPDATE) แล้วตรวจกฎ void (วันเดียวกัน, สถานะ paid, กะ, สาขา)
2	`dt_users`	อ่าน	ตรวจ username / รหัสของ admin ผู้อนุมัติ (พนักงาน)
3	`dt_branch_stock` `dt_serial_numbers` `dt_stock_movements`	แก้	ของกลับเข้าคลังที่ COGS ของบรรทัด (`receive` ; serial → in_stock)
4	`dt_tax_invoices`	แก้	ใบกำกับเต็มรูปที่ใช้ได้ของบิลถูกยกเลิก (`cancelled_at`)
5	`dt_sales`	แก้	status cancelled + voided_at / reason / voided_by / void_approved_by / void_shift_id
6	`dt_audit_log`	เพิ่ม	`sale_void`

ใบกำกับภาษีเต็มรูป (ออกทีหลัง / ยกเลิกแล้วออกใหม่)

`POST /sales/:id/tax-invoice` · ออกใหม่ (admin) `POST /sales/:id/tax-invoice/reissue`

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_sales`	ล็อก	ล็อกบิล — ต้องชำระแล้วและขายตอนจด VAT
2	`dt_doc_counters`	แก้	เลข `INV-<สาขาของบิล>-<วันขาย>-NNNN` (ลงวันที่วันขาย)
3	`dt_tax_invoices`	แก้	ออกใหม่: ใบที่ใช้ได้เดิม → cancelled_at + เหตุผล + replaced_by_no
4	`dt_tax_invoices`	เพิ่ม	ใบใหม่: ผู้ซื้อเข้ารหัส + hash เลขภาษี, at_sale เท่าเดิม, replaces_invoice_no (ถ้าออกใหม่)
5	`dt_sales`	แก้	`tax_invoice_no` = เลขใบที่ใช้ได้
6	`dt_audit_log`	เพิ่ม	`tax_invoice_reissue` (เฉพาะออกใหม่)

ใบลดหนี้ (คืนสินค้าตั้งแต่วันถัดไป)

`POST /sales/:id/credit-notes` (admin) ที่สาขาที่ขาย

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_sales`	ล็อก	ล็อกบิล
2	`dt_credit_notes` `dt_exchanges`	อ่าน	ใบลดหนี้ / ใบเปลี่ยนเดิมของบิล — คิดของที่คืนได้, serial ที่ลูกค้าถืออยู่, ของแถมที่เกินสิทธิ์ (`draftCreditNote()`)
3	`dt_company_info` `dt_shifts`	อ่าน	กติกาบริษัท + กะเปิดของเครื่อง (จ่ายเงินคืนจากลิ้นชักของกะ)
4	`dt_doc_counters`	แก้	เลข `CN-<สาขา>-YYYYMMDD-NNNN`
5	`dt_branch_stock` `dt_serial_numbers` `dt_stock_movements`	แก้	restock: รับเข้าที่ COGS ของการขาย (`receive`); ไม่ restock: serial → damaged, อื่นไม่เข้าคลัง
6	`dt_tax_invoices`	อ่าน	ใบกำกับที่ใช้ได้ตอนนี้ → snapshot เลข + ผู้ซื้อ

ขั้น	ตาราง	ทำ	รายละเอียด
7	`dt_credit_notes` `dt_credit_note_lines` `dt_credit_note_deductions` `dt_credit_note_refunds`	เพิ่ม	ใบลดหนี้ทั้งใบ
8	`dt_audit_log`	เพิ่ม	`credit_note`

ใบเปลี่ยนสินค้า

`POST /sales/:id/exchanges` — พนักงานภายใน `exchange_days` ของวันขาย ไม่ต้องมีกะ

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_sales`	ล็อก	ล็อกบิล
2	`dt_exchanges` `dt_credit_notes`	อ่าน	ของที่เปลี่ยนได้ (หักที่คืน / เปลี่ยนไปแล้ว)
3	`dt_doc_counters`	แก้	เลข `EX-<สาขา>-YYYYMMDD-NNNN`
4	`dt_branch_stock` `dt_serial_numbers` `dt_stock_movements`	แก้	**ตัดตัวใหม่ออกก่อน** (`issue` ; serial ใหม่ → sold) แล้วจึงรับตัวเก่ากลับ (`receive`) หรือตัดเสีย
5	`dt_exchanges` `dt_exchange_lines`	เพิ่ม	ใบเปลี่ยน + บรรทัดพร้อม cost_in / cost_out

ปิดกะ อนุมัติ และปรับปรุงยอด

นับแบบไม่เห็นยอด: `POST /shifts/:id/count-check` (เทียบ — ไม่บันทึก) → `POST /shifts/:id/close` · อนุมัติภายหลัง `POST /shifts/:id/approve` · ปรับยอด `POST /shifts/:id/adjustments` (admin)

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_sales` `dt_credit_notes` `dt_cash_movements` `dt_payment_methods`	อ่าน	ยอดที่คาดต่อช่องทางของกะ (`ShiftService.context` → `expectedByMethod()`)
2	`dt_users`	อ่าน	admin พิมพ์รหัสอนุมัติที่เครื่อง (เมื่อส่วนต่างเกินเพดาน)
3	`dt_shift_counts`	เพิ่ม	ยอดนับต่อช่องทาง + ยอดครั้งแรก + เหตุผลของส่วนต่าง
4	`dt_shifts`	แก้	closed + เวลา / ผู้ปิด + แบงก์ที่นับ + ยอดฝาก + เงินคงลิ้นชัก + สถานะอนุมัติ
5	`dt_audit_log`	เพิ่ม	`shift_close_other` เมื่อ admin ปิดกะของเครื่องอื่นแทน
6	`dt_shifts` `dt_audit_log`	แก้	**อนุมัติ** pending → approved + `shift_approve`
7	`dt_shift_adjustments` `dt_audit_log`	เพิ่ม	**ปรับปรุงยอด** หลังปิด (กะ / ยอดนับเดิมไม่เปลี่ยน) + `shift_adjust`

ระหว่างกะ: เงินเข้า / ออก → `dt_cash_movements` , เปิดลิ้นชัก → `dt_drawer_opens` , ล็อก / ปลดล็อกหน้าจอ → `dt_shifts.locked_at`

พักบิล / เรียกบิลคืน

`GET / POST / DELETE /pos/held-bills` ต่อสาขา + เครื่อง

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_held_bills`	เพิ่ม	พัก: ตะกร้าทั้งก้อนเข้ารหัส
2	`dt_held_bills`	อ่าน	รายการบิลพักของเครื่องนี้ (ถอดรหัสใน api)
3	`dt_held_bills`	ลบ	เรียกคืน / ทิ้ง: ลบแถว — ขายจริงเป็นบิลใหม่ตามหัวข้อ "ขาย 1 บิล"

ค่าบริการ SaaS (job รายวัน)

`BillingJob` 00:05 ทุกวัน (instance เดียวด้วย `pg_try_advisory_xact_lock`) + ตอน login / `GET /me` — role an22_platform

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_companies`	ล็อก	ล็อกแถวบริษัททีละบริษัท
2	`dt_billing_invoices` `dt_platform_doc_counters`	เพิ่ม	ออกใบ `SUB-YYYYMM-NNNN` ทุกรอบที่ถึงแล้วแต่ยังไม่ออก (ราคาของวันออก; ไม่ซ้ำด้วย unique (company, period_start))
3	`dt_companies`	แก้	เลื่อน `next_invoice_on`; ใบค้างเกินกำหนด + ระยะผ่อนผัน → active = false, suspended_reason = overdue
4	`dt_password_reset_tokens` `dt_login_throttle`	ลบ	ล้างแถวที่หมดอายุเกิน 1 วัน

ขอลบบริษัท และ purge

owner ขอ `POST /company/deletion` (พิมพ์รหัสบริษัท + รหัสผ่าน) / ยกเลิก `DELETE` · ครบกำหนดแล้วผู้ดูแลแพลตฟอร์ม purge

ขั้น	ตาราง	ทำ	รายละเอียด
1	`dt_companies`	แก้	**ขอลบ** — ระงับทันที (suspended_reason = deletion), deletion_* + purge_after; เหลือแค่ owner login ได้และเห็นแต่หน้าข้อมูลบริษัท
2	`dt_audit_log`	เพิ่ม	`deletion_request` / `deletion_cancel`
3	`dt_companies`	แก้	**ยกเลิก** — ล้าง deletion_*, เปิดใช้งาน, ตามออกใบแจ้งหนี้ที่ค้าง

4	`dt_sales`
	`dt_products`
	`dt_users`
		ลบ	**purge** (`PlatformCompanyDao.purge`) — ลบทุกตารางที่มี `tenant_id` + `dt_password_reset_tokens` เรียงลูกก่อนแม่ตาม FK (ตารางในช่องนี้เป็นตัวอย่าง)

5	`dt_companies`	แก้	deleted_at + deleted_name, active = false — รหัสบริษัทใช้ซ้ำไม่ได้; ใบแจ้งหนี้ยังอยู่

ก่อนขอลบ owner ส่งออกข้อมูลทั้งหมดได้ที่ `GET /company/export` (JSON ทุกตารางของบริษัท ไม่มีรหัสผ่าน / ตัวนับ)

รูปแบบเลขเอกสาร

ตัวนับอยู่ใน `dt_doc_counters` (prefix = ทุกอย่างก่อนเลขรัน) · `<สาขา>` = `dt_branches.code` ณ ตอนออก · วัน = วันท้องถิ่น · เลขเก่าก่อนมีสาขา / เครื่องไม่เปลี่ยน

เอกสาร	ตาราง.คอลัมน์	รูปแบบ	รันต่อ
บิล POS	`dt_sales.order_no`	`POS-<สาขา>-<เครื่อง>-YYYYMMDD-NNNN`	เครื่อง + วัน
ใบกำกับเต็มรูป	`dt_tax_invoices.invoice_no`	`INV-<สาขาของบิล>-<วันขาย>-NNNN`	สาขาของบิล + วันขาย (แม้ออกทีหลัง)
ใบลดหนี้	`dt_credit_notes.cn_no`	`CN-<สาขา>-YYYYMMDD-NNNN`	สาขา + วันออก
ใบเปลี่ยน	`dt_exchanges.ex_no`	`EX-<สาขา>-YYYYMMDD-NNNN`	สาขา + วัน

เอกสาร	ตาราง.คอลัมน์	รูปแบบ	รันต่อ
กะ	`dt_shifts.shift_no`	`SH-<สาขา>-YYYYMMDD-NN`	สาขา + วันเปิดกะ
ใบโอน	`dt_transfers.transfer_no`	`TR-<สาขาต้นทาง>-YYYYMMDD-NNNN`	สาขาต้นทาง + วัน
ใบแจ้งหนี้ค่าบริการ	`dt_billing_invoices.invoice_no`	`SUB-YYYYMM-NNNN`	เดือน ทั้งแพลตฟอร์ม (`dt_platform_doc_counters`)

บทที่ 5 CRUD matrix

package ของ an22-api (คอลัมน์) ใช้ตาราง (แถว) แบบไหน — สร้างอัตโนมัติจาก SQL ใน `src/main/java` ทุกครั้งที่ build

C = INSERT · R = SELECT (FROM / JOIN) · U = UPDATE หรือ INSERT … ON CONFLICT DO UPDATE · D = DELETE — นับตาม package ของไฟล์ที่มี SQL (dao) ไม่แยก endpoint: service ของ package หนึ่งเรียก dao ของ package อื่นได้ (เช่นการขายอ่านกะผ่าน `ShiftDao` จึงนับที่ `shift`) — ดูงานจริงตามลำดับในบทที่ 4; ช่องที่มี * = SQL ที่ประกอบชื่อตารางตอนรัน (หมายเหตุท้ายตาราง); เมนูของหน้าบ้านเรียก endpoint ของ package เหล่านี้ เช่น เมนู SKU → `product`, ขายหน้าร้าน → `sale` + `shift` + `pos`

ตาราง	auth	platform	billing	company	db	audit	user	branch	masterdata	product	price	promotion	stock	transfer	pos	shift	sale
companies	CR	RU	RU	RU													
platform_admins		R															
billing_invoices	R	RU	CR	R*													
platform_doc_counters			CU														
password_reset_tokens	CRUD	D*	D														
login_throttle	CUD		D														
company_info	CR	RD*		RU													
branches	CR	RD*		R				CRU		R							
users	CRU	RUD*		R			CRUD										
user_branches		D*		R*			CRD										
user_mfa	CRUD	D*		R*			RD										
menu_access		D*		CRU													
pos_terminals	R	RD*		R											CRUD	R	
doc_counters		D*			CU												
audit_log		D*		R*		CR											
categories		D*		R					CRUD								
suppliers		D*		R*					CRUD								
payment_methods	C	D*		R*					CRUD								
products		D*		R					R	CRUD							
product_pack_units		D*		R*						CRD							
product_suppliers		D*		R*					R	CRD							
sku_prices		D*		R						D*	CRUD						
promotions		D*		R*								CRUD					
branch_stock		D*		R*						CRUD*			CRU				
stock_movements		D*		R*									CR				
serial_numbers		D*		R*									CRU				
transfers		D*		R*										CRU			
transfer_lines		D*		R*										CRU			
shifts		D*		R*												CRU	

ตาราง	auth	platform	billing	company	db	audit	user	branch	masterdata	product	price	promotion	stock	transfer	pos	shift	sale
shift_counts		D*		R*												CR	
cash_movements		D*		R*												CR	
drawer_opens		D*		R*												CR	
shift_adjustments		D*		R*												CR	
held_bills		D*		R*											CRD		
sales		D*		R*													CRU
sale_lines		D*		R*													CR
sale_payments		D*		R*													CR
tax_invoices		D*		R*													CRU
credit_notes		D*		R*													CR
credit_note_lines		D*		R*													CR
credit_note_deductions		D*		R*													CR
credit_note_refunds		D*		R*													CR
exchanges		D*		R*													CR
exchange_lines		D*		R*													CR

package — `auth` login / signup / ลืมรหัสผ่าน · `platform` ผู้ดูแลแพลตฟอร์ม · `billing` ค่าบริการ SaaS (job รายวัน) · `company` ตั้งค่าบริษัท / แพ็กเกจ / ส่งออก / ขอลบ · `db` โครงพื้นฐาน (เลขเอกสาร, ผู้ทำ) · `audit` บันทึกการใช้งาน · `user` ผู้ใช้ · `branch` สาขา · `masterdata` หมวดหมู่ / ผู้จำหน่าย / ช่องทางชำระ · `product` SKU + สต็อก (endpoint) · `price` ราคาขาย · `promotion` โปรโมชั่น · `stock` StockLedger / serial · `transfer` ใบโอน · `pos` เครื่อง POS / บิลพัก · `shift` กะ · `sale` บิล / ใบกำกับ / ใบลดหนี้ / ใบเปลี่ยน

* `product` ลบ SKU (หมดสต็อกทุกสาขาแล้ว) ลบแถวลูก / สต็อก 0 / ช่วงราคาด้วย loop ชื่อตาราง (`ProductDao.delete`) — ประวัติ movement / serial / เอกสารอยู่ต่อ

* `company` ส่งออกข้อมูลบริษัท (`GET /company/export`) อ่านทุกตารางที่มี `tenant_id` ที่ an22_app อ่านได้ (ยกเว้นตัวนับเลขเอกสาร) + ใบแจ้งหนี้ (`CompanyExportDao`)

* `platform` purge บริษัทที่ขอลบครบกำหนด ลบทุกตารางที่มี `tenant_id` + ลิงก์ตั้งรหัสผ่าน เรียงลูกก่อนแม่ตาม FK ใน catalog (`PlatformCompanyDao.purge`) — เหลือแถว `dt_companies` (ชื่อ + `deleted_at`) และใบแจ้งหนี้

บทที่ 6 ความปลอดภัยของฐานข้อมูล

แยกสิทธิ์ด้วย role 3 ตัว + Row-Level Security ต่อบริษัท + เข้ารหัสข้อมูลส่วนบุคคลในแอป — ค่าในบทนี้ส่วนหนึ่งอ่านจาก DB จริง

6.1 role

role	ใช้ทำ	ต่อทาง	RLS	ค่าตั้งของ role (จาก DB)
`an22_owner`	เจ้าของ database / ตารางทั้งหมด — Flyway migrate เท่านั้น (แอปไม่ใช้ตอนทำงาน)	PostgreSQL ตรง (5432)	ถูกบังคับด้วย (FORCE)	—
`an22_app`	งานของผู้ใช้ร้านทุก request (`appTx`) — ไม่ใช่เจ้าของตาราง	PgBouncer (6432)	อยู่ใต้ RLS	`statement_timeout=30s` `idle_in_transaction_session_timeout=60s`
`an22_platform`	งานก่อน login (login, signup, ลืมรหัส), ผู้ดูแลแพลตฟอร์ม, job ค่าบริการ, purge (`platformTx`)	PgBouncer (6432)	**BYPASSRLS**	`statement_timeout=30s` `idle_in_transaction_session_timeout=60s`

- สร้างด้วย `db/roles.sql` โดย admin ที่**ไม่ใช่ superuser** (ใช้ได้บน Azure) — รหัสผ่านของ role อยู่ในที่เก็บความลับ (env) ไม่อยู่ใน repo
- schema `public` : ยกเลิกสิทธิ์ CREATE ของ PUBLIC; ให้แค่ USAGE แก่ an22_app / an22_platform — ทั้งสองสร้าง / แก้โครงสร้างไม่ได้
- timeout: query เกิน `statement_timeout` ถูกยกเลิก (api ตอบ 503), transaction ค้างเกิน `idle_in_transaction_session_timeout` ถูกตัด — an22_owner ไม่ตั้ง (migration อาจนาน)

6.2 Row-Level Security (แยกข้อมูลบริษัท)

- ทุกตารางที่มี `tenant_id` : `ENABLE` + `FORCE ROW LEVEL SECURITY` และ policy `tenant_isolation` แบบเดียวกัน: `USING (tenant_id = app_tenant_id()) WITH CHECK (tenant_id = app_tenant_id())` — อ่าน / แก้ / ลบได้เฉพาะแถวของบริษัทใน transaction และเขียนแถวของบริษัทอื่นไม่ได้
- `app_tenant_id()` อ่านค่า `app.tenant_id` ที่ api ตั้งด้วย `set_config(…, true)` (SET LOCAL) **ทุก transaction** จากบริษัทใน token — ไม่ตั้ง / ค่าว่าง = null → **เห็น 0 แถว** (ไม่ error) จึงไม่มีทางหลุดเป็น "เห็นทุกบริษัท"
- FORCE ทำให้แม้เจ้าของตาราง (an22_owner) ก็ถูกบังคับ; มีแค่ an22_platform (BYPASSRLS) ที่ข้ามได้ — ใช้กับงานที่ต้องเห็นหลายบริษัทเท่านั้น
- ตาราง platform: `dt_companies` (policy บน `id`) และ `dt_billing_invoices` (บน `company_id`) เปิด RLS ให้ร้านอ่านของตัวเอง; ตาราง platform อื่น an22_app ไม่มีสิทธิ์เลย
- ชั้นถัดจาก RLS: api ใส่บริษัทเองตอน INSERT, FK รวม `(tenant_id, …)` กันอ้างข้ามบริษัท และ test ของ api ยิงทุก endpoint ที่มี id ด้วย id ของบริษัทอื่น → ต้องได้ 404 (`CrossTenantSweepIT` , `RlsIT`)

6.3 สิทธิ์ต่อตาราง (จาก DB)

S = SELECT · I = INSERT · U = UPDATE · D = DELETE — ตารางเอกสาร / ประวัติไม่มี DELETE (และ append-only ไม่มี UPDATE) แม้มีบั๊กในแอปก็ลบเอกสารไม่ได้; an22_platform มีครบเพื่อ purge

ตาราง	RLS	an22_app	an22_platform	หมายเหตุ
`dt_companies`	ENABLE + FORCE	**S**	**SIUD**	an22_app อ่านแถวของบริษัทตัวเองเท่านั้น (RLS `id`)
`dt_platform_admins`	—	**—**	**SIUD**	platform เท่านั้น
`dt_billing_invoices`	ENABLE + FORCE	**S**	**SIUD**	an22_app อ่านใบของบริษัทตัวเอง (RLS `company_id`)
`dt_platform_doc_counters`	—	**—**	**SIUD**	platform เท่านั้น

ตาราง	RLS	an22_app	an22_platform	หมายเหตุ
dt_password_reset_tokens	—	—	SIUD	platform เท่านั้น
dt_login_throttle	—	—	SIUD	ใช้ก่อนรู้บริษัท — platform เท่านั้น
dt_company_info	ENABLE + FORCE	SIUD	SIUD	
dt_branches	ENABLE + FORCE	SIUD	SIUD	
dt_users	ENABLE + FORCE	SIUD	SIUD	
dt_user_branches	ENABLE + FORCE	SIUD	SIUD	
dt_user_mfa	ENABLE + FORCE	SIUD	SIUD	
dt_menu_access	ENABLE + FORCE	SIUD	SIUD	
dt_pos_terminals	ENABLE + FORCE	SIUD	SIUD	
dt_doc_counters	ENABLE + FORCE	SIU	SIUD	ไม่มี DELETE
dt_audit_log	ENABLE + FORCE	SI	SIUD	append-only
dt_categories	ENABLE + FORCE	SIUD	SIUD	
dt_suppliers	ENABLE + FORCE	SIUD	SIUD	
dt_payment_methods	ENABLE + FORCE	SIUD	SIUD	
dt_products	ENABLE + FORCE	SIUD	SIUD	
dt_product_pack_units	ENABLE + FORCE	SIUD	SIUD	
dt_product_suppliers	ENABLE + FORCE	SIUD	SIUD	
dt_sku_prices	ENABLE + FORCE	SIUD	SIUD	
dt_promotions	ENABLE + FORCE	SIUD	SIUD	
dt_branch_stock	ENABLE + FORCE	SIUD	SIUD	
dt_stock_movements	ENABLE + FORCE	SI	SIUD	append-only
dt_serial_numbers	ENABLE + FORCE	SIUD	SIUD	
dt_transfers	ENABLE + FORCE	SIU	SIUD	ไม่มี DELETE
dt_transfer_lines	ENABLE + FORCE	SIU	SIUD	ไม่มี DELETE
dt_shifts	ENABLE + FORCE	SIU	SIUD	ไม่มี DELETE
dt_shift_counts	ENABLE + FORCE	SI	SIUD	append-only
dt_cash_movements	ENABLE + FORCE	SI	SIUD	append-only
dt_drawer_opens	ENABLE + FORCE	SI	SIUD	append-only
dt_shift_adjustments	ENABLE + FORCE	SI	SIUD	append-only
dt_held_bills	ENABLE + FORCE	SIUD	SIUD	ของชั่วคราว ลบได้
dt_sales	ENABLE + FORCE	SIU	SIUD	ไม่มี DELETE (แก้ได้แค่ void / เลขใบกำกับ)
dt_sale_lines	ENABLE + FORCE	SI	SIUD	append-only
dt_sale_payments	ENABLE + FORCE	SI	SIUD	append-only
dt_tax_invoices	ENABLE + FORCE	SIU	SIUD	ไม่มี DELETE (แก้ได้แค่ยกเลิก)
dt_credit_notes	ENABLE + FORCE	SI	SIUD	append-only
dt_credit_note_lines	ENABLE + FORCE	SI	SIUD	append-only
dt_credit_note_deductions	ENABLE + FORCE	SI	SIUD	append-only
dt_credit_note_refunds	ENABLE + FORCE	SI	SIUD	append-only
dt_exchanges	ENABLE + FORCE	SI	SIUD	append-only
dt_exchange_lines	ENABLE + FORCE	SI	SIUD	append-only

6.4 ข้อมูลส่วนบุคคลและความลับ

ข้อมูล	ตาราง.คอลัมน์	วิธีเก็บ
ผู้ซื้อของใบกำกับ / ใบลดหนี้ (ชื่อ เลขผู้เสียภาษี / passport ที่อยู่)	`dt_tax_invoices.buyer_enc`, `dt_credit_notes.buyer_enc`	เข้ารหัสทั้งก้อนในแอป AES-256-GCM รูปแบบ `[version][nonce][ciphertext+tag]`, AAD = `<ตาราง>.<ช่อง>:<tenant_id>` (ย้ายข้ามแถว / บริษัทแล้วถอดไม่ได้)
บิลที่พักไว้ (อาจมีผู้ซื้อ)	`dt_held_bills.payload_enc`	เข้ารหัสแบบเดียวกัน
ค้นผู้ซื้อด้วยเลขภาษี	`dt_tax_invoices.buyer_tax_id_hash`	HMAC ด้วยกุญแจ `PII_HASH_KEY` + tenant — ค้นได้โดยไม่ถอดรหัส ไม่ย้อนกลับเป็นเลขได้
รหัสผ่าน	`dt_users.password_hash`, `dt_platform_admins.password_hash`	BCrypt — ไม่ส่งออก API / export
ลิงก์ตั้งรหัสผ่าน	`dt_password_reset_tokens.token_hash`	เก็บ hash ของ token, ใช้ครั้งเดียว หมดอายุ 1 ชม.
ตัวนับ login ผิด	`dt_login_throttle.key_hash`	HMAC ของ (บริษัท + username) / IP — ไม่เก็บ username / IP ตรง ๆ
บัตรเครดิต	`dt_sale_payments.reference`	**ห้ามเก็บเลขบัตรเต็ม** (api ปฏิเสธก่อนถึง DB) — เก็บแค่เลขอนุมัติ / 4 หลักท้าย

> **กุญแจเข้ารหัส**
> กุญแจ (`PII_KEYS`, `PII_HASH_KEY`) อยู่ใน env ของ api ไม่อยู่ใน DB / repo — **ต้องสำรองแยกจาก backup ของ DB**: กุญแจหาย = ข้อมูลผู้ซื้อถอดไม่ได้ถาวร; หมุนกุญแจ = เพิ่ม version ใหม่ใน `PII_KEYS` (เขียนด้วยตัวใหม่สุด อ่านได้ทุกตัว) ไม่ต้องเขียนข้อมูลเก่าใหม่ทันที

- อีเมล / ชื่อผู้ใช้ / ชื่อผู้จำหน่าย เก็บเป็นข้อความปกติ (ต้องค้น / แสดง) — คุ้มครองด้วย RLS + สิทธิ์ของ role
- ต้นทุน / กำไร: DB เก็บครบ แต่ api ใส่ null ในทุกช่องทุนของ response ให้ role user เมื่อบริษัทปิด `staff_see_cost` (ซ่อนที่ api ไม่ใช่ที่ DB)
- log ของ api ไม่บันทึกค่าจาก DB ที่ถอดรหัสแล้ว / รหัสผ่าน / token และ mask คีย์ที่อ่อนไหวใน body

ภาคผนวก

ก MongoDB session ของ an22-web

an22-web เก็บ session ของ login (token ของ api ที่ browser ไม่เคยเห็น) ตาม env `SESSION_STORE` : `memory` (เครื่องเดียว หายเมื่อ restart) หรือ `mongodb` (หลาย instance / อยู่รอด restart — ต้องตั้ง `MONGODB_URI`) — ไม่เกี่ยวกับ PostgreSQL

รายการ	ค่า
database / collection	ตาม `MONGODB_URI` (เช่น `an22`) / `sessions` (connect-mongo)
เอกสาร 1 ชิ้น	`_id` = session id (ค่าใน cookie `an22.sid` / `__Host-an22.sid` ที่เซ็นแล้ว) · `expires` = หมดอายุ · `session` = ข้อมูล session **แบบเข้ารหัส** ด้วย `SESSION_SECRET` (Web Crypto)
ข้อมูลใน session (หลังถอด)	`shopToken` token ของผู้ใช้ร้าน · `platformToken` token ของผู้ดูแลแพลตฟอร์ม (ช่องแยก) · `createdAt` เวลา login · ค่า cookie ของ express-session
หมดอายุ	ไม่ใช้งานเกิน `SESSION_IDLE_HOURS` (ค่าเริ่ม 12) · นานสุด `SESSION_MAX_DAYS` (ค่าเริ่ม 7) นับจาก login — TTL index `{ expires: 1 }` `expireAfterSeconds: 0` ให้ MongoDB ลบเอง
เมื่อถอดรหัสไม่ได้	(เปลี่ยน `SESSION_SECRET` / ข้อมูลเสีย) ถือว่าไม่มี session → ผู้ใช้ login ใหม่
logout	an22-web ลบเอกสาร session ทิ้ง (api เป็น JWT stateless ไม่มี logout — token ที่หลุดจาก session ใช้ไม่ได้เพราะ browser ไม่เคยถือ)

ข ประวัติ migration

ไฟล์ใน `an22-api/db/migration` — `afterMigrate.sql` (callback) รันหลัง migrate ทุกครั้ง: ให้ an22_app อ่าน `flyway_schema_history` ได้ (โหมด validate); ข้อมูลตัวอย่าง `db/seed` ใช้ใน dev / test เท่านั้น ไม่อยู่ในเอกสารนี้

version	ไฟล์	เนื้อหา
V1	`V1__base.sql`	function กลาง (`app_tenant_id()` , `set_row_stamp()`), ตาราง platform, บริษัท / สาขา / ผู้ใช้ / เมนู, audit log, ตัวนับเลขเอกสาร, RLS + grant ชุดแรก
V2	`V2__auth.sql`	RLS ของ `dt_companies` / `dt_billing_invoices` ให้ร้านอ่านของตัวเอง + ตัวนับ login ผิด
V3	`V3__master_data.sql`	ข้อมูลหลัก: หมวดหมู่, ผู้จำหน่าย, ช่องทางชำระเงิน, เครื่อง POS
V4	`V4__products_stock.sql`	SKU, หน่วยแพ็ก, ผู้จำหน่ายของ SKU, สต็อกต่อสาขา, บัตรสต็อก, serial
V5	`V5__prices_promotions.sql`	ราคาขายตามช่วงวัน (extension btree_gist + EXCLUDE) และโปรโมชั่น
V6	`V6__transfers.sql`	ใบโอนสินค้าระหว่างสาขา
V7	`V7__shifts_held_bills.sql`	กะ + ยอดนับ + เงินเข้าออก + เปิดลิ้นชัก + ปรับปรุงยอด, บิลที่พักไว้
V8	`V8__sales_tax_invoices.sql`	บิลขาย + บรรทัด + การชำระ, ใบกำกับภาษีเต็มรูป
V9	`V9__credit_notes_exchanges.sql`	ใบลดหนี้ (+ บรรทัด / ยอดหัก / คืนเงิน), ใบเปลี่ยนสินค้า (+ บรรทัด)
V10	`V10__mfa.sql`	mfa

ค การสร้างเอกสารนี้ใหม่

- คำสั่ง `docs/db-manual/build.sh` (ใน repo an22-api): เปิด PostgreSQL ชั่วคราวใน docker → รัน `roles.sql` + Flyway migrate ของ `db/migration` → อ่าน catalog (`catalog.sql`) → สร้าง HTML + ER (Graphviz) → PDF ด้วย Chrome

headless — ไม่แตะ DB อื่น, container ถูกลบเมื่อจบ

- เพิ่ม / แก้ตาราง: เขียน migration ใหม่ → เพิ่มคำอธิบายใน `descriptions.mjs` (ตาราง, คอลัมน์, กลุ่ม, ref ที่ไม่มี FK, ขั้นตอนธุรกิจที่เกี่ยว) → `build.sh --check` ต้องผ่าน (ขาด / เกิน → exit 1) → `build.sh`
- ต้องมี docker (image `postgres:18.6`, `flyway/flyway:12.4.0`), Node 24, google-chrome, ฟอนต์ Noto Sans Thai

ง คำศัพท์

tenant	บริษัทลูกค้าของ SaaS 1 ราย = 1 แถวใน `dt_companies`; ทุกแถวของร้านมี `tenant_id`
RLS	Row-Level Security ของ PostgreSQL — กรองแถวที่ query เห็นตาม policy
สาขา / คลัง	1 สาขา = 1 คลังสินค้า = 1 สถานประกอบการบนเอกสารภาษี
หน่วยฐาน / หน่วยแพ็ก	สต็อกนับด้วยหน่วยฐาน (เช่น ชิ้น); หน่วยแพ็ก (กล่อง = 4 ชิ้น) ใช้ขาย / รับได้ — สต็อกเดิน qty × factor
serial	หมายเลขเครื่อง 1 เครื่อง = 1 แถว; SKU คุม serial มีสต็อก = จำนวน serial ที่อยู่ในคลัง
ทุนมาตรฐาน / ทุนเฉลี่ย	ทุนมาตรฐาน (`dt_products.cost`) เป็นค่าเริ่ม / ประมาณการ; ทุนจริงคือทุนเฉลี่ยเคลื่อนที่ต่อสาขา (`dt_branch_stock.avg_cost`) หรือทุนของเครื่อง (serial)
COGS	ต้นทุนขาย ไม่รวม VAT บันทึกในบรรทัดบิลตอนขาย
บัตรสต็อก / movement	รายการเดินของสินค้าพร้อมยอดคงเหลือหลังรายการ (`dt_stock_movements`)
void	ยกเลิกบิลวันเดียวกัน — ของกลับคลัง บิลเปลี่ยนสถานะเป็น cancelled
ใบลดหนี้	คืนสินค้าตั้งแต่วันถัดจากวันขาย — คืนเงิน + กลับรายการ VAT; บิลเดิมไม่เปลี่ยน
ใบเปลี่ยน	เปลี่ยนเป็นตัวใหม่ของ SKU เดียวกัน ไม่มีเงิน มีแต่สต็อกเดิน
ใบกำกับอย่างย่อ / เต็มรูป	ใบเสร็จ POS ของร้านจด VAT เป็นใบกำกับอย่างย่อ; เต็มรูปออกตามคำขอพร้อมข้อมูลผู้ซื้อ
POS ID	หมายเลขเครื่อง POS ที่กรมสรรพากรอนุมัติ พิมพ์บนใบเสร็จ
กะ / X / Z report	กะ = ช่วงที่แคชเชียร์ 1 คนรับผิดชอบลิ้นชัก 1 ใบ; X = รายงานระหว่างกะ, Z = รายงานปิดกะ
blind count	ปิดกะโดยแคชเชียร์นับเงินโดยไม่เห็นยอดที่ระบบคาด
snapshot	ค่าที่ copy ลงเอกสาร ณ ตอนออก ไม่อ่านจาก master ทีหลัง
append-only	ตารางที่เพิ่มแถวได้อย่างเดียว ไม่แก้ / ไม่ลบ
BFF	Backend for Frontend — an22-web ถือ session แทน browser